IMPLEMENTATION AND RESOURCE ANALYSIS OF FPGA-BASED MULTIPHASE MOTOR CONTROL ARCHITECTURES

by

Caleb Alexander Binfet

A thesis submitted in partial fulfillment
of the requirements for the degree

of

Master of Science

in

Engineering

MONTANA STATE UNIVERSITY
Bozeman, Montana

May 2026

©COPYRIGHT

by

Caleb Alexander Binfet

2026

All Rights Reserved

DEDICATION

I dedicate this to my father for his love and support and for showing me what determination looks like.

To my mother for always encouraging my curiosity and desire to learn.

To Camille, Tristan, Russell and Christine for their support and for making me feel like family.

ACKNOWLEDGEMENTS

I would first like to thank Dr. Edens for being my research advisor and bringing me in to this fantastic research group as an undergraduate and continuing to mentor me as I completed my graduate degree. I would also like to thank Dr. Miller and the rest of the SE3 lab for supporting me and working with me. Their support and this research opportunity has allowed me to grow as engineer in ways I never thought possible. Thank you Los Alamos National Laboratory for making both my undergraduate and graduate research possible.

I'm incredibly thankful to Dr. Snider for his mentorship and support. His genuine interest in my success was evident the first time we spoke about my research and he has continued to be an incredible source of support and knowledge ever since. He will always say his bias lies in support of math and FPGAs but I believe his bias is toward supporting his students and I will always be grateful for that.

Thank you to my parents, as well as Camille and her family, for their continued support throughout my journey of higher education. I would not be the student, engineer, and researcher I am without their unwavering support and guidance.

I would also like to recognize my research partners Drew and Kyle. It has been a pleasure to be part of such a fantastic team and I am excited to see how they continue our work. I would like to thank Nat for allowing me to work in his lab and for keeping me sane while we battled with the FPGAs. In addition, navigating the struggles to finish this degree with my friend Sam has made the process much more pleasant.

TABLE OF CONTENTS

ACRONYMS xi

1. INTRODUCTION AND BACKGROUND 1

Machining 2
Computer Numerical Control Machining Tools 2
Lathe 3
Milling Machine 3
Electric Motors 4
Permanent Magnet Direct Current Motor (PMDC) 5
Hybrid Stepper Motor 8
Brushless Direct Current Motor 10
Feedback Controls 13
Proportional Integral Derivative Controller 14
Cascaded PID Control 15
State-space Control 16
Field-oriented Control 19
Digital Control Systems 21
Central Processing Units (CPU) 22
Microcontrollers (MCU) 22
Field-Programmable Gate Arrays 24
Digital Signal Processing Slices 24
Block Random Access Memory 25
Programmable Logic Blocks 25
System-on-Chip FPGAs 25
System-on-Module FPGAs 25

2. RASPBERRY PI MOTOR CONTROL IMPLEMENTATION 27

Control Plant 27
Mechanical Hardware 28
Load Cell Mounting and force sensing 29
Electrical Hardware 31
PCB layers and power distribution 31
Motor and Motor Drivers 36
Current Sensing 37
Load Cell Integration 39
Communication 43
PCB Issues 44
Feedback Control Design 47

TABLE OF CONTENTS – CONTINUED

Digital Control 48
Implementation Conclusion 49

3. FPGA MOTOR CONTROL IMPLEMENTATION 51

Control Plant 51
Electrical Hardware 52
Encoder Counter 52
PWM Generation 54
Single Channel ADC 55
ADC Communication 56
Motor Angle Calibration 59
Feedback Control Design 60
Simulink Model 60
Position PID controller 62
Speed PID controller 64
Current PID Controller and FOC blocks 64
Digital Control 65
Component Wrapper 66
Register Control Values 67
DE10-Nano Development Board 67
Quartus Prime software and top level file 70
Platform Designer 70
Implementation Conclusion 71
Issues and Future Improvements 72

4. CONTROLLER ARCHITECTURE AND FPGA RESOURCE UTILIZATION 74

Active Disturbance Rejection Control (ADRC) 74
Simulink Models 76
First Order ADRC Model 76
Second Order ADRC Model 78
Resource Utilization 78
Implementation Conclusion 81
Future Improvements 81

5. CONCLUSION 83

Future Work 84

TABLE OF CONTENTS – CONTINUED

REFERENCES CITED 87

APPENDIX: FPGA Controller Source Code 90

Top-Level Wrapper 91
Encoder Interface 102
Zeroable Encoder Counter 102
Encoder Counter 104
ADC Interface 106
PWM Generation 109
Full Bridge Single Output Driver 109
PWM Controller 111
Electrical Angle Calibration 112

LIST OF TABLES

Table Page

1. Table 1 Common electric motor types, typical drive methods, and applications. 5
2. Table 2 Variables and parameters used in the PMDC motor model. 7
3. Table 3 Variables and parameters used in the hybrid stepper motor model 10
4. Table 4 Variables and parameters used in the BLDC motor model 12
5. Table 5 Sources of Non-Deterministic Latency in CPU-Based Systems (adapted from [7, 16]). 23
6. Table 6 Avalon-MM register map for `Stepper_FOC_Control_hps` 68
7. Table 7 FPGA Resources of DE10-Nano (Cyclone V SE) 69
8. Table 8 Simulink HDL Resource Comparison of Controller Architectures 79
9. Table 9 Estimated Controller Resource Utilization Relative to KR260 FPGA Resources 80

LIST OF FIGURES

Figure Page

1. Figure 1 Diagram of basic lathe components performing a turning between centers operation (adapted from [18]).....3
2. Figure 2 Diagram of a common milling operation (adapted from [19]. 4
3. Figure 3 DC Motor Electro-mechanical Model with inertial load (adapted from [15]). 6
4. Figure 4 Equivalent Stepper motor phase A and B stator circuits.[13] 9
5. Figure 5 Brushless direct current motor equivalent circuit model.[13] 11
6. Figure 6 Proportional-integral-derivative controller block diagram (adapted from [20]). 15
7. Figure 7 Cascaded PID control system diagram illustrating inner and outer feedback loops (adapted from [21])..... 16
8. Figure 8 State-space feedback controller diagram reproduced from [3] 19
9. Figure 9 Image of the Raspberry Pi prototype mechanical system. 28
10. Figure 10 Pocket NC diagram with the machine coordinate system. 29
11. Figure 11 Three dimensional rendering of the top side of the PCB taken from Altium Designer. 32
12. Figure 12 Three dimensional rendering of the bottom side of the PCB taken from Altium Designer. 33
13. Figure 13 A section view of the PCB with layer descriptions. 34
14. Figure 14 Image of the Custom PCB power plane layout in Altium Designer with labeled supply voltages. 35
15. Figure 15 Circuit Diagram of the 3.3 Volt regulator for the PCB. 36
16. Figure 16 Circuit Diagram of the negative voltage generator integrated circuit. 36

LIST OF FIGURES – CONTINUED

Figure Page

17. Figure 17 Shunt resistor and amplifier circuit diagram. 39

18. Figure 18 Circuit diagram of the ADC used to measure the motor current..... 39

19. Figure 19 Load cell Wheatstone bridge circuit diagram supplied by Transducer Techniques..... 40

20. Figure 20 Circuit diagram of the instrumentation amplifier used with the load cell..... 41

21. Figure 21 Circuit diagram of the second stage amplifier for the load cell. 42

22. Figure 22 Circuit diagram of of the gain selection circuit for the second gain stage of the load cell amplification circuit..... 43

23. Figure 23 Circuit diagram of the I2C level shifting IC and its connections..... 44

24. Figure 24 Shunt amplifier board layout with highlighted trace that was cut..... 46

25. Figure 25 Diagram of the cascaded PID field oriented system implemented on the Raspberry Pi. 47

26. Figure 26 Image depicting the 90 degree phase shifted signal produced by the quadrature encoder with phase labels (reproduced from [14]). 53

27. Figure 27 Connection diagram for the 3-wire no busy indicator mode configuration of the MAX11162 ADC (reproduced from [2]). 57

28. Figure 28 Critical Timing diagram for SPI communication with the MAX11162 ADC (reproduced from [2]). 57

29. Figure 29 Simulink controller block with blocks modeling PCB and fabric hardware as well as the stepper motor control plant model 60

30. Figure 30 Internal blocks to the stepper motor feedback controller including PID blocks, coordinate transform blocks and other required computation blocks. 62

LIST OF FIGURES – CONTINUED

Figure Page

31. Figure 31 Position PID data type settings in the Simulink block. 63

32. Figure 32 Image of the DE10-Nano development board with connections and key components labeled. [10] 69

33. Figure 33 Quartus Platform designer system connections. 70

34. Figure 34 Cascaded ADRC and PID based Simulink stepper motor controller model. 77

35. Figure 35 ADRC first order Simulink motor rotational speed controller model. 77

36. Figure 36 Second order ADRC Simulink model. 78

ACRONYMS

ADC	Analog-to-Digital Converter
ADRC	Active Disturbance Rejection Control
BLDC	Brushless Direct Current
BRAM	Block Random Access Memory
CNC	Computer Numerical Control
CPU	Central Processing Unit
DSP	Digital Signal Processing
ESO	Extended State Observer
FPGA	Field-Programmable Gate Array
FOC	Field-Oriented Control
HPS	Hard Processor System
MCU	Microcontroller Unit
PCB	Printed Circuit Board
PID	Proportional–Integral–Derivative
PWM	Pulse Width Modulation
SoC	System on Chip
SoM	System on Module
SPI	Serial Peripheral Interface

ABSTRACT

Small-scale computer numerical control (CNC) machines often lack the rigidity needed to achieve high precision, thus limiting part quality in comparison to larger CNC machines. Combining advanced feedback control methods with additional sensor feedback is a potential solution to improve quality with smaller CNC machines. This thesis explores the utility of using advanced feedback control methods such as field-oriented control (FOC), cascaded proportional integral derivative (PID) control, and active disturbance rejection with modern digital hardware as a potential framework to close this performance gap. Two multiphase motor control systems are examined. The first system is a central processing unit (CPU)-based system integrated with a custom printed circuit board (PCB). The second uses a Field Programmable Gate Array (FPGA). The FPGA-based system provides an important advantage over the CPU-based system, which is low deterministic computational latency. This thesis also examines the utilization of FPGA resources in multiple control architectures. Increasing parallelization of the feedback control algorithm results in increased FPGA resource utilization, but reduces time delay for computation. Integrating FPGA-based motor control into small-scale CNC machining provides a promising pathway to implement advanced control methods to improve small-scale CNC machining precision.

1

INTRODUCTION AND BACKGROUND

It is very customary for high precision subtractive manufacturing computer numerical controlled (CNC) machines such as mills and lathes to be quite large and take up significant floor space in manufacturing facilities. Making these machines smaller by applying modern and advanced control methods could make smaller subtractive manufacturing CNC machines capable of producing higher quality products. This desire to produce highly accurate smaller manufacturing machines has led to the need for more data to be collected during the machining process and the addition of various sensors to smaller machines with the end goal of making smaller, compliant machines have similar or better performance to larger machines. The typical CNC machines utilize large cast masses to increase rigidity which in turn reduces vibrations and flexion of the machine in order to produce highly precise parts [6]. Smaller machines do not have the luxury of inherent rigidity, so integrating additional sensor feedback and advanced control techniques is a potential solution to overcome the shortcomings of the smaller machines. In addition, improving the capabilities of machines with smaller footprints would allow for more machines to fit in a given area on a production floor, improving throughput.

The most common types of machining tools are mills and lathes, which makes them the best targets for potential improvements. Small scale CNC mills and lathes often have stepper motors or brushless direct current (DC) motors as their motion actuators. Advanced controls methods are targeted at these motors specifically because they are commonly the actuation motors for small CNC machines. It would be particularly advantageous to implement a control scheme known as field-oriented control (FOC) because it simplifies motor current sensing, which in turn makes it easier to relate motor current to motor torque. The

torque generated by the motor directly affects how hard the cutting tip of a mill or lathe push on the machining tools working part, affecting the precision of the final part. The application of FOC will allow for a direct correlation between multiphase motor current and cutting pressure which then could be used to improve machine precision and error detection. FOC combined with advanced feedback control methods, such as state-space control could provide a significant improvement to typical control methods for small CNC mills but does require continuous high-speed computation to perform well. The requirement for high-speed, complex, and continuous computation that is performed with low predictable latency during operation of the machine tool, makes this a good application for FPGAs.

Machining

Machining is a general term used to describe a subtractive manufacturing method in which cutting tools are used to remove material from a large piece of stock, known as a workpiece, until it reaches the desired final form. Lathes and mills are the most common machining platforms and will be described in depth in the following sections along with a brief discussion of computer numerical control (CNC) machining tools.

Computer Numerical Control Machining Tools

Computer numerical control machining tools or CNC machines, have become very common and the primary target to be improved by advanced control and improved sensing. The primary difference between CNC machine tools and manual machine tools is that CNC machines utilize motors and a computer system to follow pre-determined tool paths to create the desired part, whereas a machine has an operator that is responsible for moving the cutting tool along the desired path. The paths taken by a CNC machine is determined by a computer file which contains a list of position targets, speed targets, and various other information required to direct the machining tool as needed.

Lathe

In routine lathe operations the workpiece to be cut is rotated at a high-speed in the lathes spindle and a cutting tool is commonly used in line with the rotational axis or orthogonal to the rotational axis. In-line operations are often used for drilling or tapping operations and orthogonal operations, as shown in Figure 1, are used for external shaping. Lathes are normally used when there is some form of axial symmetry as they excel at creating axially symmetric geometry. The pressure applied by the cutting tool is critical to the precision and accuracy of the final part in a lathe.

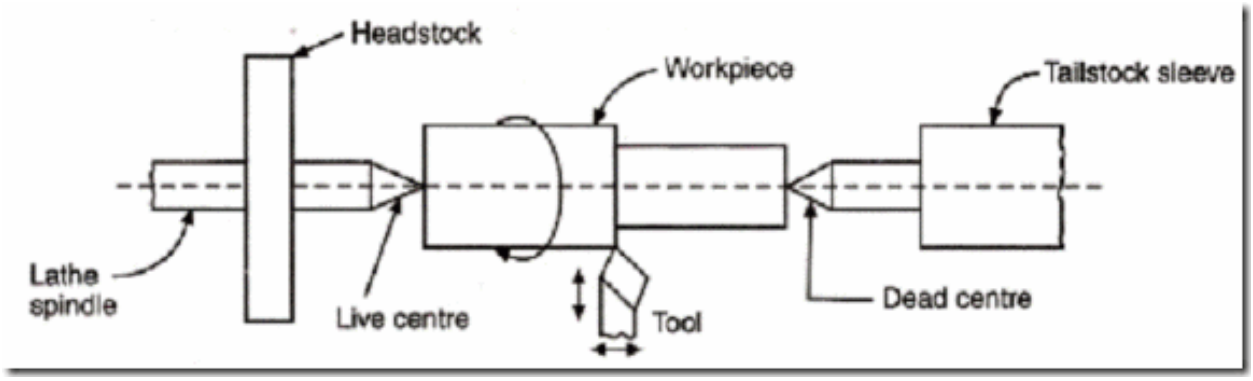


Figure 1: Diagram of basic lathe components performing a turning between centers operation (adapted from [18]).

Milling Machine

Milling machines or mills are a subtractive manufacturing machine where the workpiece is fixed in place, then the workpiece is moved to contact the cutting tool while rotating at a high-speed, or the tool is moved into contact with the stationary workpiece. A diagram of a milling operation with a common cutting tool known as an end mill is shown in Figure 2. Mills can operate with many different types of cutting tools and thus can produce very complex final part geometries. Similar to the lathe, the pressure applied by the cutting tool during operation has a significant effect on the final part's surface finish and geometry. Many CNC mills have started to include additional axis beyond the traditional 3 axis of travel.

These additional axis are known as a fourth and fifth axis and allow for rotation of the workpiece. Rotation of the workpiece can occur during the cutting operation to create more complex geometry or between individual cutting operations to reduce the time required to manually reposition the workpiece in the mill. The increased utilization of additional axis is another motivating factor to increase precision and sensor integration as issues caused by positional errors can be a significant problem.

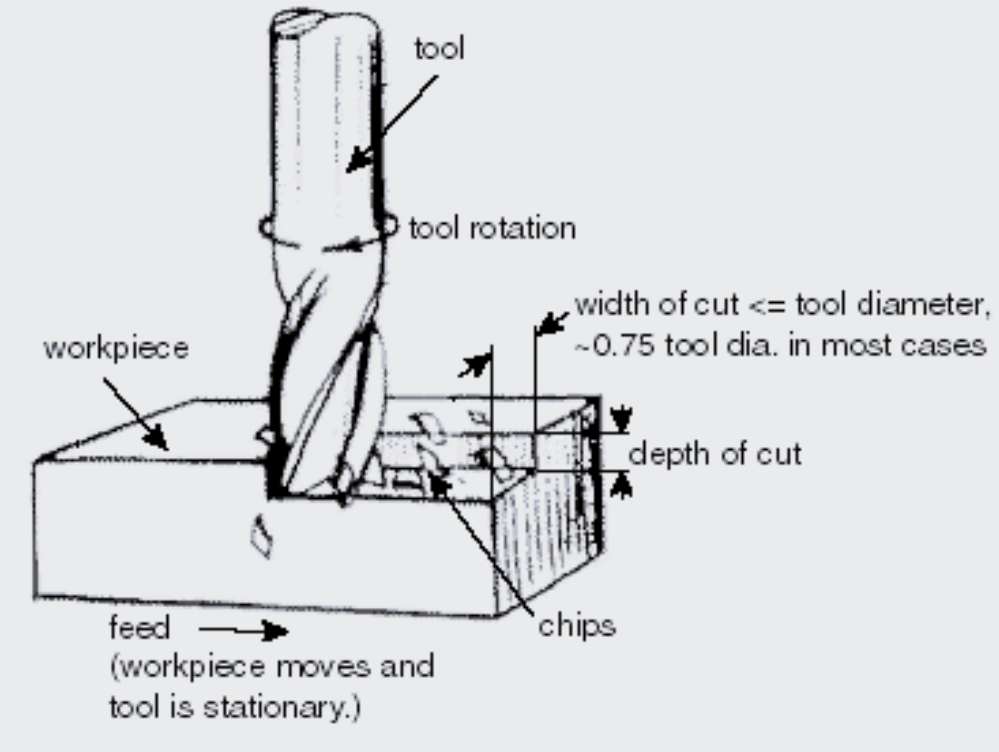


Figure 2: Diagram of a common milling operation (adapted from [19].

Electric Motors

An electrical motor is an electromechanical system that transforms electrical energy into mechanical energy. These motors are also some of the most common plants discussed and studied in feedback control theory. Most electrical motors manipulate a magnetic field generated by flowing current through a coil of wire to generate rotation of a motor shaft.

There are many different kinds of motors, but the three that are most relevant and will be discussed are the direct current (DC) motor, the hybrid stepper motor, and the brushless direct current (BLDC) motor. Table 1 shows common motors and their typical applications.

Motor type	**Supply / drive**	**Common applications**
Brushed DC (PMDC)	DC voltage (H-bridge for reversal)	Toys, small actuators, automotive accessories, hobby robots
Brushless DC (BLDC)	3-phase inverter (trapezoidal commutation)	Drones, fans, e-bikes, appliances, pumps
Permanent Magnet Synchronous Motor (PMSM)	3-phase inverter (often FOC)	EV traction, servos, industrial automation
AC Induction (Asynchronous)	AC mains or VFD (variable frequency drive)	Industrial pumps/fans, conveyors, compressors
Stepper (Hybrid)	Stepper driver (current controlled phases)	3D printers, CNC, camera sliders, small position stages
Switched Reluctance (SRM)	Specialized inverter + current control	High-speed tools, SoMe EV/industrial drives, harsh environments
Single-Phase Induction (PSC / Capacitor-start/run)	AC mains + capacitor (SoMetimes relay)	HVAC blowers, pumps, compressors
DC Shunt / Separately Excited (wound-field)	DC armature + DC field supply	Legacy industrial drives, labs/test stands

Table 1: Common electric motor types, typical drive methods, and applications.

Permanent Magnet Direct Current Motor (PMDC)

The permanent magnet DC motor is one of the most common plants to be controlled in control theory as they have relatively simple dynamics [15]. The PMDC motor is also common in feedback control theory because it is a good application for the PID controller.

Figure 3 shows a circuit that models the electrical qualities of the DC motor. The voltage supply on the left of Figure 3 is the applied voltage that generates the current through the motor windings. The motor winding is modeled as a resistor in series with an inductor and a second voltage supply that opposes the applied voltage. The motor's winding

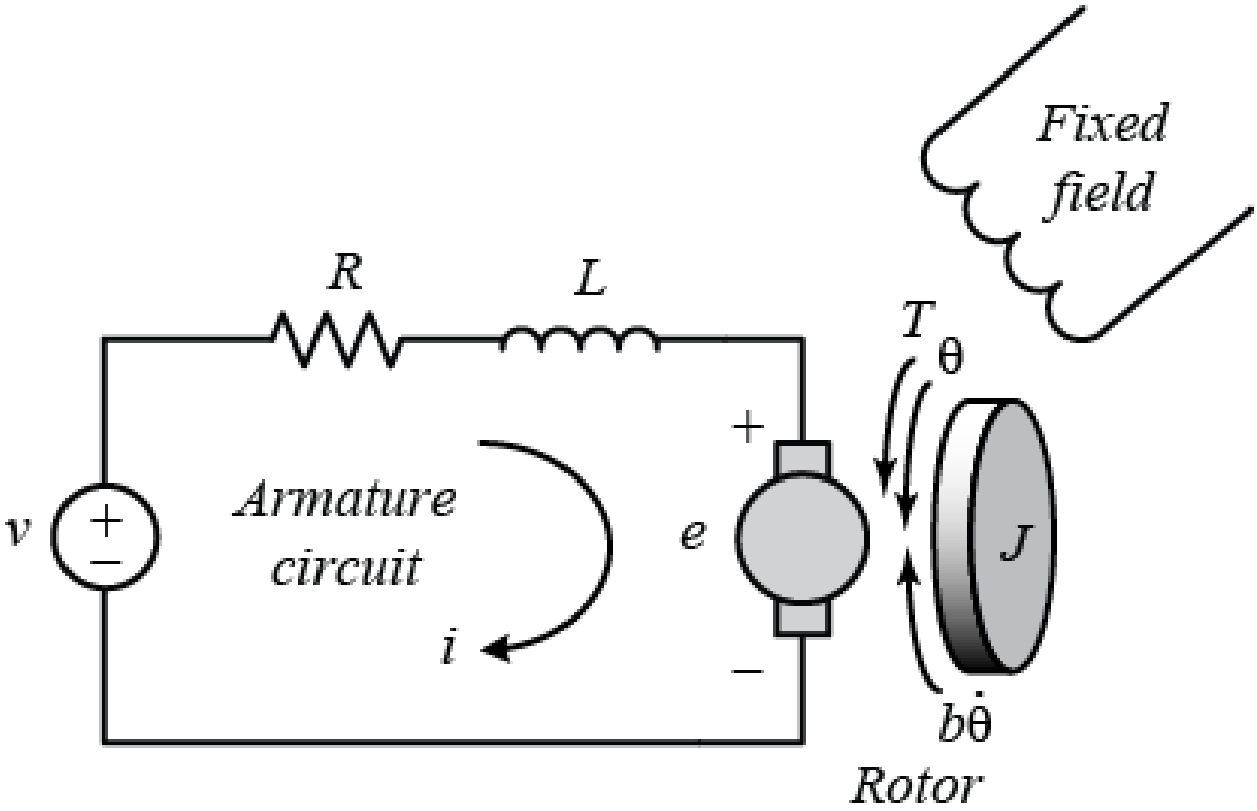


Figure 3: DC Motor Electro-mechanical Model with inertial load (adapted from [15]).

resistance, winding inductance, and motor constant must be known to fully characterize the motor electrically. In addition, to accurately model the dynamics of the motor, the rotor's rotational moment of inertia and the damping constant must also be known. The following equation describes the relationship between the electrical characteristics of the motor and the torque generated by the DC motor.

$$T = k_t * i \tag{1}$$

$$e = k_e * \dot{\theta} \tag{2}$$

Symbol	Description	Units
T	Motor torque	N·m
k_t	Torque constant	N·m/A
k_e	Back EMF constant	V·s/rad
i	Armature current	A
$\dot{i}$	First derivative of armature current	A/s
V	Applied armature voltage	V
θ	Rotor angular position	rad
$\dot{\theta}$	Rotor angular velocity	rad/s
$\ddot{\theta}$	Rotor angular acceleration	rad/s^2
R	Armature resistance	Ω
L	Armature inductance	H
J	Rotor moment of inertia	kg·m^2
b	Viscous damping coefficient	N·m·s/rad

Table 2: Variables and parameters used in the PMDC motor model.

Equation 2 shows how the angular speed of the motor is related to the back electromotive force (EMF) of the motor. The back EMF is represented as a voltage generated that opposes the driving voltage of the motor reducing the current flowing through the motor in proportion to the motor's angular speed.

A state space representation of the DC motor dynamics is shown in equation 3.

$$\frac{d}{dt}\begin{bmatrix}\theta\\ \dot{\theta}\\ i\end{bmatrix} = \begin{bmatrix}\dot{\theta}\\ \ddot{\theta}\\ \dot{i}\end{bmatrix} = \begin{bmatrix}0 & 1 & 0\\ 0 & -\frac{b}{J} & \frac{K}{J}\\ 0 & -\frac{K}{J} & -\frac{R}{J}\end{bmatrix}\begin{bmatrix}\theta\\ \dot{\theta}\\ i\end{bmatrix} + \begin{bmatrix}0\\ 0\\ \frac{1}{L}\end{bmatrix} V \tag{3}$$

This state-space representation of the DC motor treats the input as the motor voltage and has the ability to select either the motors angular position or angular speed as the parameter to be controlled. Table 2 contains a description of all of the DC motor related variables as well as their typical units.

Hybrid Stepper Motor

The hybrid stepper motor is a hybrid of the permanent magnet stepper motor and the variable reluctance stepper motor. The hybrid has a permanent magnet rotor like the permanent magnet motor but includes a variable reluctance stator. The permanent magnet allows the motor to generate high amounts of torque while the stator from the variable reluctance motor makes position and speed control easier to manage [9].

Hybrid stepper motors are very commonly used in open-loop configurations, meaning that their position is not measured while they are being controlled. This is because hybrid stepper motors are designed to make small, highly repeatable movements known as steps. These motors are commonly used in small scale mills and 3D printers because they can be controlled in an open loop configuration which reduces the cost and complexity of the mill or 3D printer.

The electrical circuits that represent the hybrid stepper motor are almost identical to that of the DC motor shown in Figure 3. The key difference is that the hybrid stepper motor is a 2-phase system and thus has two circuits which are shown in Figure 4. The other key difference is that the back EMF depicted in the circuits is directly related to the motors angular position as well as its angular speed. The voltage equations for each phase of the hybrid stepper motor are adapted from established models in the literature [13].

$$v_a = Ri_a + L\frac{di_a}{dt} - K_e\omega_m Sin(N_r\theta_m) \tag{4}$$

$$v_b = Ri_b + L\frac{di_b}{dt} - K_e\omega_m Sin(N_r\theta_m) \tag{5}$$

Just as before, R, L, and K_e are the stator circuit resistance, inductance, and motor

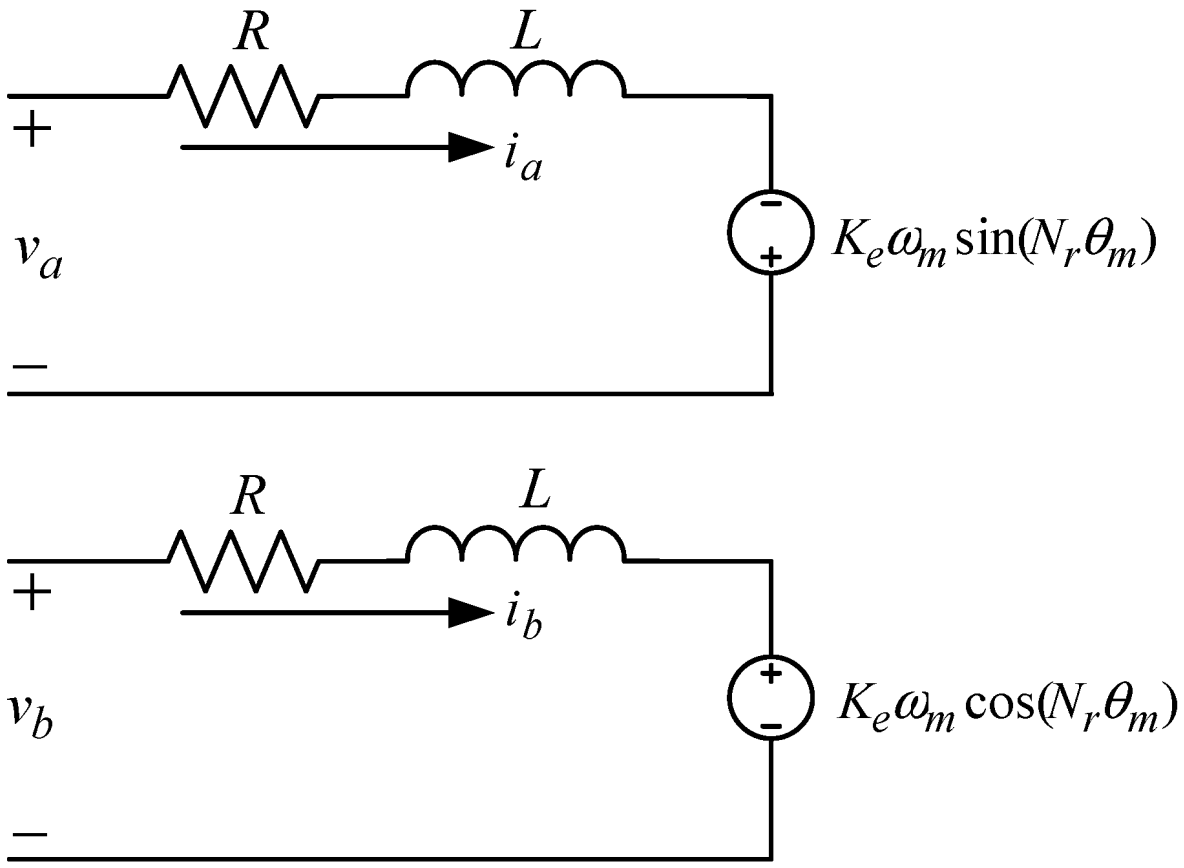


Figure 4: Equivalent Stepper motor phase A and B stator circuits.[13]

back EMF constant. The constant N_r is the number of rotor teeth in the motor and relates the motors mechanical angular position to the electrical angular position. T_c is the motors cogging torque which is a result of a magnetic interaction between the permanent magnets on the rotor and the stator teeth [13].

$$T_e = K_m[-i_a \sin(N_r\theta_m) + i_b \cos(N_r\theta_m)] - F_c \sin(4N_r\theta_m) \tag{6}$$

Equation 6 relates the motor current per phase, the cogging torque constant, and the angular position to the torque generated by the stepper motor. Table 3 contains a description

Symbol	Description	Units
v_a	Phase A stator voltage	V
v_b	Phase B stator voltage	V
i_a	Phase A current	A
i_b	Phase B current	A
R	Stator phase resistance	Ω
L	Stator phase inductance	H
K_e	Back EMF constant	V·s/rad
ω_m	Mechanical angular velocity	rad/s
θ_m	Mechanical rotor position	rad
N_r	Number of rotor teeth	integer
T_e	Electromagnetic torque	N·m
K_m	Motor torque constant	N·m/A
F_c	Cogging torque constant	N·m

Table 3: Variables and parameters used in the hybrid stepper motor model

of the stepper motor related variables and their common units.

Brushless Direct Current Motor

The brushless direct current motor (BLDC) is very commonly controlled with field oriented control (FOC) . The BLDC is a three-phase system which does increase the computational complexity required to control it but does offer SoMe advantages over the DC motor discussed above. The BLDC motor does not require a physical connection between the power input and the rotating armature. This connection is commonly accomplished in DC motors through brushes which are essentially a sliding piece of conductive material that makes direct contact with the armature and is stationary relative to the rotating armature. The lack of brushes is why they are named as such and is also the reason they are usually more efficient than DC motors. The lack of brushes results in reduced friction when rotating relative to the DC motor which is advantageous because not only does it increase efficiency, but it reduces the need for maintenance and allows BLDC motors to have much higher rotational speeds than their DC counterparts in most cases. The lack of brushes also means that the BLDC motors produce far less electromagnetic interference then DC motors. Figure

5 shows the equivalent circuit for a wye wound BLDC motor.

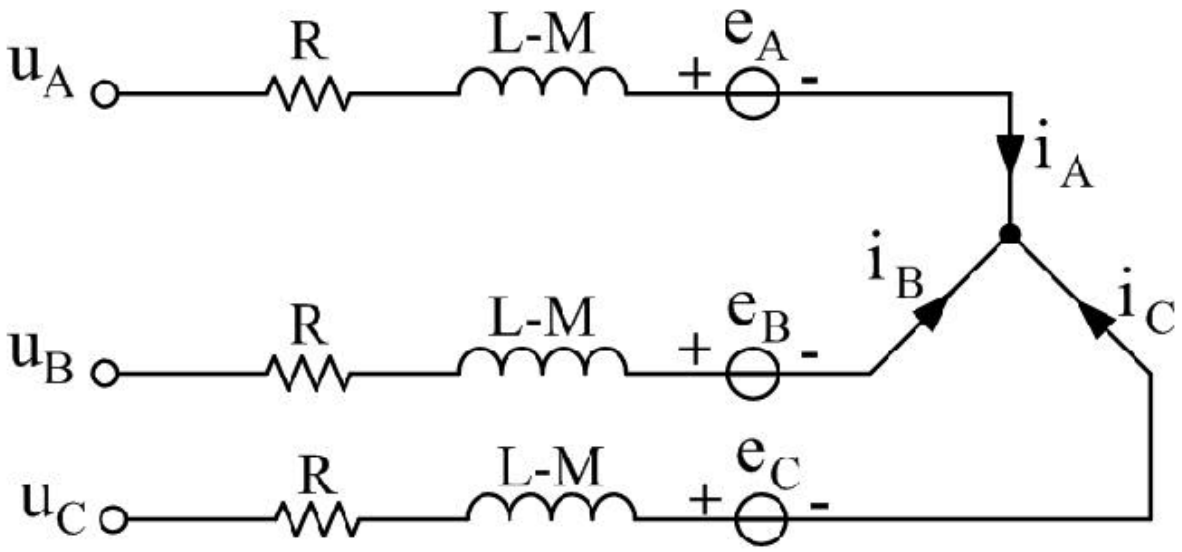


Figure 5: Brushless direct current motor equivalent circuit model.[13]

As can be seen in Figure 5 the brushless direct current motor is a 3-phase wye connected system. These phases are referred to as phases A, B, and C, and each has a current associated with it. The standard convention for positive current flow is into the neutral point or center of the wye. Each phase has an associated resistor, inductor and voltage source that represents the back electromotive force generated by the rotation of the motor. The following equations describe the electrical and mechanical dynamics of the BLDC motor and are based on standard three-phase permanent magnet machine models [11, 12].

$$\mathbf{L} = \begin{bmatrix} L_s & M & M \\ M & L_s & M \\ M & M & L_s \end{bmatrix} \tag{7}$$

Symbol	Description	Units
v_a, v_b, v_c	Phase A, B, and C stator voltages	V
i_a, i_b, i_c	Phase A, B, and C stator currents	A
R_s	Stator phase resistance	Ω
L_s	Stator self inductance	H
M	Mutual inductance between phases	H
e_a, e_b, e_c	Back electromotive force of each phase	V
ω_m	Mechanical angular velocity	rad/s
ω_e	Electrical angular velocity	rad/s
θ_m	Mechanical rotor position	rad
θ_e	Electrical rotor position	rad
p	Number of pole pairs	–
λ_f	Permanent magnet flux linkage	Wb
τ_e	Electromagnetic torque	N·m

Table 4: Variables and parameters used in the BLDC motor model

$$\begin{bmatrix} v_a \\ v_b \\ v_c \end{bmatrix} = R_s \begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix} + \mathbf{L} \frac{d}{dt} \begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix} + \begin{bmatrix} e_a \\ e_b \\ e_c \end{bmatrix} \tag{8}$$

$$v_a = R_s i_a + L_s \dot{i}_a + M(\dot{i}_b + \dot{i}_c) + e_a \tag{9}$$

$$v_b = R_s i_b + L_s \dot{i}_b + M(\dot{i}_a + \dot{i}_c) + e_b \tag{10}$$

$$v_c = R_s i_c + L_s \dot{i}_c + M(\dot{i}_a + \dot{i}_b) + e_c \tag{11}$$

$$\begin{bmatrix} e_a \\ e_b \\ e_c \end{bmatrix} = \omega_e \lambda_f \begin{bmatrix} f_a(\theta_e) \\ f_b(\theta_e) \\ f_c(\theta_e) \end{bmatrix} \tag{12}$$

$$\omega_e = p\,\omega_m, \qquad \dot{\theta}_e = \omega_e \tag{13}$$

$$f_a(\theta_e) = \sin\theta_e, \quad f_b(\theta_e) = \sin\left(\theta_e - \frac{2\pi}{3}\right), \quad f_c(\theta_e) = \sin\left(\theta_e + \frac{2\pi}{3}\right) \tag{14}$$

$$\tau_e = p\,\lambda_f\Big(f_a(\theta_e)i_a + f_b(\theta_e)i_b + f_c(\theta_e)i_c\Big) \tag{15}$$

Equations 7-11 describe how the voltages of each phase change depending on inputs to the motor. The voltages are a function of the phase's resistance, inductance, current and back EMF as well as the current flowing in the other phases. Equation 13 describes how the electrical angular speed and the mechanical angular speed are related by the number of poles possessed by the motor. Equation 14 describes how each phase of the motor is shifted by $\frac{2\pi}{3}$ relative to each other and shows that standard convention is to choose for phase A to be aligned. Finally Equation 15 provides insight into how much torque is generated by the motor and is dependent on the motor electrical position, current in each phase, number of motor poles and a motor constant. Table 4 contains a description of the common variables and constants for a BLDC motor with typical units for both.

Feedback Controls

Control systems fall into two categories: open-loop and closed-loop. Open loop systems don't have feedback and closed loop systems do have feedback from the system being controlled also known as the plant. Feedback is real-time information about the state of

the plant that is being controlled. In the case of a heater for a house, a feedback signal may be the temperature of the room, or for a motor, a feedback signal could be the rotational speed of motor's rotor.

Two common closed-loop feedback control methods are classical control methods and state-space methods. Classical control methods involve analyzing the system being controlled in the frequency domain and applying changes to the input to move the poles of the system to more desirable locations. State-space involves analyzing the system using the differential equations that govern the system's dynamics and then applying linear algebra methods to directly place the poles of the system in any location achieving the desired system dynamics.

Proportional Integral Derivative Controller

The proportional-integral-derivative controller (PID) controller is a very common control method because of its ability to stabilize many different plants. The PID controller applies a scaler value known as a gain to the signal fed back from the plant as well as to the integral and derivative of that signal, hence the name of the controller. The inclusion of the derivative smooths the systems response and the integral This gives the PID controller the ability to drive second order plants to zero steady state error. The following equations are the time domain and the Laplace domain equations for a PID controller.

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\, d\tau \; + K_d \frac{de(t)}{dt} \tag{16}$$

$$L(s) = \frac{K_d s^2 + K_p s + K_i}{s} \tag{17}$$

As is the case with many controllers, the output is a function of the feedback error for

the system, the current state of the system, and time. Figure 6 shows the block diagram for a PID controller where w is the set point or target of the control system and the difference between w and the actual state of the system is fed into the controller. This difference is commonly referred to as the error signal.

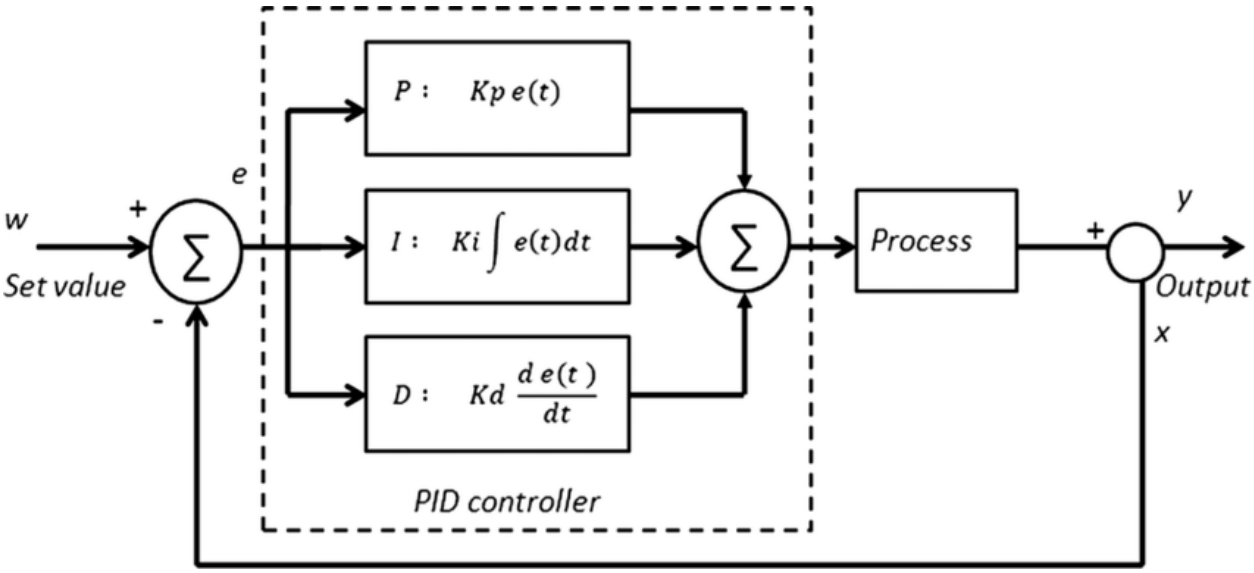


Figure 6: Proportional-integral-derivative controller block diagram (adapted from [20]).

Cascaded PID Control Cascaded PID control is a control architecture in which multiple PID controllers are used in a cascaded manner. This means that at least one PID controller's output is used to set the target for a different PID controller. An example of this architecture is shown in Figure 7. In the cascaded PID architecture the system is usually broken in to inner and outer loops which can be seen in Figure 7. Both the inner and outer loop utilize feedback directly from the system they are controlling but the outer loop is responsible for setting the target of the inner loop. The cascaded PID architecture offers greater stability when a disturbance is introduced because ordinarily the inner loop has higher gains allowing it to react and counteract disturbances faster. This architecture is common in motor control as well as heating systems.

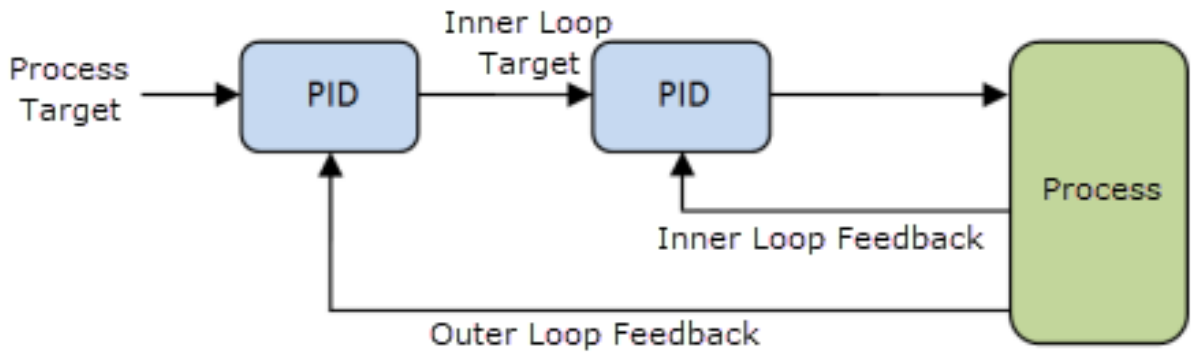


Figure 7: Cascaded PID control system diagram illustrating inner and outer feedback loops (adapted from [21]).

State-space Control

State-space control is a method that involves taking the differential equations that describe the dynamics of the system and placing them in matrices and performing matrix operations to design a controller for the system. The general form of the system is made to fit the standard format shown in Equation 18a which is equivalent to the expanded form shown in Equation 18b.

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) \tag{18a}$$

$$\begin{bmatrix} \dot{x}_1(t) \\ \dot{x}_2(t) \\ \vdots \\ \dot{x}_n(t) \end{bmatrix} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{bmatrix} + \begin{bmatrix} b_{11} & \cdots & b_{1m} \\ \vdots & \ddots & \vdots \\ b_{n1} & \cdots & b_{nm} \end{bmatrix} \begin{bmatrix} u_1(t) \\ u_2(t) \\ \vdots \\ u_m(t) \end{bmatrix} \tag{18b}$$

Where $\mathbf{x}(t) \in \mathbb{R}^n$ is the state vector containing the n state variables that fully describe

the system dynamics, and $\mathbf{u}(t) \in \mathbb{R}^m$ is the input vector containing the m control inputs to the system. The matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ defines the internal dynamics of the system, while $\mathbf{B} \in \mathbb{R}^{n \times m}$ describes how the input vector influences the state evolution. The parameter n represents the number of states in the system, and m represents the number of independent inputs applied to the system.

$$\mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) \tag{19a}$$

$$\begin{bmatrix} y_1(t) \\ y_2(t) \\ \vdots \\ y_p(t) \end{bmatrix} = \begin{bmatrix} c_{11} & \cdots & c_{1n} \\ \vdots & \ddots & \vdots \\ c_{p1} & \cdots & c_{pn} \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{bmatrix} \tag{19b}$$

Where $\mathbf{y}(t) \in \mathbb{R}^p$ is the output vector containing the p measurable outputs of the system, and $\mathbf{C} \in \mathbb{R}^{p \times n}$ maps the internal state vector to the system outputs. The parameter p represents the number of outputs of the system.

In this form $\mathbf{x}$ is a vector containing the state variables of the system. State variables are the characteristics of the dynamic system that must be known to fully describe the state of the system. $\mathbf{A}$ in equation 18a is a matrix that describes the dynamics of the system and B is a matrix that describes how the input, $u(t)$, affects the next state of the system. Equivalent Equations 19a and 19b show how the output of the system, $\mathbf{y}$ is affected. Often systems are single input, single output systems and $\mathbf{C}$ can be simplified to a row vector with a single scaler. This form of modeling is very useful when trying to control linear systems.

State space modeling in this manner has a distinct advantage over classical PID control because it allows for easy analysis of the system to determine its stability, controllability,

and observability [5]. A stable system will return to a steady state eventually whereas a unstable system will continue to change. Controlability is a measure of whether a system can be driven to a specific state by adjusting the input to the system. An observable system can have its state variables reconstructed from the available outputs of the system.

$$\mathbf{u}(t) = -\mathbf{K}\mathbf{x}(t) \tag{20a}$$

$$\begin{bmatrix} u_1(t) \\ u_2(t) \\ \vdots \\ u_m(t) \end{bmatrix} = - \begin{bmatrix} k_{11} & \cdots & k_{1n} \\ \vdots & \ddots & \vdots \\ k_{m1} & \cdots & k_{mn} \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{bmatrix} \tag{20b}$$

State space control also has the distinct advantage of being able to place the poles of the system in any location, allowing for very precise control over the dynamics of the system. In state-space, with the system modeled in the standard form from equation 18a, it is possible to perform simple linear algebra operations on $\mathbf{A}$ to design a controller that results in a closed loop system with precisely placed poles. This controller is represented by a matrix known as $\mathbf{K}$ and is incorporated into the system using the typical control law shown in equivalent Equations 20a and 20b.

Figure 8 is a diagram of a control plant with an integrated state-space feedback controller. The diagram illustrates what each matrix in the state-space representation of the system represents the system as well as how the controller K interacts with the system to control the output, y. As is common with state-space the input signal on the left of the diagram is r which is referred to as the reference signal and is similar to the target signal discussed in the PID section. This signal contains the target state for the system which is

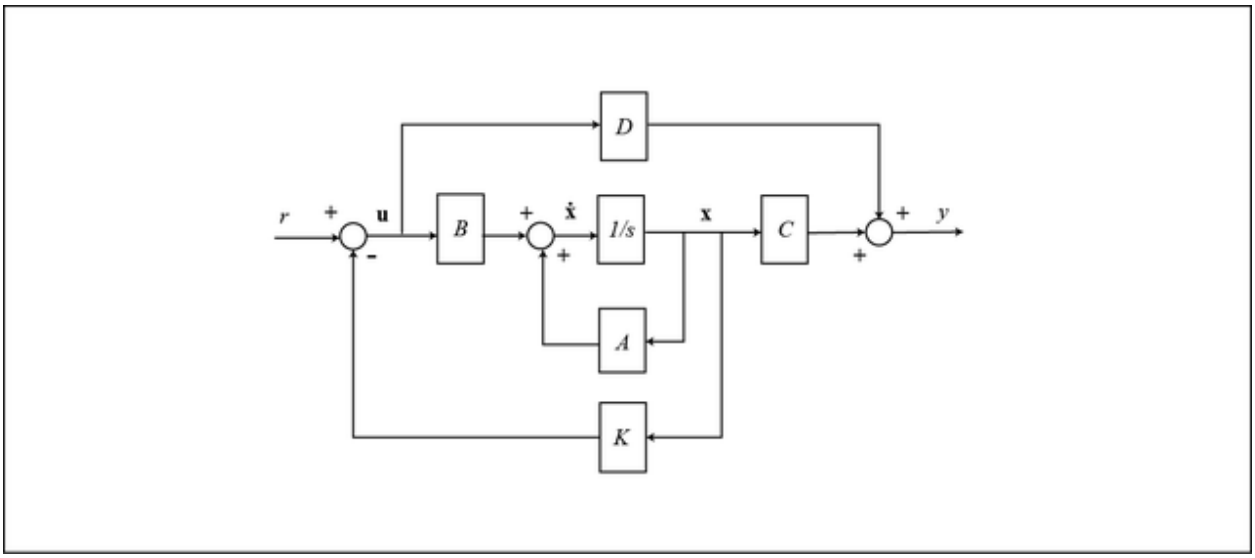


Figure 8: State-space feedback controller diagram reproduced from [3]

then subtracted from the current state which creates the previously discussed signal u. The $\frac{1}{s}$ section is simply included to show that the plant being controlled inherently outputs the integral of the $\dot{x}$ vector which results in the x vector that is utilized for feedback and effects the next state.

Field-oriented Control Field-oriented control (FOC) also known as vector control, is a common method of improving controllability of multiphase motors. FOC is method of applying multiple coordinate transforms to a motor's position and current to make delivering current to the motor more efficient [4]. The coordinate transforms used to implement field-oriented control are the Clarke transform and the Park transform. The Clarke transform takes in current measurements for a 3-phase 120 degree shifted system and transforms them in to an equivalent 2-phase 90 degree shifted system. The following equation is the matrix

representation of the Clarke transform applied to the current of a 3-phase system.

$$\begin{bmatrix} i_\alpha \\ i_\beta \\ i_0 \end{bmatrix} = \frac{2}{3} \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \end{bmatrix} \begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix} \tag{21}$$

The resulting vector from equation 21 contains the alpha and beta currents of the 2-phase system along with a zero signal. These signals are 90-degree shifted sinusoids that are then passed in to the Park transform. The linear algebra representation of the Park transform is shown in the following equation.

$$\begin{bmatrix} i_d \\ i_q \\ i_0 \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} i_\alpha \\ i_\beta \\ i_0 \end{bmatrix} \tag{22}$$

In equation 22, θ represents the electrical angle of the motor which is defined as the position of the rotating magnetic field. The resulting currents, i_d and i_q, are known as the direct and quadrature current. These currents are also commonly referred to as current vectors because they always point in the same direction relative to the electrical angle θ.

The resultant quadrature current is responsible for generating the force that spins the motor shaft. It is directly proportional to the torque generated by the motor in a similar manner to the current in the direct current motor discussed in the following section. The direct current is wasted current that is responsible for pulling the motor's rotor toward the motor wall. It is common practice to minimize the direct current and modulate the quadrature current to meet a target torque or rotational speed.

Once the controller has determined how to modify the quadrature current being output

by the system, it must then go through the inverse transforms to return to the original 3-phase domain. Equation 23 is the inverse park transform and is responsible for transforming the quadrature and direct current back into the 2-phase alpha-beta domain. Then equation 24 is used to transform back to the standard 3-phase domain.

$$\begin{bmatrix} i_\alpha \\ i_\beta \\ i_0 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} i_d \\ i_q \\ i_0 \end{bmatrix}. \tag{23}$$

$$\begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{1}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix} \tag{24}$$

Digital Control Systems

Digital control is a method of using digital systems to directly affect the state of another separate system through the use of digital computation and sensor feedback. Digital feedback controls are heavily dependent upon the amount of time the digital control system takes to interpret the feedback signal and compute the next output. The primary types of digital systems that perform digital control are microcontrollers, central processing units (CPUs), and field programmable gate arrays (FPGAs). These digital systems have drastically different architectures which affects their performance, complexity, and associated development time. The metrics that are typically used to determine when one system should be used over another are the systems digital precision, computation time, development time, and cost. In order to perform feedback control effectively, the time required to complete the control loop calculation must be considered, as it determines the controller period and

directly affects the control output.

Central Processing Units (CPU)

CPUs have high-speed computation ability and with multiple core architectures, which are now common, they can parallelize tasks. CPU based systems can be very effective at feedback control but do have slightly longer development times compared to a system such as a microcontrollers but do generally offer better computation performance. CPU based control systems can struggle with timing since the completion of computational tasks are not deterministic, which means that the time required to run a computation may differ each time. This is caused by the operating system interrupting the computation and the inconsistent time delay caused by the CPU having to store and retrieve data from memory.

There are multiple causes of the non-deterministic latency in CPU based systems. The system must buffer data in dynamic random access memory (DRAM), which does not have deterministic latency because the time it takes to read/write data from/to the DRAM can be interrupted by DRAM refresh cycles. Furthermore, writing data to DRAM also is not deterministic because the timing changes depending on whether the row of the DRAM is already active or whether it must be activated prior to writing. A cache miss is when the system needs to get data from the cache and it is no longer in the cache, which slows down the time to retrieve the data. The CPU also has to contend with operating system needs and interrupts, which adds additional uncertainty in the timing. Table 5 summarizes these sources of non-deterministic latency which are well documented in computer architecture literature [7, 16].

Microcontrollers (MCU)

Microcontrollers are one of the simplest processor based systems and are commonly used as digital control systems due to their low cost and low development time. Microcontrollers are simple reduced instruction set computers that are very similar to conventional CPU

Table 5: Sources of Non-Deterministic Latency in CPU-Based Systems (adapted from [7, 16]).

Source	**Subsystem**	**Description**	**Latency (cycles)**
Cache Hit (L1)	Cache	Data found in L1 cache with minimal delay	1–4
Cache Miss (L2/L3)	Cache	Data retrieved from lower cache levels	10–50
Main Memory Access	DRAM	Access latency depends on row state and contention	100–300
DRAM Row Activation	DRAM	Additional delay when required row is not active	50–100
Cache Coherency	Cache / Interconnect	Multi-core shared memory synchronization overhead	10–100+
Pipeline Stall	CPU Core	Delays due to hazards or branch misprediction	10–20
Interrupt Handling	OS / CPU	CPU pauses execution to service interrupts	100–1000+
OS Scheduling	OS	Context switching and task scheduling delays	1000–10000+
I/O Contention	Bus / I/O	Delays from shared bus or peripheral access	100–1000+

based systems. The key differences being that microcontrollers usually have much fewer processing cores and much lower clock frequencies. They also don't typically have a standard operating system and utilize custom assembly, C or C++ code as their primary system control. Microcontrollers also commonly come with peripherals such as analog to digital converters and communication hardware that make integration with sensors and actuators easier.

The downside of microcontrollers is that their precision for computation is often relatively low and they have slower computation times due to their lower clock frequency. Microcontrollers are cost efficient and offer quick development time due to their additional peripherals making them a good choice for early prototyping or when high precision and low computation time are not necessary.

Field-Programmable Gate Arrays

Field programmable gate arrays (FPGA) allow for system reconfiguration at the hardware level, which directly contrasts with conventional computing where instructions are executed in a primarily serial manner. FPGAs are a fully parallel system with a deterministic timing which contrasts with conventional computing as they have to contend with the operating systems needs and delay due to their use of memory. The deterministic timing of FPGAs makes them ideal for real-time computing applications such as feedback controls, digital signal processing, and hardware acceleration. FPGAs offer many of the benefits of having a custom hardware solution with less of a cost investment and the high configurability makes them very adaptable to design changes. The tradeoff of the FPGA lies in the increased development time and slower processing speed compared to a custom hardware solution.

The configurable portion of the FPGA, often referred to as the "fabric", contains digital signal processing (DSP) slices, programmable logic blocks, and block RAM. The functionality of these components is crucial as it allows the FPGA to have deterministic latency and maintain high computational throughput. Additionally, since the FPGA does not have an OS it does not have to contend with interrupt handling or general OS tasks which, as shown in Table 5, can lead to significant uncertainty in latency.

Digital Signal Processing Slices DSP slices are included in the FPGA fabric that accelerate mathematic operations such as addition and multiplication. They are commonly made up of an accumulator and a multiplier. The discrete accumulator and multiplier have limited size which means that in order to multiply or add data that doesn't fit in a single DSP slice the FPGA must utilize multiple DSP slices in parallel. The data size can drastically increase the amount of DSP slices required to implement a design in the fabric.

Block Random Access Memory Block RAM is low latency static RAM (SRAM) memory that the FPGA can utilize for quick storage of data during operation. It does not require refresh cycles like DRAM in typical computer systems. Since the block RAM is static and does not require refresh cycles, it has deterministic latency. The inclusion of static block RAM in fabric means that FPGAs do not suffer from the delays caused by DRAM shown in Table 5. BRAM also can store the output of highly complex computations in large look-up table which reduces computational delay.

Programmable Logic Blocks Programmable logic blocks are utilized to create arbitrary logic functions as well as to implement simple digital logic designs. These logic blocks are crucial to the inter connectivity of the FPGA. They are configurable to connect the other key components of the fabric, which enables the system to be highly adaptable to design changes.

System-on-Chip FPGAs The SoC FPGA configuration has a CPU paired with the FPGA to allow the end user to utilize the benefits of both. The CPU paired is commonly a low level CPU such as an ARM cortex which is often found in phones or other embedded systems. The CPU is routinely used to feed data to the FPGA, or to act as the high level orchestrator of the data processing [17]. The combination of both systems has several advantages as it allows for a combination of serial computation and parallel computation and it allows a true operating system such as Linux to utilize the hardware acceleration offered by the FPGA.

System-on-Module FPGAs System-on-module (SoM) FPGAs contain SoC FPGAs but are designed to be easier to implement from a hardware perspective. SoM FPGAs have the same CPU and FPGA architecture as the SoC but include other additional peripherals to reduce the amount of hardware design needed to reach a viable product. The key draw of a

SoM FPGA is the included DRAM on the SoM, allowing for the FPGA and CPU to access large amounts of memory without any additional hardware. The SOM is designed to be mounted to a separate PCB that is designed for the users specific application, which allows the user to focus on the design of the PCB board, instead of spending time developing the FPGA bsystem that is readily available as a commercial-off-the-shelf (CoTS) SoM.

RASPBERRY PI MOTOR CONTROL IMPLEMENTATION

The Raspberry Pi implementation is a preliminary prototype of a Field Oriented Control system for controlling a stepper motor where a Raspberry Pi Computer acted as the primary controlling computer that interfaced with the motor and its sensors. The system utilized a Linux installation on the Raspberry Pi to execute C++ code responsible for operating the system. The intention of the prototype as a whole was to mimic the construction and operating conditions of a small scale CNC milling machine. This influenced the motor choice as well as the mechanical hardware used in the test fixture, which will be discussed more in depth in the following sections. The system also utilized a custom designed PCB to both drive the motor and monitor its current and position. The ultimate goal was to prove that FOC could be applied to stepper motors and that the quadrature current vector resulting from the FOC computation increased relatively proportionally to the force applied to oppose the motors movements. The key hypothesis was that if the quadrature current was minimized to be only what was required to overcome the opposing force, then the quadrature current could be utilized to determine an accurate estimate for the force being applied to the system. The intention of the opposing force was to mimic the loads that a small CNC mill would experience during a cutting operation. It is worth mentioning for clarity that the load experienced by the load cell is equal and opposite the force being applied to the connection on the Z-axis of the mill as it is a Newton's third law pair by design. The following sections will discuss the prototype's mechanical design, electrical system, feedback control architecture, and the data collected from its operation.

Control Plant

As mentioned above the goal of the system mechanically was to mimic the construction of a small CNC mill and thus it was constructed from components from a small CNC mill.

The mill in question is known as a Pocket NC and the test apparatus essentially mounted a single axis of the small 5 axis CNC mill to a fixture plate for testing. Figure 9 is an annotated picture of the test apparatus for reference.

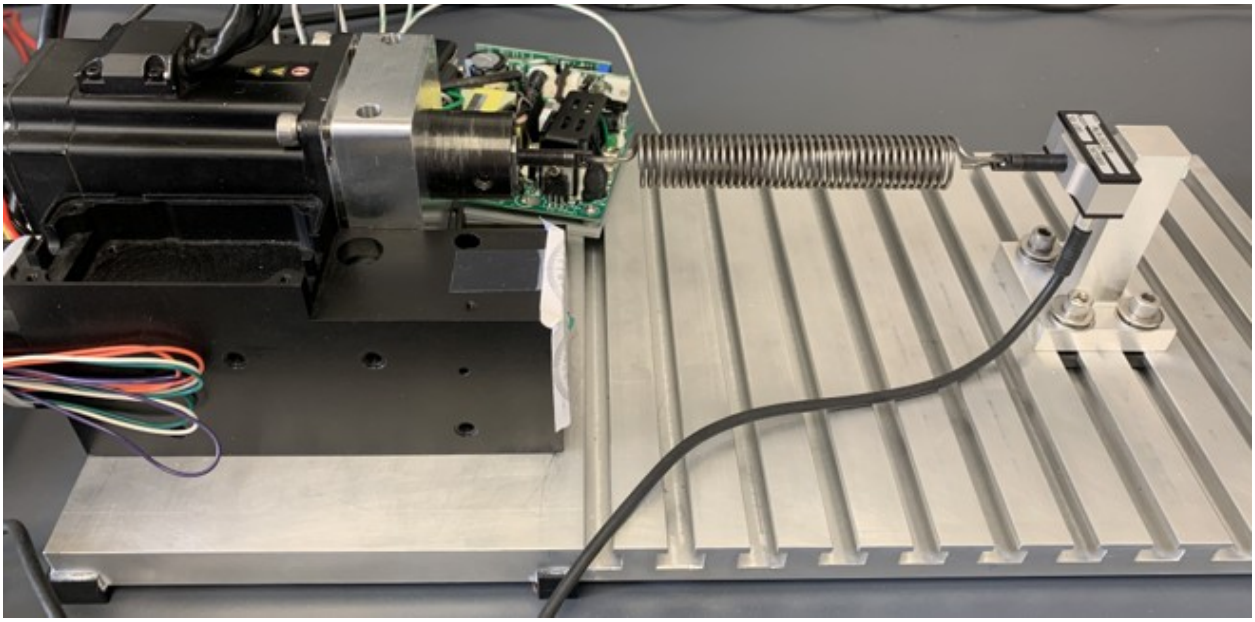


Figure 9: Image of the Raspberry Pi prototype mechanical system.

Mechanical Hardware

Figure 10 is a low fidelity model of the afore mentioned pocket NC with the machines coordinate system for reference. The Z-axis mechanical motion system, which can be seen in Figure 9 of a pocket NC was mounted to a custom aluminum plate for testing including the spindle motor of the mill. The key difference between the custom test bench Z-axis and the standard Z-axis of the Pocket NC is inclusion of a ball screw in place of the standard lead screw. The ball screw is a critical addition to the design because ball screws are back drivable whereas lead screws are not considered back drivable. The stage being back drivable is crucial to proving that with FOC it is possible to sense load changes because otherwise little to none of the load would be transferred to the motor and then it would not be possible

to make the assumption that the quadrature current in the motor is proportional to the load applied to the motion system.

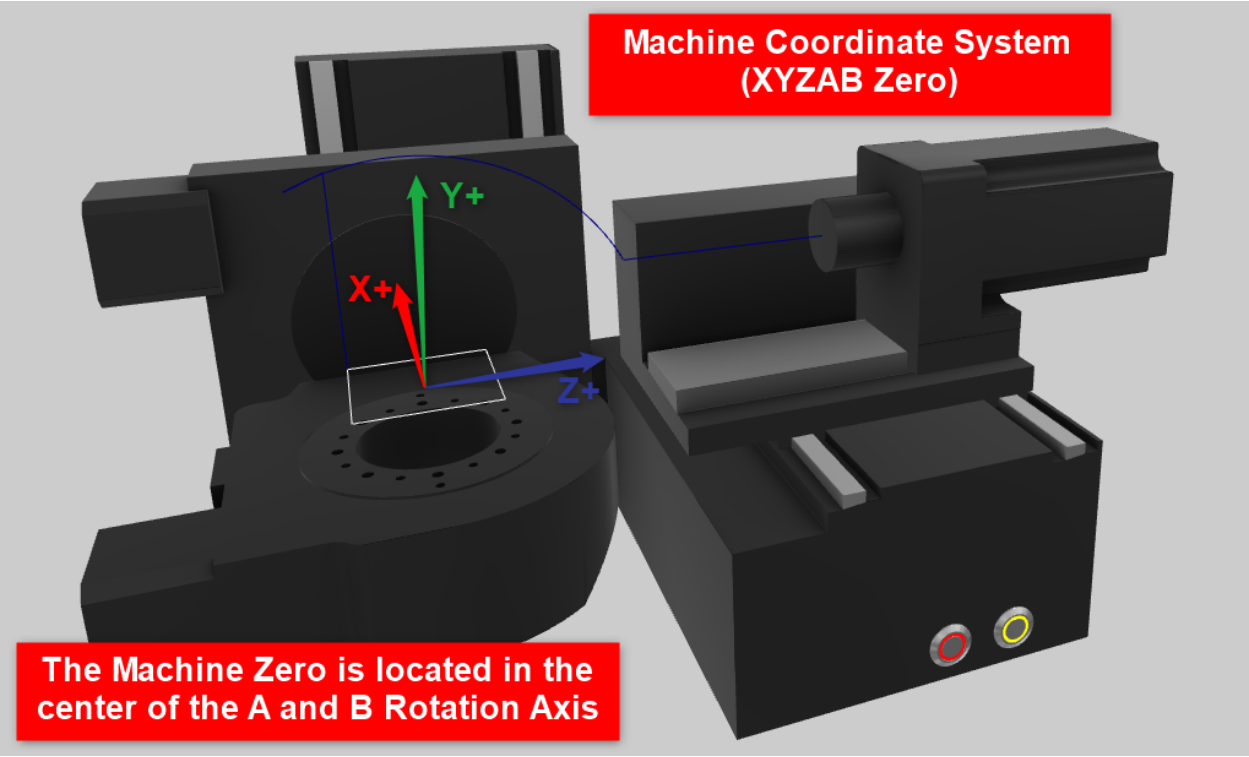


Figure 10: Pocket NC diagram with the machine coordinate system.

The spindle motor mounting was redesigned to accommodate the slightly different mounting hole locations of the ball screw but otherwise the standard Pocket NC Z-axis remained mostly unchanged including the sliding linear rail that allowed for the travel of the spindle motor and its mount.

Load Cell Mounting and force sensing The custom aluminum fixture plate shown in Figure 9 was designed to make a modular method for mounting a load cell for force sensing during testing. It would have been more ideal to make a less modular load cell mounting method but at the time of design and manufacturing it was the intention to use the test bed to emulate several different loading conditions experienced during machining, but unfortunately

it was only used in a single one due to other issues and a need for better linear rails and overall motion system.

As can be seen in Figure 9 the load cell used for force sensing was secured to a post mounted via T-slot nuts to the fixture plate. The critical dimension for the post securing the load cell was the height of the mounting hole as the intention was to align the mounting hole concentrically with the spindle motors chuck so that force was applied orthogonal to load cell. The intention was to minimize the twisting force applied to the load cell and ensure that the load data collected accurately represented the force being applied by the motor. The Z-axis spindle motor was connected to the load cell via a tension spring because as the Z-axis moved at a constant speed the force being applied in opposition to the motors movements would be proportional to the springs changing length in accordance with Hooke's law. This allowed for a secondary confirmation that the data collected from the load cell was accurate and was a convenient way to smoothly increase the force experienced by the motor which was advantageous for the feedback control design implemented.

There was a few problems with the operation of the mechanical system that were discovered during the testing of the system. The issues encountered were primarily with the parts re-used from the disassembled Pocket NC. The primary issue noted during operation was that the linear bearing rubbed on the plate it was designed to slide along. This resulted in additional friction in the system that was inconsistent along its travel length which resulted some irregularities in the current output by the motor. The additional friction resulted in a need for more force applied by the motor which in turn lead to an increase in the quadrature current. This additional noise in the current due to the friction made it more challenging to determine a linear relationship between the motors quadrature current and the load being applied to it.

Electrical Hardware

The electrical system of the initial Raspberry Pi implementation of FOC was responsible for driving the motor, sensing the motor current, interfacing with the load cell, and communicating back to the Raspberry Pi computer. The motor used to actuate the motion system was the same as the standard motor that comes with the Pocket NC with a full integrated ball screw instead of a lead screw as previously mentioned. The motor was the primary focus of the control system and also influenced the design of the rest of the electrical system. The motor in question is a hybrid stepper motor with the same construction as previously discussed in the introduction and background section. A custom, 4-layer, printed circuit board (PCB) was designed to interface with the motor, Raspberry PI and other various sensors in order to create a system capable of implementing FOC on the hybrid stepper motor. The following sections will discuss this PCB as well as the motor characteristics and their relevance to the system design. Three dimensional views of the top and bottom sides of the PCB as well as a cross section view are shown in figures 11-13 for reference during the following discussion.

PCB layers and power distribution The documentation for the Pocket NC indicated that the original system utilized a 36 volt power supply to drive the motors during operation so the Raspberry Pi testing system was designed to be capable of supplying the same voltage to the motors in order to best mimic the original operating conditions.

The PCB was designed with four electrically conductive layers. The two internal layers were used as the power and ground planes for the board which is shown in the layer stackup in Figure 13. Using the internal layers as power and ground planes is common practice on four layer boards as is using the top and bottom layers of the board for signal routing. It is also worth noting that the top and bottom layers had ground pours wherever possible and the whole board utilized via stitching in an attempt to reduce any issues with noise

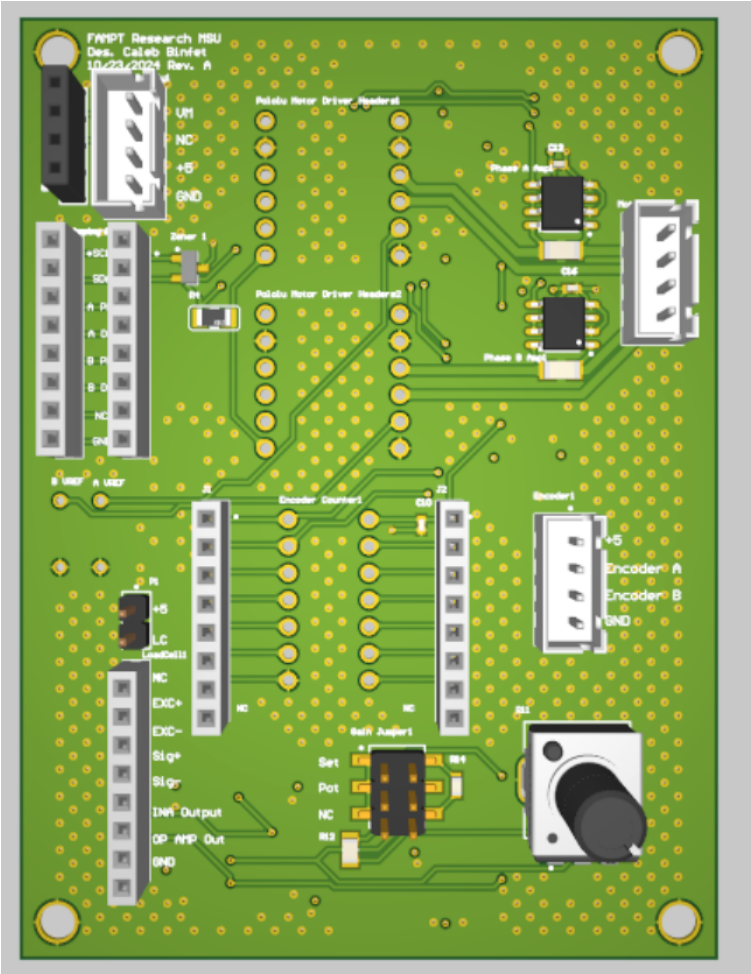


Figure 11: Three dimensional rendering of the top side of the PCB taken from Altium Designer.

from external sources or noise transfer from the PWM driving the board. Via stitching is essentially creating a grid of vias that connect the top and bottom ground polygon pours directly to the ground plane. The vias minimize the distance between any two ground connections on the board therefore reducing electrical potential differences between these points as well.

Figure 14 shows the different sections of the PCB power plane. The separate sections are known as polygon pours and each polygon pour has been labeled in Figure 14 with the operating voltage for that particular polygon pour. The general idea for the layout was to

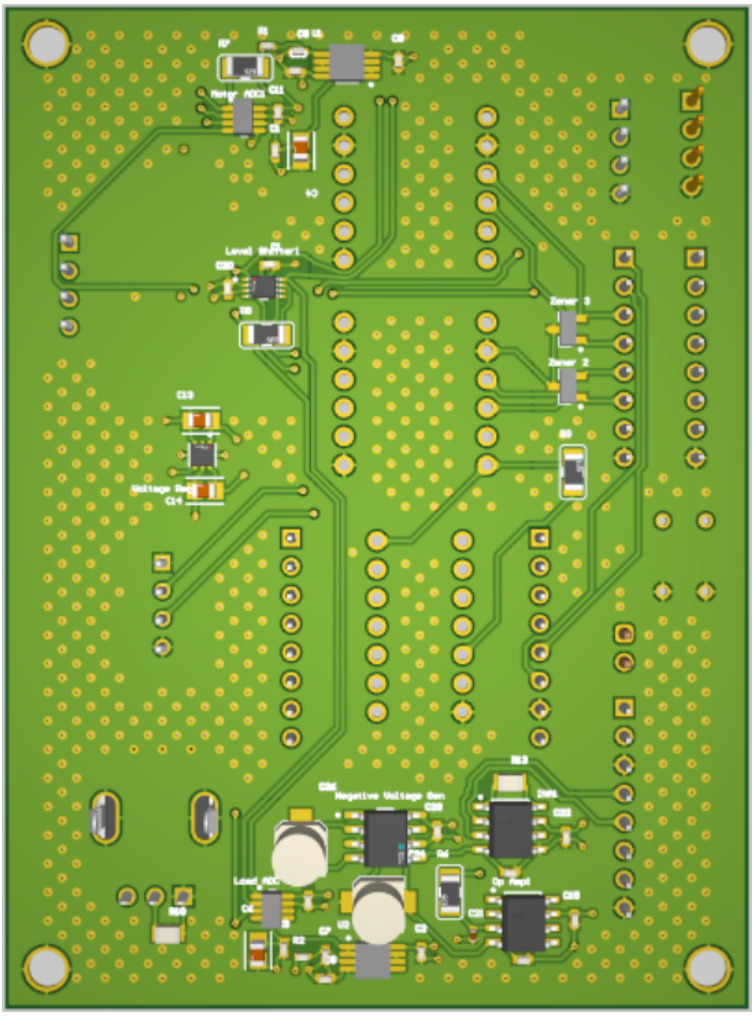


Figure 12: Three dimensional rendering of the bottom side of the PCB taken from Altium Designer.

make supplying voltage to the components mounted on the top and bottom layers simple. Placing the polygon pour with the input voltage for a particular device directly under it makes supplying power to that device trivial as a via that is connected to the power plane can be placed very close to the device then a trace can be connected from the afore mentioned via directly to the voltage input of the device.

The power input to the PCB was supplied by a desktop DC power supply. The desktop power supply was responsible for supplying the motor voltage as well as five volts. All devices on the board and the DC power supply shared a common ground. The connector located

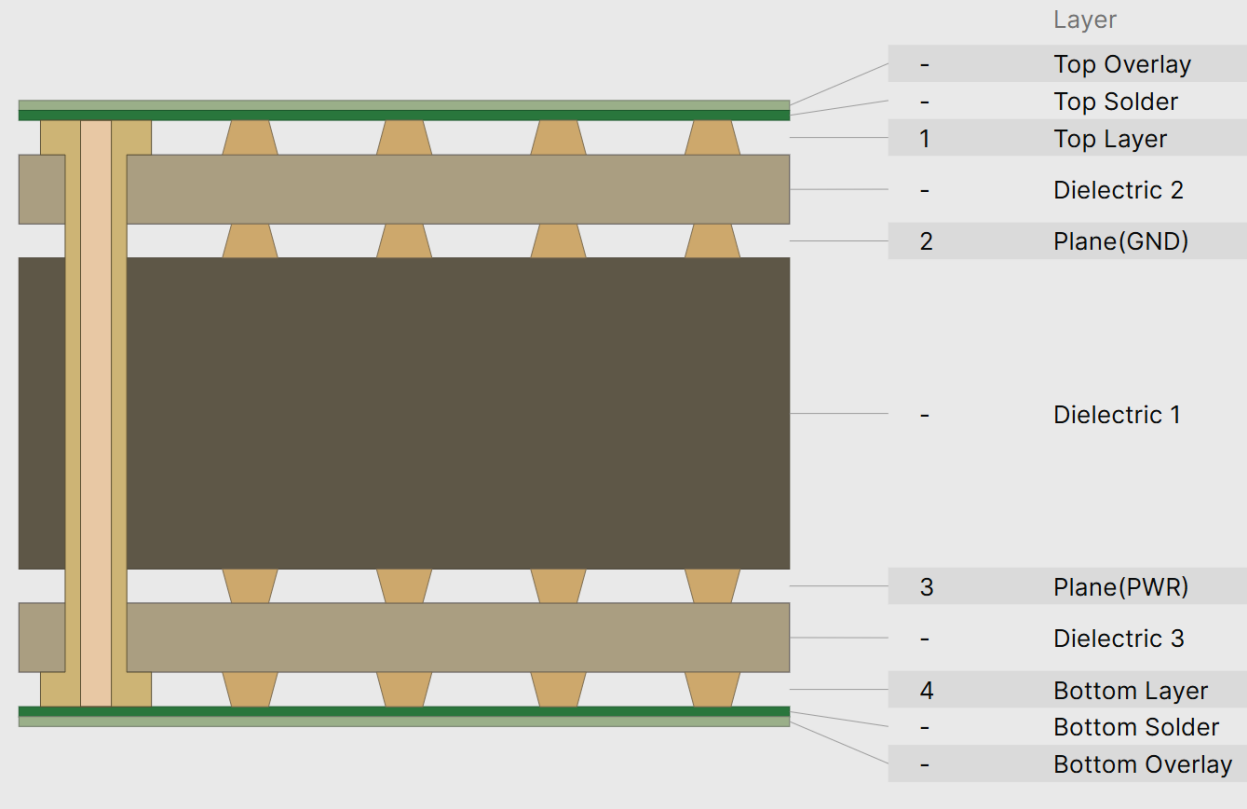


Figure 13: A section view of the PCB with layer descriptions.

on the top left of the PCB was where the DC power supply was connected, which is why sections of both the five volt polygon pour and the motor voltage polygon pour are located under that connector. The power plane has four distinct sections two of which supply 5 volts, one supplies 3.3 volts and the other is responsible for distributing the motor voltage to the motor drivers. The Lower 5 Volt section of the board is separated from the other 5 volt polygon pour to allow for that section to be powered separately via a jumper connector. The design intention was to allow the load cell circuitry to be powered off when not in use to reduce noise in the system.

The 3.3 V section of the power plane was powered by a linear voltage regulator IC (Integrated Circuit). A linear voltage regulator was chosen over a switching regulator to reduce the systems noise. The IC utilized was a LP5923-3.3DRVT and Figure 15 shows its

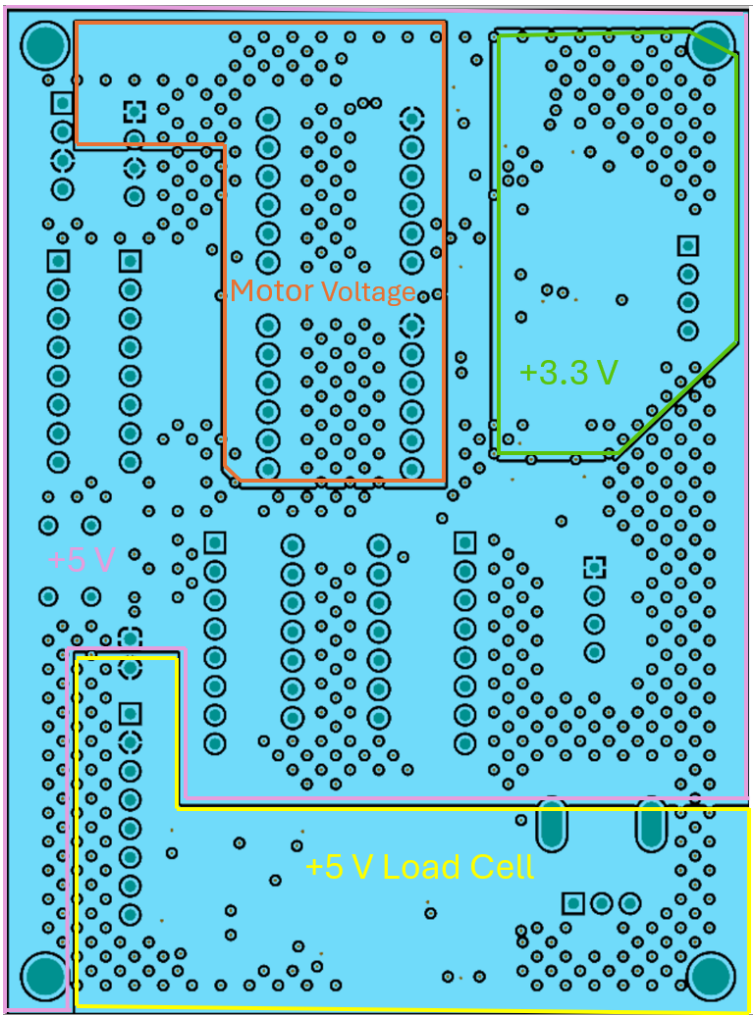


Figure 14: Image of the Custom PCB power plane layout in Altium Designer with labeled supply voltages.

routing. In Figure 15 VCC is the 3.3 V output from the voltage regulator.

The other voltage used as a supply on the PCB was -5V which was primarily used in the various amplification circuits as a negative rail voltage for the amplifiers. This voltage was generated by an IC (Integrated Circuit) designed for this particular application. The IC in question was a MAX1044 and Figure 16 shows the circuit diagram for how the MAX1044 was connected on the PCB. The circuit layout utilized to generate the negative voltage was based off of one of the example cases from the datasheet of the MAX1044. The only crucial consideration when laying out the circuit was to ensure that the capacitors were polarized

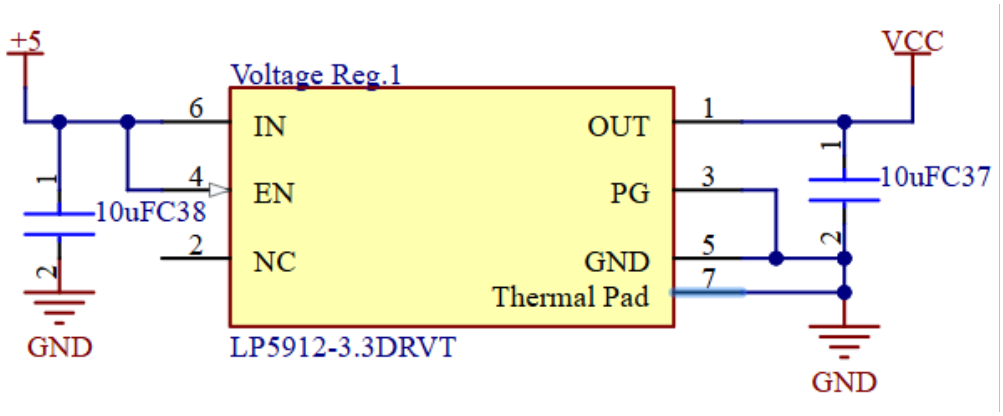


Figure 15: Circuit Diagram of the 3.3 Volt regulator for the PCB.

and the positive and negative sides were connected according to the devices recommended layout. The capacitors in question are labeled C50 and C48 in Figure 16. The whole circuit was powered by 5V jumped over to the load cell power plane so that this circuit could be powered off while the rest of the board was operational to reduce overall system noise.

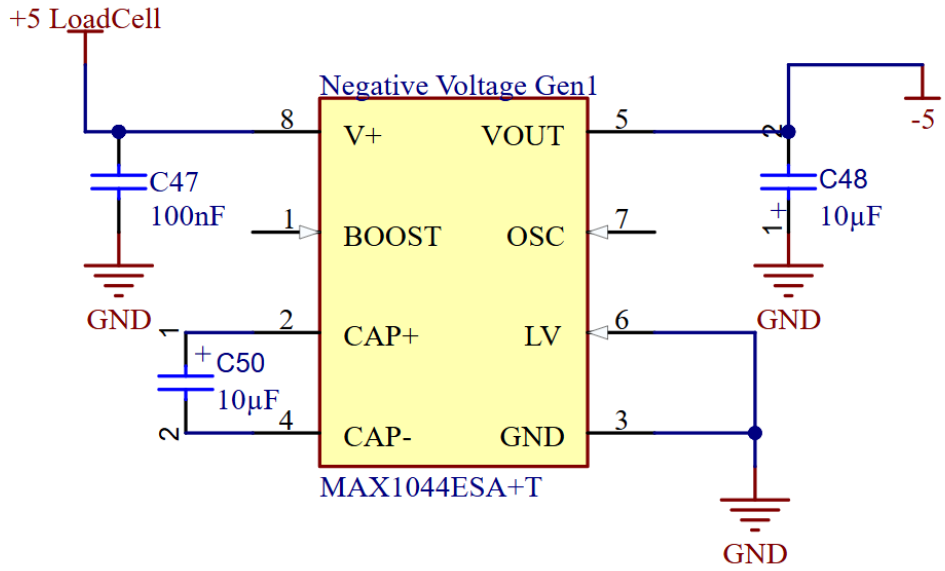


Figure 16: Circuit Diagram of the negative voltage generator integrated circuit.

Motor and Motor Drivers The motor selected was a hybrid stepper motor with a maximum continuous current rating of 0.5 Amp per phase. The 0.5 amps of continuous

current required and the need for a motor voltage of at least 36 volts lead to the use of motor driver breakout boards from Pololu robotics and electronics as the breakout boards met all the requirements and were simple to integrate saving development time. The motor driver breakout boards were mounted via female header pins to the board for this prototype. The motor drivers had relatively simple routing as they only required a pulse-width modulation (PWM) signal and a binary direction signal from the Raspberry Pi to control the motor. It is worth noting that because the stepper motor was not being driven in a typical stepper motor fashion, the drivers used were actually meant to drive a single DC motor and thus one motor driver was required per two phases of the motor. The current driving the motor then flows through the current sensing circuitry to the motor connection.

The motor also had a 5000 line quadrature encoder mounted to the rear of its shaft to allow for accurate position tracking. The quadrate count for this encoder was 20,000 because it is four times the number of lines. This means that the relative positional accuracy of the motor had an error of $\pm$ 0.000314 radians. The tradeoff of such a high positional accuracy was that when the motor is rotating at a high speed the device responsible for interpreting the signals from encoder must be able to correctly interpret the high frequency output of the encoder. The device used was an encoder counter that claimed to be able to handle the high quadrature count of the selected encoder. The device had previously had some issues during its initial testing so instead of mounting it directly to the PCB there was press in header pins mounted with the correct connections for the device. This would allow for quick swapping of the device for a new one in the event of an issue. The device communicated with the Raspberry Pi via I2C which allowed for the Raspberry Pi to get the motors relative position and for the Raspberry Pi to configure the encoder counter.

Current Sensing In this particular application each phase of the motor required a current sensing circuit. The current sensing circuitry utilized a shunt resistor and shunt

amplifier to measure the current flowing through each phase of the motor. The shunt amplifier in question was a Texas Insturments INA241A. Figure 17 shows a circuit with a typical shunt amplifier and shunt resistor layout. In this configuration the shunt resistor is placed in series with a phase of the motor so that current flowing through the motor also flows through the shunt resistor. When current is flowing through the circuit there is a voltage drop across the shunt resistor proportional to the amount of current flowing. This voltage difference is then amplified by the shunt amplifier and the output voltage from the shunt amplifier was routed directly to an analog to digital converter (ADC). The shunt amplifier also had to apply an offset voltage to the voltage drop to enable sensing of current flow in both directions. For example because the supply voltage to the amplifier is 3.3 volts when there is no current flowing through the shunt resistor and motor, the amplifier output is 1.65 volts or half of the supply voltage. Current flow in the positive direction results in a voltage greater than 1.65 volts and current flow in the negative direction results in a voltage less then 1.65 volts.

The shunt resistor has an intentionally low resistance to minimize its affect on the current draw of the system as accurately measuring the current flowing through each phase of the stepper motor was the goal. The precise resistance of the shunt resistor was selected such that at the maximum continuous current of the motor the amplifier output a voltage slightly less than the supply voltage to ensure that the full range of expected currents could be accurately measured with the system.

The ADC used on the PCB was a 12 bit ADC that had 3 voltage input channels, a reference voltage channel, and utilized the inter-integrated circuit communication protocol (I2C) to communicate the conversion results. As shown in Figure 18, the output of the two onboard shunt amplifiers was routed to channel 0 and channel 1. The 3.3 volt supply was routed to channel 2 because early on in testing it was discovered that when the input voltage to the shunt amplifier fluctuated so too did the output which resulted in inaccurate current

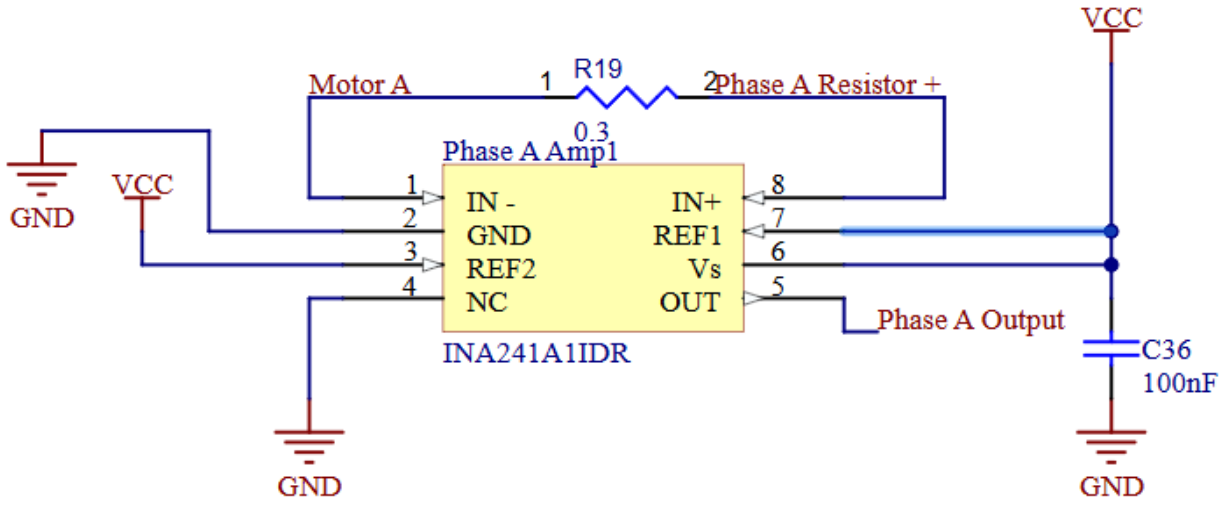


Figure 17: Shunt resistor and amplifier circuit diagram.

measurements.

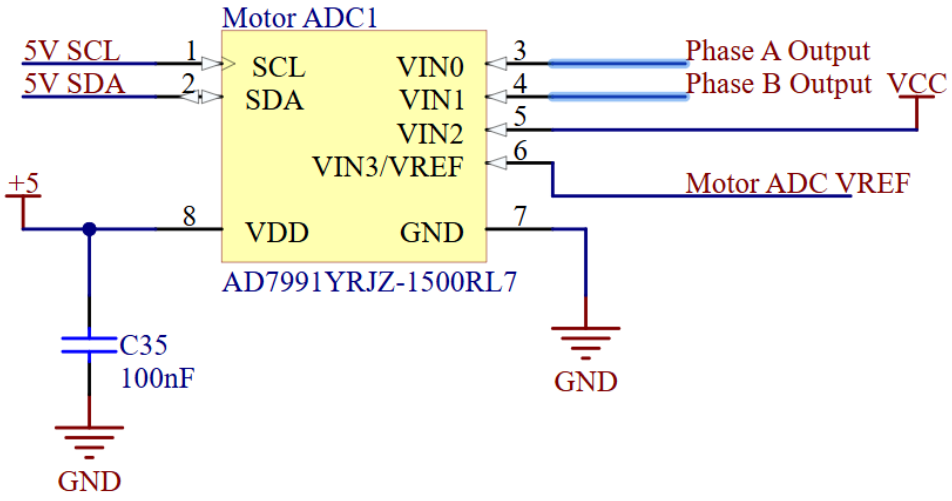


Figure 18: Circuit diagram of the ADC used to measure the motor current.

Load Cell Integration The load-cell integration circuit was designed to integrate directly with the MLP-50-C model from Transducer Techniques. Figure 19 is the circuit diagram supplied by Transducer Techniques for the load-cell. The load-cell is a Wheatstone bridge circuit as shown in Figure 19 which means that to utilize the sensor a voltage must be applied

to the excitation leads and as the applied load changes the voltage difference between the signal leads changes proportionally.

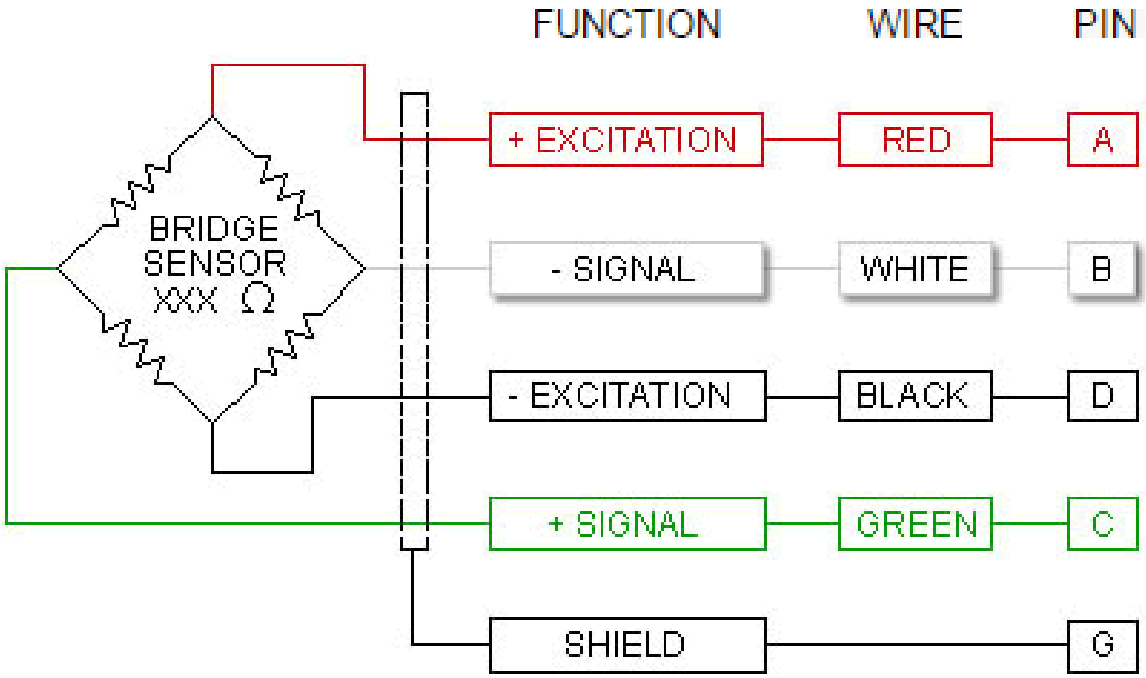


Figure 19: Load cell Wheatstone bridge circuit diagram supplied by Transducer Techniques.

The PCB was designed to supply +5 volts to the + Excitation lead and the - Excitation lead was connected to ground, meaning that the overall applied excitation voltage was 5 volts. The 5 Volt excitation directly sets the maximum output difference between the signal wires. This device has a rated output of 2 mV/V meaning that when 5 Volts were supplied, the maximum output from the sensor was a 10 mV difference between the signal wires. This means that in this configuration when 50 lbs of load is applied the voltage difference between the signal wires will be 10 mV.

In order to utilize an ADC to accurately measure the output from the load cell the differential voltage between the signal wires need to be amplified. The circuit used to do this was a two stage system. The first stage being a instrumentation amplifier which served

two purposes. The first being to transform the voltage difference between the signal leads in to a single voltage and the second being to amplify that voltage. The instrumentation amplifier used was a INA118U/2K5 and it was configured to have gain of 51 V/V.

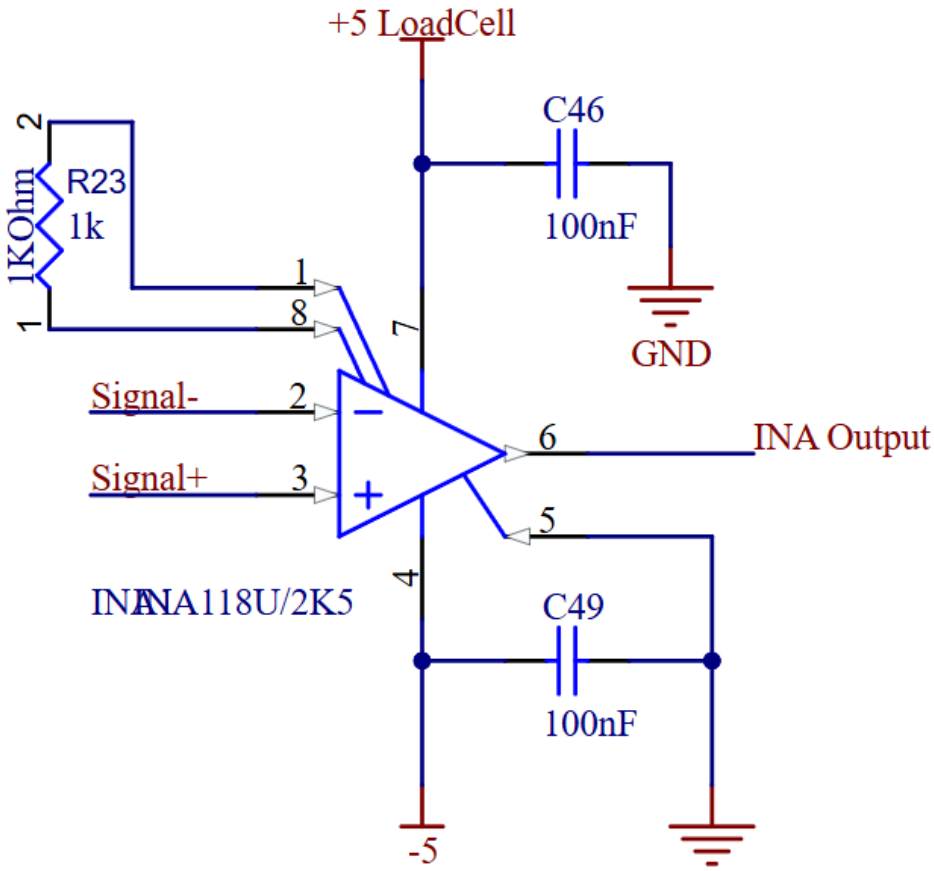


Figure 20: Circuit diagram of the instrumentation amplifier used with the load cell.

Figure 20 shows the circuit diagram used to layout the instrumentation amplifier connections in Altium Designer. The signal+ and signal- connections are directly connected to the load cell signal leads and the 1 KOhm resistor is responsible for setting the aforementioned gain. The output signal labeled INA Output was then routed to the second stage amplifier circuit which can be seen in Figure 21.

The second stage amplifier circuit utilized an Operational Amplifier (Op-Amp) in a non-inverting configuration to apply more gain to the output of the load cell. This stages gain

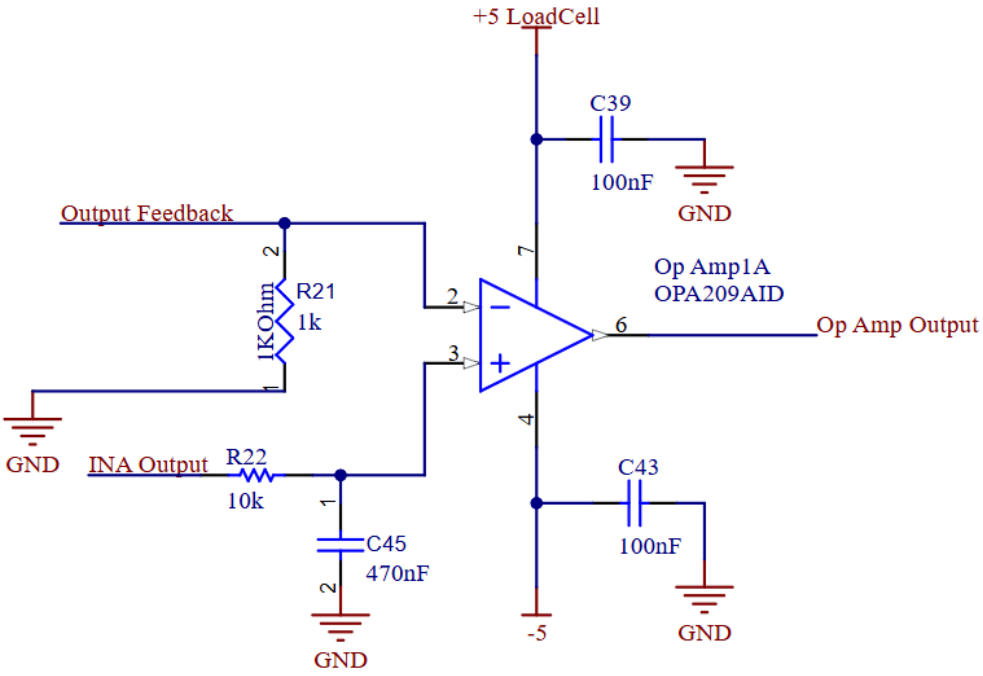


Figure 21: Circuit diagram of the second stage amplifier for the load cell.

was dependent on the resistor connected between the signal labeled Op Amp Output and the one labeled Output Feedback in Figure 21. The circuit for selecting this resistor is shown in Figure 22. This circuit allows the user to select between three different resistances and therefore three different gains on the Op-Amp gain by simply moving a jumper connector. The op amp circuit was also intended to low pass filter the the output of the previous gain stage. The filter topology used was an active Sallen-Key filter with the 3dB frequency set to approximately 34 Hz.

The first position for the jumper connector is between pins 1 and 2 on the gain jumper in Figure 22 and this position sets the gain for this amplifier to 8.25 V/V which was chosen to effectively use almost the full range of possible input voltages for the ADC that was connected to the Op Amp output. The total gain of the system is 420.75 V/V which means that when 50 lbs is applied to the load cell and it is outputting an effective voltage of 10 mV the output of the Op Amp would output 4.2 V. The maximum range of the ADC is 5 V

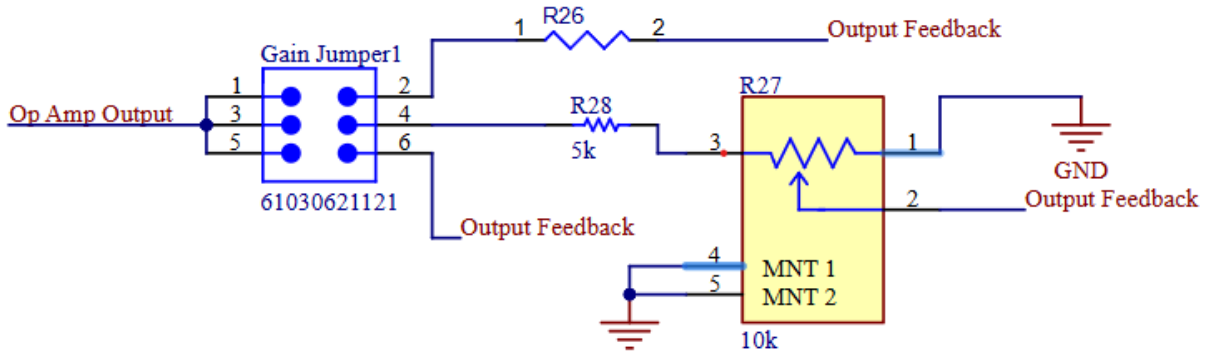


Figure 22: Circuit diagram of of the gain selection circuit for the second gain stage of the load cell amplification circuit.

but the intentional overhead room allowed for sensing of potential overloading of the sensor. This stage was the one used primarily during the testing process.

The intention of the second position of the gain stage jumper was to allow the user to easily adjust their gain to a desired value. This was accomplished by making the second positions resistance adjustable by placing a 10 KOhm potentiometer in series with a 5 kOhm resistor. The potentiometer had a minimum resistance of 500 Ohms and a max of 10 KOhm meaning the possible gains for this stage are 5 V/V to 15V/V.

The third jumper position was intended to give the user the ability to use a resistor external to the PCB to set the second stage gain.

After both gain stages the output from the op amp was routed to an ADC so that using the results from the ADC the Raspberry Pi could determine the load applied to the load cell. The conversion from the voltage value determined by the ADC requires knowledge of the amount of gain applied or for there to be known loads applied to the system to calibrate the conversion factor.

Communication The Raspberry Pi utilized I2C to communicate to the motor ADC, load cell ADC, and the encoder counter. To protect the Raspberry Pi from over voltage, zener diodes were utilized on the communication lines. The Raspberry Pi utilized 3.3 V logic

but it was advantageous to power the ADCs with 5V and thus they communicated via I2C with 5 V logic. In order to allow communication between the ADCs and the Raspberry Pi a level shifting IC was used to convert between the logic levels. Figure 23 shows how the 3.3V logic enters the circuit on SCL A and SDA A and then leaves at the 5 V logic level on SCL B and SDA B. The B side of the circuit also includes two pull-up resistors as is the standard for I2C.

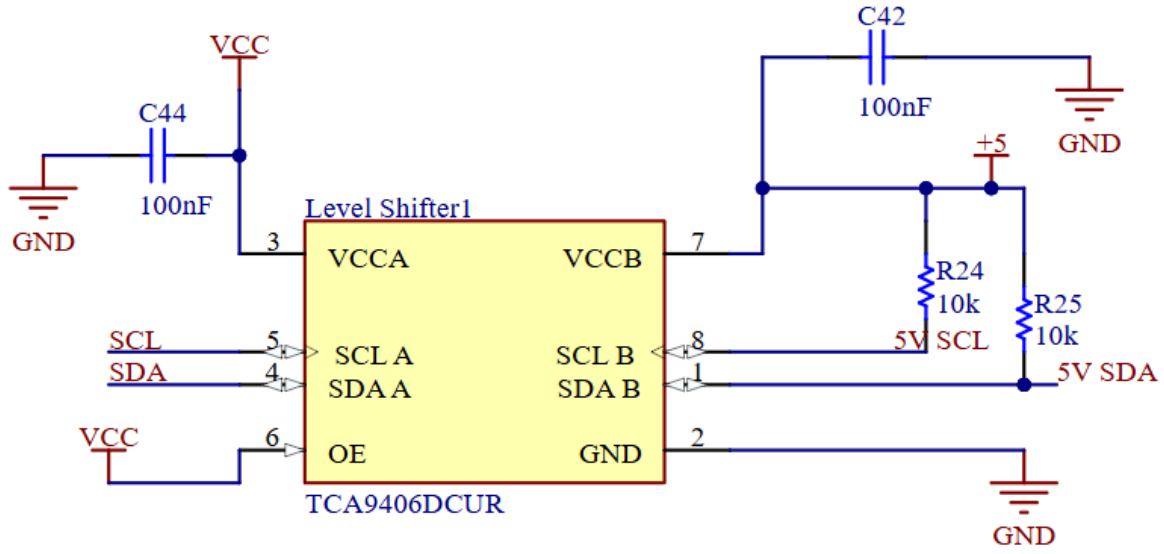


Figure 23: Circuit diagram of the I2C level shifting IC and its connections.

PCB Issues The PCB had a few issues that are worth discussing as well as how these issues were overcome so that testing could progress. The First issue encountered was with the I2C communication on the PCB. The level shifting chip appeared to be failing as the 5 V I2C devices would not respond to communication attempts from the Raspberry Pi. This issue was exclusive to the 5V devices as the encoder counter that utilized 3.3 V logic for I2C responded without issue. Since the problem was localized to the 5 V devices it was relatively easy to determine the issue was with the level shifter IC shown in Figure 23. To confirm this was the issue and it wasn't the I2C devices themselves the level shifter was removed

from the PCB and a new level shifting circuit was constructed with MOSFETs off-board and then jumper wires were soldered in to the 5 V I2C bus. This fixed the issue and allowed the Raspberry Pi to communicate with the 5 V I2C devices on the PCB. It was never fully explored what caused the device to fail, but it is believed that during handling it was exposed to excessive electrostatic discharge or that the device overheated when it was soldered to the PCB.

The second issue encountered was with the shunt amplifier connections. The intention was to set the voltage output of the shunt amplifier to 1.65 V when there was no current flowing in the motors but an error was made in the circuit layout that incorrectly set this voltage. The voltage is set by the two reference pins shown in Figure 17. To correctly set the no-current voltage to half of the source voltage one reference pin needed to be connected to the boards ground and the other to the voltage source of the IC. To fix the issue the highlighted trace in Figure 24 was cut and the pad on the right side of the trace in Figure 24 was shorted to pin 4 of the device. Intentionally soldering the pin to pin 4 made the required ground connection and correctly set the no-current output voltage to the desired 1.65 V.

An additional issue encountered with the system was that the data returned from the encoder counter was erratic and did not match the expected values. It was initially assumed that the issue was with the encoder counter IC and that despite the data sheet for the device claiming to be able to meet the encoders high-frequency output it could not. To solve this a microcontroller with a relatively high clock frequency was used to count the encoder pulses and report the data back to the Raspberry Pi. This fixed the issue for some time but then issues returned with this system which then led to the postulation that the encoder itself was not operating correctly so it was then replaced. Once the new encoder was installed the system utilizing the micro controller started performing as expected and no issues were encountered afterward. It is still unclear if the encoder counter IC that the PCB originally used had issues or if it was the encoder the whole time but by the time the issue with the

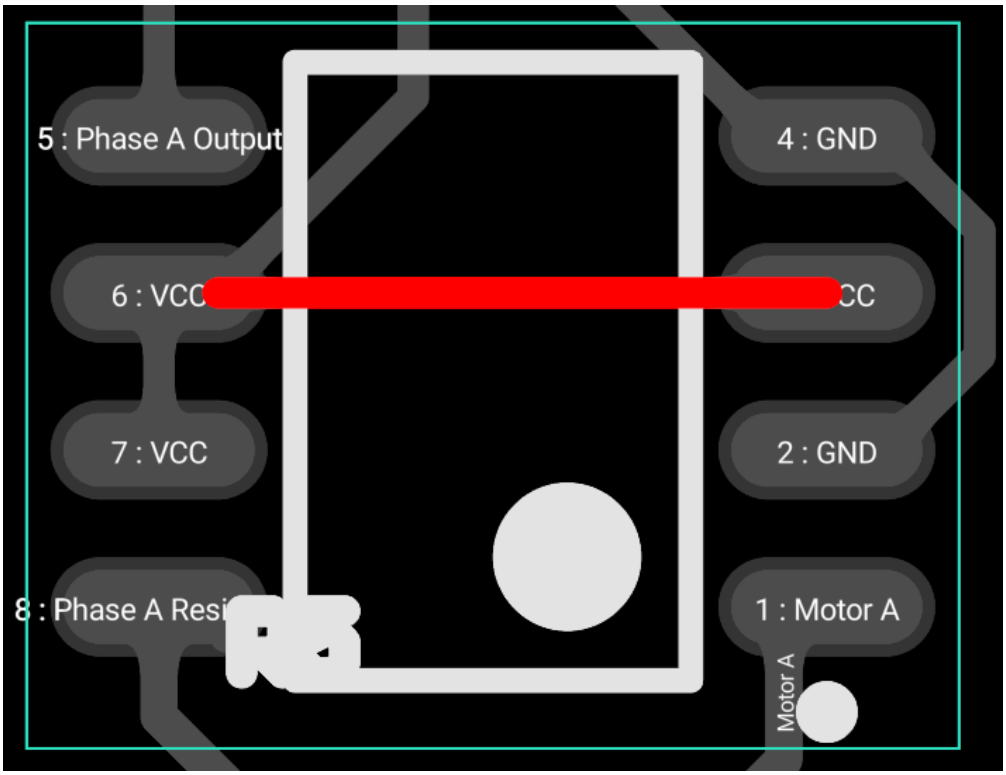


Figure 24: Shunt amplifier board layout with highlighted trace that was cut.

original encoder was discovered development of the next version of the system had begun and did not require an encoder counter so exploring more was not a priority.

The other issue encountered during testing was that the voltage potential of the ground net of the Raspberry Pi would consistently fall below the ground for the PCB and the desktop power supply. It is believed that this issue led to all of the current for the system flowing to the ground on the Raspberry Pi which resulted in the Raspberry Pi failing due to over current. Connecting all of the ground pins on the Raspberry Pi to the PCB to reduce the potential difference along with adding an additional protection circuit between the Raspberry Pi and the PCB eventually mitigated the issues. It is possible that the voltage potential difference issue could have been mitigated by mounting the Raspberry Pi directly to the PCB thus reducing the path between grounds.

Feedback Control Design

The feedback control design for this system was implemented fully on the Raspberry Pi in C++ code. The architecture utilized a Cascaded PID as discussed in the cascaded PID subsection of the background. The design featured 3 PID controllers and implemented FOC on a two phase system. A diagram depicting the controller architecture is shown in Figure 25.

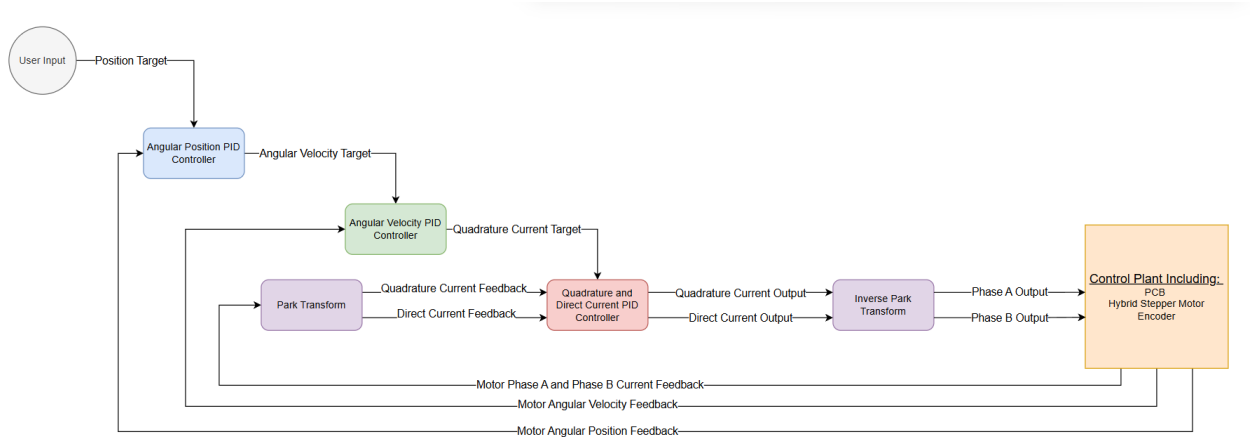


Figure 25: Diagram of the cascaded PID field oriented system implemented on the Raspberry Pi.

The primary goal of the control system depicted in Figure 25 is to drive the motor to a desired position while being able to accurately track and control the quadrature current. To simplify the diagram it can be assumed that the feedback error signals are computed inside the PID controller blocks.

The user is responsible for supplying the system with a position target which is the target for the angular position PID controller shown in blue in Figure 25. This system worked in mechanical radians as that is easiest to transition between mechanical radians and linear travel distance. The output of the Position PID controller is responsible for setting the target for the angular velocity PID controller which is green in Figure 25. Then

the output of the angular velocity controller sets the current target of the quadrature current PID controller. The quadrature current controller then determines the voltage to be output to the motor in order to reach the desired current. The direct current output by the system was controlled by a separate PID controller with a target of 0.

The PID controllers each receive their own individual feedback signals from the motor. The angular position PID controller receives position feedback via the quadrature encoder on the motor. The angular speed PID controller gets its feedback from the same signal by computing the derivative of the position using current and previous position feedback readings. The current per phase of the motor was fed back using the shunt amplifier and ADC circuitry. After the position feedback and speed feedback are determined, they are directly used by the controllers whereas the current readings are fed into the Park transform before the PID controller can affect the current output. The inverse park transform then takes the output quadrature current from the quadrature and direct current PID controllers and outputs it.

Digital Control

The digital control for this design was a CPU based control system as it utilized a Raspberry Pi computer to perform the control. The Raspberry Pi utilized one of its two cores to perform the feedback control. The system utilized the standard Raspberry Pi OS which is a distribution of Linux. A single core of the CPU was utilized to perform the feedback control. The Raspberry Pi does include prebuilt I2C hardware which was utilized to retrieve the motor position and current readings. The control system did encounter the timing struggles discussed in the digital control section above. The code utilized C++ and a single core so the code to get feedback as well as the control outputs was entirely serialized. The time to complete a cycle is very important for the control output calculation but it was affected by the CPU non deterministic timing. The rough frequency for completing a full

control output calculation was 200 Hz or a period of 5 ms. This was relatively slow and inconsistent.

Implementation Conclusion

Several aspects of the system could be improved in the next version. To improve the system mechanically, the friction of the sliding stage could be reduced by simply using higher quality linear bearings and potentially making the load cell mounting location a set location. This would reduce the afore mentioned friction issues and make gathering load data along with the current data simpler because it would reduce the amount of time taken to align the load cell with the motor. Additionally, it would be beneficial to build the system in such a way that it could be used to machine parts as that would offer further insight in to sensor readings during machining operations.

The PCB itself had several issues that simply needed to be fixed with either new components or correcting initial layout errors. The shunt amplifier needed to be wired correctly to set the no current voltage and applying additional filtering to the output could provide more accurate current readings. Additional protection circuitry between the digital control system and PCB could protect the digital control system from being damaged by the rest of the control circuitry.

The digital control system could also use improvements especially with the issue of the non-deterministic latency caused by the CPU system and the relativity slow computation time. These issues are primarily caused by issues discussed previously in the digital control section of the background. This application would benefit from the use of an FPGA due to its deterministic timing. Utilizing an FPGA would reduce the hardware on the PCB, increase the control systems frequency, and allow for more complex control architectures.

The Raspberry Pi implementation of the control system offered a lot of promise as a proof of concept and provided a jumping off point for continued work in to a FOC stepper

motor driver system. This system provided support to the hypothesis that it is possible to implement FOC on a stepper motor and that the quadrature current does increase relatively proportionally with an applied load.

FPGA MOTOR CONTROL IMPLEMENTATION

The DE10-Nano based implementation of the FOC stepper motor control system was the next control system iteration after the Raspberry Pi implementation and utilized the DE10-Nano SOC FPGA development kit. This system was intended to increase the operating frequency of the control system and resolve some hardware related issues on the PCB of the previous version. Additionally, building the control system on an FPGA adds potential for higher complexity control architectures and allows for the inclusion of additional sensors in the future. The most significant difference is that by utilizing an FPGA based control system the computation latency is deterministic. This means that the period utilized to compute the feedback controllers output is consistent, which is critical for discrete feedback control.

The use of an FPGA also enabled the use of Simulink (MathWorks) and its VHDL design library and synthesis tools for the development of the feedback control system. Once the feedback control design was finalized in Simulink, it was integrated with custom fabric based components and compiled utilizing Quartus Prime design software.

The system was designed such that the targets and gains were adjustable via AXI bus communication with the SoC on the development board. Integration with the SoC also allows for monitoring and storage of data from the system during operation. This data storage and monitoring is critical because in a milling operations the quadrature current measured by the system should be proportional to the pressure being applied by the cutting tip allowing for estimation of the the applied force.

Control Plant

The mechanical hardware target for this implementation is the same as the previous one discussed in Section 2.

Electrical Hardware

The primary electrical hardware changes in this implementation were mostly due to the replacement of the Raspberry Pi from the previous implementation with the FPGA based development board. This mostly had to do with the interface between the fabric and the physical sensors such as the encoder and the current sensing hardware. Some of the hardware discussed in this section was entirely implemented in the fabric of FPGA but because it is replacing physical hardware ICs from the previous implementation and acts more in a sensor role than a feedback control role it will be discussed in this section. These components are responsible for counting the encoder pulses, generating the PWM signals to control the motors drivers, and communicating with the ADC. The HDL used to create the custom fabric based components is provided in the Appendix.

Encoder Counter The encoder Counter component is responsible for interpreting the 90 degree phase shifted signal produced by the encoder mounted to the rear of the stepper motor. This component functionally replaced the encoder counter IC and the microcontrollers utilized in the Raspberry Pi based system. It is treated as part of the plant because it is responsible for feeding the motors relative position to the feedback controller block and therefore, functions more as a sensor than as part of the controller.

The encoder counter has three additional input signals beyond the standard clock and reset signals. The additional signals are the A and B signals produced by the encoder as well as a zero signal. The A and B signals are used to track the change in rotational position of the motor while the zero input signal allows the user to reset the components counter to zero. The outputs of the component are a signal confirming that the counters total has been reset to zero and an integer known as counts that holds the number of phases the motor has traveled either forward or backward. An image of these encoder output signals is shown in Figure 26 along with a phase count for reference.

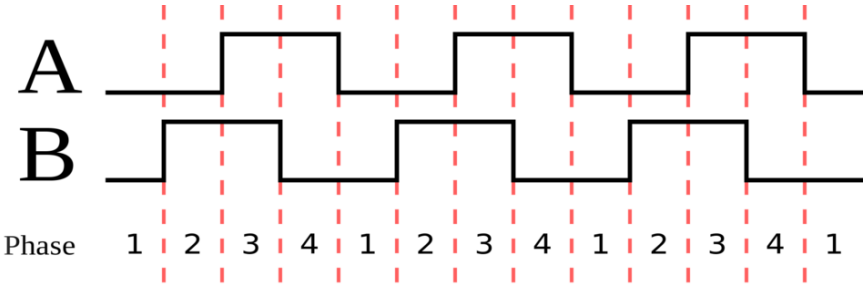


Figure 26: Image depicting the 90 degree phase shifted signal produced by the quadrature encoder with phase labels (reproduced from [14]).

The encoder counter component named `encoder_counter_zeroable` is a synchronous design meaning all of its actions are synchronized with the input clock. The issue is that signals `A`, `B`, and `zero` are not synchronous signals as they dependent on the continuous rotation of the motor and a user input. In order to correctly interpret these signals, the first thing the component does is synchronize them with the clock of the FPGA by passing them through a synchronization process. This essentially performs a sample and hold on the signal for 2 clock edges to prevent any metastability issues that could be caused by a transition while sampling a signal. Once signals `A`, `B`, and `zero` have been synchronized the logic to determine how to update the output `counts` signal was implemented based on the synchronized signals. The `counts` signal is either increased, decreased, or left the same based on the previous state and current state of the synchronized input signals. Essentially, if the previous phase and current phase differ such as they do traveling left to right on Figure 26, the counts signal is increased if the phase has increased and it is decreased if the phase has decreased. The exception to this being that transitioning from phase 4 to phase 1 is treated as an increase as well.

It is very common for a mill to run a homing procedure where the motion axes travel to their limit in the negative direction and then that location is set as the zero location for those axes. The inclusion of the user input `zero` signal makes doing this kind of homing

possible either by connecting a common limit switch to the zero signal or allowing the digital control system to zero the counter by controlling that signal directly.

PWM Generation The PWM generation is implemented in the fabric of the FPGA and is directly controlled by the feedback controller and the user. The PWM generation system has an adjustable period which is intended to be adjusted by the user to match the best the desired input frequency of either the motor drivers or the motor being driven. The duty cycle is intended to be controlled by the feedback controller directly so that way it can directly affect the current flowing in the phases of the motor.

The PWM generation system is a hierarchical design as it incorporates a higher level component and a lower level component instantiated inside the higher level component. The higher level component is called `Full_bridge_single_output_driver` and the inner component is simply called `pwm_controller`.

The `pwm_controller` component is responsible for taking in the desired period and duty cycle and then output a PWM signal matches the desired output as close as possible. The period is input in milliseconds then the component computes how many clock cycles occur in a single period. Then by counting clock edges the system is able to closely match the desired period. The process is relatively similar for determining when the output signal should be set high to achieve the desired duty cycle. The duty cycle is supplied to the system as a percentage which is then multiplied by the total number of clock edges in a period to calculate the number of clock edges that the output should be high during the cycle. During a period the system is constantly counting clock edges and when the the number of edges is equal to the number of edges it should be high for the system then sets the output low and waits for the period to end and restart the cycle. If the duty cycle percentage is higher than 100% then the system defaults to an always high output state.

The motor drivers discussed previously in the Raspberry Pi implementation section rely

on a PWM input signal as well as a direction signal to determine which side of the motor phase is connected to positive and which is connected to the negative output of the driver. The `Full_bridge_single_output_driver` is responsible for correctly setting the direction output from the FPGA based on the signed duty cycle input signal. The duty cycle input to the highest level component is a signed two's complement number and the sign of the number is utilized to determine the directional output bit. First the component checks the sign of the input number and sets the directional output bit high if it is positive and low if it is negative. Then the component inverts the duty cycle value if it is negative to achieve the correct absolute value for the inner component and then the sign bit is removed prior to passing the value to the `pwm_controller`.

Single Channel ADC The ADC for this implementation was changed to better utilize the parallel computation capabilities of the FPGA system as well as make the system more compatible with a three-phase BLDC motor. The ADC utilized is a MAX11162EUB from Analog Devices, which is a 16 bit, single input channel ADC capable of 500 kilo-samples per second. In order to measure the current in phases A and B of the hybrid stepper the system utilized and ADC for each phase of the motor. Utilizing multiple single channel ADCs, the system can sample current readings from the shunt amplifiers at the same time. This is an improvement over the previous system as the previous system could have suffered from changes in current readings of a single phase while the current of the other phase was being measured. Additionally, this setup makes the system more viable for applications with three-phase motors because if both phases of a three-phase motor are measured synchronously it is possible to apply Kirchhoff's current law (KCL) to the measured phases and determine the non measured phase by summing the two measured phases.

$$i_a(t) + i_b(t) + i_c(t) = 0 \tag{25}$$

$$i_c(t) = -\left(i_a(t) + i_b(t)\right) \tag{26}$$

Equation 25 applies KCL to the equivalent BLDC circuit shown in Figure 5 and Equation 26 solves for the non measured current $i_c(t)$ by applying a negative and summing the measured currents, $i_a(t)$ and $i_c(t)$. Synchronous sampling two of the currents makes this a more reliable method for accurately estimating the non-sampled current.

The ADCs utilizes the Serial Peripheral Interface (SPI) communication protocol, which led to the creation of a custom component in the FPGA fabric to communicate with the ADCs. The FPGA being a fully parallel system made synchronous communication with each individual ADC relatively trivial. The FPGA also does not require any additional hardware the communication can simply be performed by the FPGA fabric and routed out to on the GPIO pins shown in Figure 32.

ADC Communication The signals output by the FPGA for the ADC are directly influenced by the configuration the ADC is placed in. The SPI configuration chosen is referred to as 3-Wire, No-Busy indicator mode and the connection diagram for this mode from the ADCs data sheet is shown in Figure27.

The digital host referenced in Figure 27 in this case is the FPGA fabric which means it is responsible for generating both the clk signal and the convert signal shown in the Figure. Then the FPGA fabric is responsible for accurately receiving the data from the ADC on its output SDO line. The key consideration when creating the ADC communication component in the fabric is that the timing requirements set by the ADC must be matched as closely as possible.

Figure 28 is the timing diagram for communication with the MAX11162 ADC. The

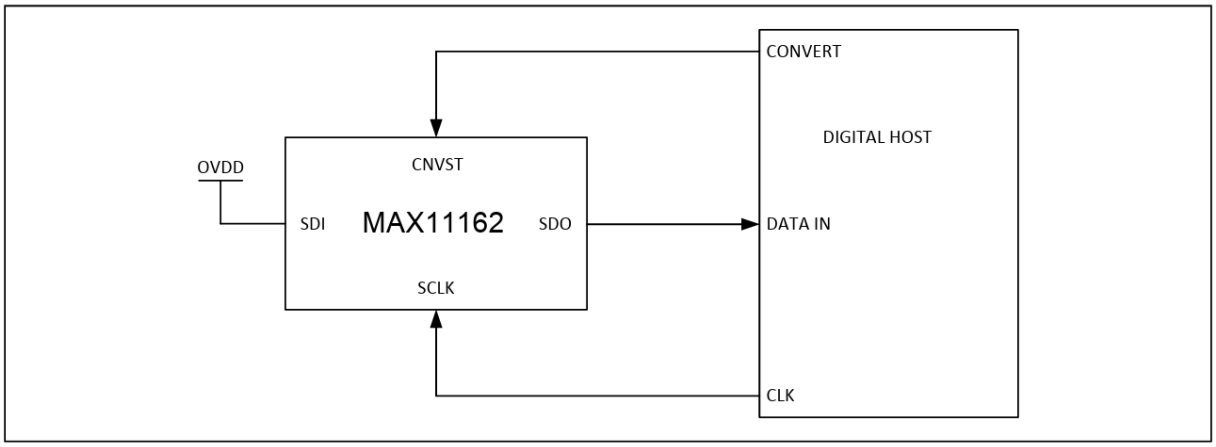


Figure 27: Connection diagram for the 3-wire no busy indicator mode configuration of the MAX11162 ADC (reproduced from [2]).

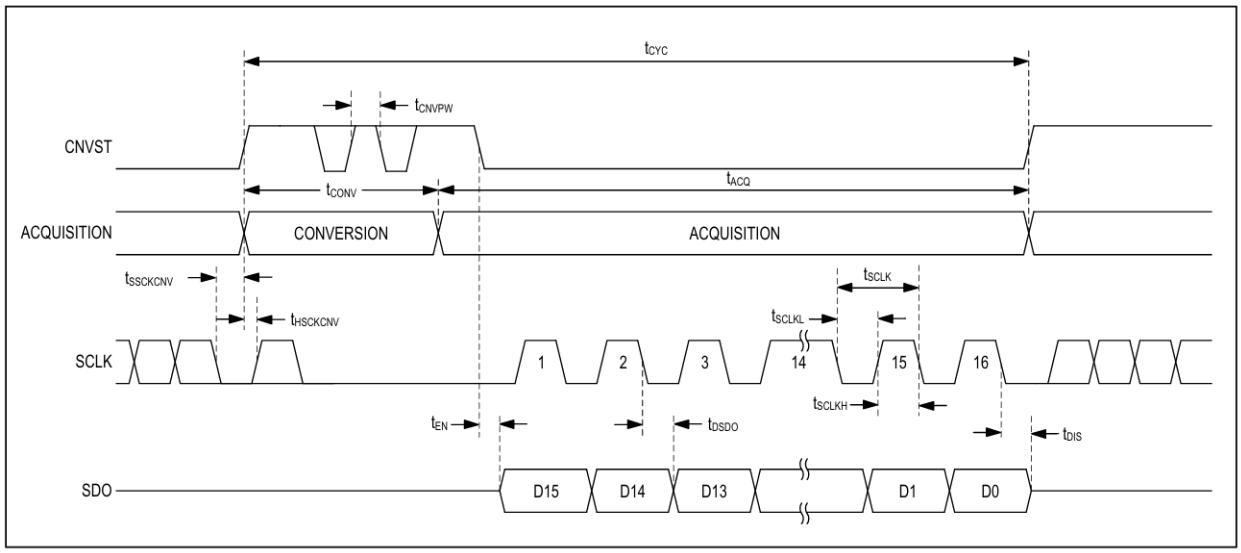


Figure 28: Critical Timing diagram for SPI communication with the MAX11162 ADC (reproduced from [2]).

process to best match the desired signal pattern will be discussed broadly for exact timing requirements refer directly to the data sheet. The FPGA fabric component responsible for communicating with the ADC is named `adc_interface_16bit`. This component is responsible for generating the conversion pulse shown in Figure 28 as well as transmitting the SCLK signal to the ADC and accurately capturing the data returning data on the SDO

line. The entire component functions by simply counting clock edges and then basing all input and output logic on the number of edges counted.

The DE10-Nano has a clock frequency of 50 MHz so to complete a communication cycle the counter would ideally count to 100 to meet the desired 500 KHz conversion rate of the ADC but some issues with the timing found that counting to 108 seemed to get more consistent readings. This results in a communication frequency of 463KHz. The counter process in the component simply counts to the target of 108 and resets back to 0 and the other processes in the component base their logic on the inputs as well as the count generated by the counter process.

The CONVST signal in Figure 28 is generated by a process in the component called `CONVST_Pulse_gen`. The logic simply sets the CONVST pulse output signal high when the components count variable is less than 75 and low otherwise.

The SCLK signal from Figure 28 was generated by a process called `SCK_Pulse_gen`. The process checks if counts is greater than the 75 from the previous process and then if that is true is increases a counter to keep track of the number of pulses sent out on the SCK line and inverts the current signal on the SCK signal. The logic inverting the signal creates clock pulses that trigger the ADC to transition through the resulting conversion bits.

The next process in the component is the `Store_SDO_data` which is responsible for storing the converted ADC value that is transmitted by the ADC on the SDO line in Figure 28. The process utilizes a signal generated in the `SCK_Pulse_gen` process that keeps a count of the number of SCK edges the FPGA has generated thus far. Then based on the counted of edges the value on the SDO line is stored in the correct location in the SDO vector for use in the feedback controller. Once a full 16 bits has been stored by the component a signal called `data_ready` is set to high to signal the system utilizing the ADC communication component that the data in the data signal is ready for use.

Motor Angle Calibration To properly implement FOC on the stepper motor as the startup of the system the electrical angle of the motor must be set to a known angle. Knowing the motor electrical angle is crucial because the Park transform requires the electrical angle in order to correctly perform its coordinate transform. This has posed a problem both in the FPGA implementation and the Raspberry Pi implementation of the control system. A process to calibrate the angle before enabling the FOC controller was developed and performed by a component in the fabric called `Stepper_Motor_Electrical_Angle_Calibration`.

This component was not meant to be the final solution it was only intended to be used for testing during the initial phases of the FPGA based system. This system has some complex code but the functionality is relatively simple. The component is intended to be used at the startup of the motor to move the motor to known electrical angle before allowing the FOC control to take over. This component accomplished that by driving the afore mentioned PWM control with a roughly 50% duty cycle and iterating through the coils in series. The duty cycle percentage was limited to 50% to limit the current driving the motors thus preventing motor winding damage.

Turning the coils on in this order: A, B, A', B', will eventually result in the motors rotor aligning with the one of the phases when it is powered then the series of phases is continued until the motor is aligned with phase A. Aligning the rotor with phase A sets the electrical angle to zero so the FOC controller can take over from that known angle. This component also tracks which phase it is powering and will notify the end user that the systems angle is set to zero degrees as well as what phase is currently being powered. This is done by counting the phases in order and displaying the count on LEDs as well as setting a separate LED high when the angle is set to zero. The functionality of the system is that when the user presses a button on the Development board the next phase in the series is powered and the corresponding number for that phase is displayed as a binary count on two LEDs. For example when the system is powering B', the count LEDs would have two lights on which

corresponds to 3 in binary then if the button is pressed it would switch to powering phase A, display zero on the count LEDs, and set the zero indicator LED on.

Feedback Control Design

The feedback control for this implementation is the same overall architecture as the cascaded PID design discussed in the Raspberry Pi implementation section. The key difference in control design is how the controller was created and implemented. The creation process for this design began in Simulink and utilized the VHDL toolbox and generation tools to create the desired controller as well as simulate the controllers ability to control the stepper motor. The Simulink blocks utilized and their settings will be the primary focus of this section.

Simulink Model

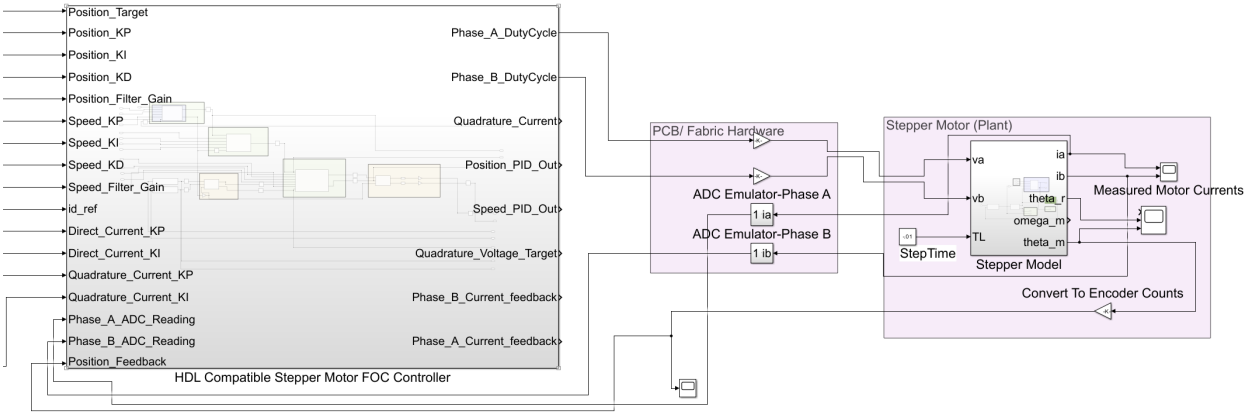


Figure 29: Simulink controller block with blocks modeling PCB and fabric hardware as well as the stepper motor control plant model

Figure 29 depicts the stepper motor control block created in Simulink utilizing HDL compatible blocks. The inputs shown on the left of the block are all intended to be register controlled values via the AXI communication bus once the design in implemented on the FPGA. These values are primarily the individual PID controller block parameters such as

gains and filtering constants. A key parameter that is register controlled is the `id_ref` signal which is the direct current output value. The functionality of this signal will be discussed in the following sections as the specifics of each block that makes up the larger controller are discussed.

The right portion of Figure 29 in the pink blocks labeled `Stepper Motor (Plant)` and `PCB/Fabric Hardware` simulate the stepper motor plant and the hardware utilized to interface with the motor. The inclusion of these blocks allows for verification of the controllers functionality prior to implementation on the FPGA by performing simulation in the Simulink workspace. In order to simulate the design and ensure that the block can be implemented in the fabric of the FPGA the simulation solver must be set discrete mode and all data types must be fixed point values or integers.

The feedback signals from the plant are shown on the lower left of the main controller block. These feedback signals are labeled `Phase_A_ADC_Reading`, `Phase_B_ADC_Reading`, and `Position_Feedback`. The `Position_Feedback` value is set to be a standard integer value when it enters the main controller block because that is the resulting output of the encoder counter component discussed in the electronic hardware section. This integer value is scaled utilizing Equation 27 solved for the encoder counts (C) to ensure that the integer value matches the value that would be returned by the custom component when it is interfaced with the physical rotary encoder. The constant 20,000 is specific to the encoder mounted to hybrid stepper motor. Essentially, 20,000 counts corresponds to a full mechanical revolution of the motor shaft. This number is specific to the encoder itself.

$$\theta_{\text{mech}} = \frac{2\pi}{20000} C \tag{27}$$

The individual blocks that make up the top level stepper motor control block are shown

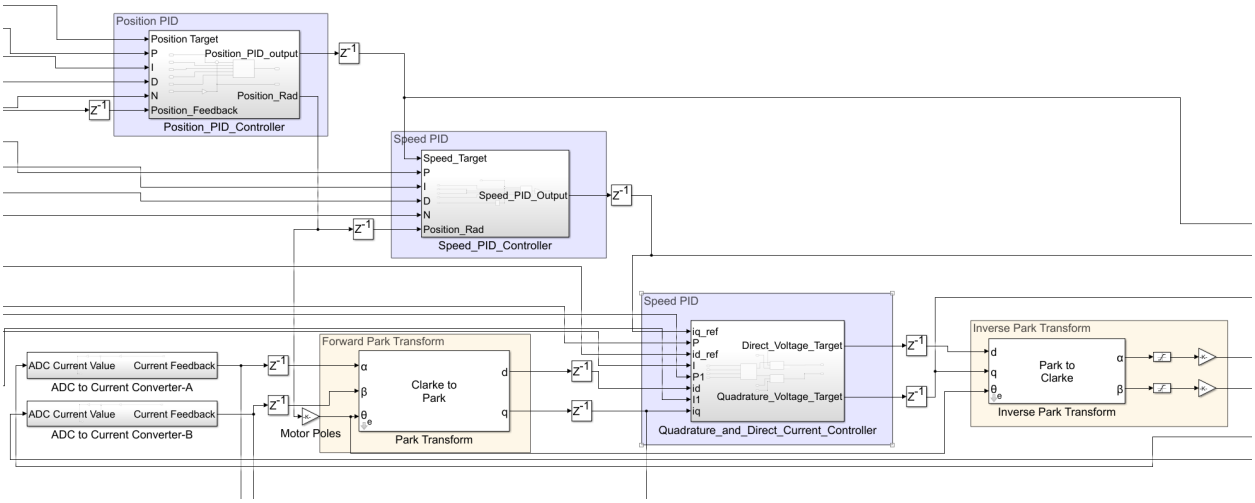


Figure 30: Internal blocks to the stepper motor feedback controller including PID blocks, coordinate transform blocks and other required computation blocks.

in Figure 30. The control scheme is identical to the previous iteration and therefore contains three individual PID controllers cascaded.

Position PID controller The first PID controller in series is responsible for performing feedback control on the position of the stepper motor. The block preceding the `Position PID Controller` block is responsible for converting the integer encoder count value returned by the `encoder_counter_zeroable` in to radians by performing the computation in Equation 27. This feedback signal is then subtracted from the user input mechanical position target to create the error signal fed in to the PID controller. The settings for the PID controller must be set correctly to maintain the desired variable type output. The saturation limits for the `Position PID Controller` were set to -2π and 2π which essentially sets a maximum rotational speed target to -2π rad/s and 2π rad/s. Figure 31 shows some of the settings used for the `Position PID Controller`. Most values in 31 are set to same as first input but directly setting the `Integrator output:` and `Filter output:` to `fixdt(1,16,8)` prevented the output value from the PID controller changing to a non-HDL compatible data type. The controller is configured to operate in discrete mode, where the PID algorithm is

implemented in the z-domain as shown in Equation 28. In this form, the controller output $U(z)$ is expressed as a function of the error signal $E(z)$ and consists of proportional, integral, and derivative components.

The proportional term K_p scales the instantaneous error, while the integral term $\frac{K_i T_s}{z-1}$ accumulates the error over time to eliminate steady-state error. The derivative term $\frac{K_d N}{1+\frac{NT_s}{z-1}}$ represents a filtered discrete derivative, where the coefficient N is used to limit high-frequency noise amplification.

In this formulation, T_s is the controller sampling period, K_p, K_i, and K_d are the proportional, integral, and derivative gains, respectively, and z is the discrete-time complex variable. At each sampling instant, the controller computes the output signal based on the current and past values of the error signal.

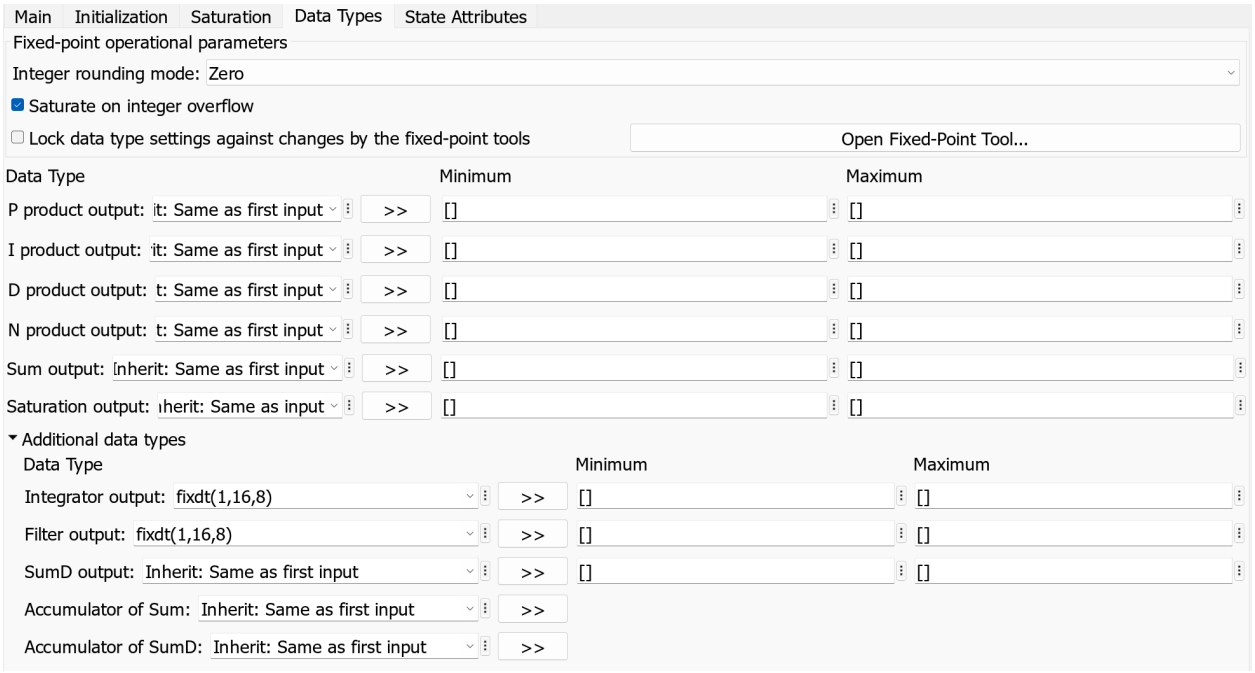


Figure 31: Position PID data type settings in the Simulink block.

$$U(z) = \left(K_p + \frac{K_i T_s}{z-1} + \frac{K_d N}{1+\frac{NT_s}{z-1}} \right) E(z) \tag{28}$$

Speed PID controller The following PID controller in the series is the `Speed PID Controller`, which has its target signal set by the output of the previous PID controller. The block that takes in the position feedback signal and feeds the summing junction before entering the `Speed PID Controller` computes a low-pass filtered derivative of the motor's rotational position θ, producing an estimate of the angular velocity $\dot{\theta}$. The discrete-time computation used to obtain this filtered derivative is shown in Equations 29 and 30. After exiting the discrete derivative block, the signal is fed into the summing junction and is subtracted from the speed target set by the previous PID block. The settings for this PID block and the previous are the same, with the only differences being the user-defined PID gain values and the saturation limits.

$$x[k] = x[k-1] + 120KT_s\left(e[k-1] - x[k-1]\right) \tag{29}$$

$$u[k] = 120\left(e[k] - x[k]\right) \tag{30}$$

In equations 29 and Equation 30, $x[k]$ represents the internal state of the filter, $u[k]$ is the estimated derivative output, $e[k]$ is the input signal, T_s is the sampling period, and K is a tuning parameter that determines the cutoff frequency of the filter.

Current PID Controller and FOC blocks The last controller in the cascaded system is the `Current PID Controller`. This blocks feedback signal is first passed through ADC conversion block to determine the current flowing in phase A, and B of the stepper motor. This block takes in the 16 bit ADC value produced by the component discussed in the ADC section of this implementation, and converts it in to an accurate current measurement. This is done by first computing the voltage that corresponds with the value returned by the ADC and then accounting for the offset applied by the shunt amplifier discussed previously in

the PCB section. Then utilizing the known resistance value of the shunt resistor and the constant gain of the shunt amplifier the current flowing through the resistor and therefore its associated motor phase is determined. This block is utilized to determine the current in both phase A and phase B of the stepper motor. The current values are then passed in to the `Park Transform block` and converted in to the quadrature and direct currents, i_q and i_d. The quadrature current is then passed in to the summing junction and subtracted from the reference current signal set by the `Speed PID Controller`. The resulting error signal is then fed in to the PI controller. This block utilizes a PI controller instead of a PID controller as is common for controllers responsible for current control

After exiting the current control block the output current target is passed in to the `Inverse Park Transform` block and transformed back in to the two-phase coordinate system. This value must then be transformed a duty cycle to be fed in to the PWM generator discussed in the PWM controller section. The output of the Park Transform block is treated like a voltage and then gets divided by the motor control voltage to determine the percentage of full motor voltage that should be produced by the system. Equation 31 represents this calculation and is solved for the duty cycle percentage. To match the input of the PWM generator the final signal is a signed 20 bit fixed point value with 18 bits after the decimal point.

$$d = \frac{v_{\text{ref}}}{V_{dc}} \tag{31}$$

Digital Control

The digital control on this system was performed by the DE10-Nano SoC FPGA development board. The FPGA in question has a base clock speed of 50 MHz which was

utilized as the primary clock for the majority of the instantiated hardware components. The FPGA based system was setup such that the control architecture as well as the components that interface with the control plant were implemented in the FPGA fabric. The SoC portion was utilized to tune the feedback controller during operation as well as monitor the state of the controller.

Component Wrapper

Once the stepper motor controller has been generated and output by the Simulink HDL generation tool, the HDL must be instantiated in a wrapper file. The two primary purposes of the wrapper file are to properly connect the components generated using Simulink to the custom components and to connect the components to the AXI bus. Connecting the components to the AXI bus is what allows the CPU direct memory access to the custom fabric components. In this design, the primary use of these control registers is to set the motors position target and adjust the PID gain values inside the controller.

The reset logic for the controller was implemented such that when the motor FOC controller component is held in reset the motor angle calibration component is enabled and vice versa. This was done by simply giving each component the same reset and applying an inverse to the signal when connecting it to the controller component. This reset was tied to a manual switch on the DE10-Nano board to allow the user to select which component was enabled.

Logic was also implemented in the wrapper file so that the PWM duty cycle that was fed in to the PWM components was generated by the angle calibration component when it was enabled and by the FOC controller component when it was not.

A data handler process was also instantiated in the wrapper file to determine when to retrieve the ADC data based on the `data_ready` signal discussed in the ADC communication section above.

Register Control Values The standard data type for the system in Simulink as well as in the wrapper file is a fixed point data type with 16 total bits and 8 of them fractional. This affects how the register controlled values are placed on the AXI bus in the wrapper file. The standard word size for the bus is 32 bits meaning that when the register controlled values are written or read by the CPU system they values must be truncated. When the CPU reads from the registers, the most significant 16 bits of the value returned by the fabric component is padded with zeros to account for the bus size. When a write occurs to those values,the 32 bits is transferred to the memory location but only the lower 16 bits of the full word have an affect on the system. The exception to this is that when a write to the position feedback register occurs, the wrapper utilizes the zeroing capability of that component to reset the position feedback signal. Table 6 lists the registers, whether they are read/write, their size, and a description of their functionality.

The data type size for the signals is primarily set in Simulink and adjusting the size and precision of the signal directly affects the amount of FPGA resources utilized by the design. The primary concern when utilizing the DE10-Nano board is utilization of the DSP blocks as the development board has a relatively small number of DSP blocks compared to other FPGAs. The exact value is shown in Table 7. Additionally, a feedback control design inherently requires several arithmetic operations and thus utilizes a high number of DSP resources.

DE10-Nano Development Board

The DE10-Nano development boards utilizes a Cyclone 5 SE SoC FPGA along with various other peripheral devices. The Cyclone 5 SoC FPGA includes an ARM Cortex-A-9 processor paired with the FPGA fabric. The resources available to the DE10-Nanos Cyclone 5 SoC FPGA are shown in Table 7.

Using the development board minimizes development time as it is already designed to

Table 6: Avalon-MM register map for `Stepper_FOC_Control_hps`

Rigister	R/W, Bits	Signal Name	Description
0	R/W,16	Position_Target	Desired position command input
1	R/W,16	Position_KP	Proportional gain for position controller
2	R/W,16	Position_KI	Integral gain for position controller
3	R/W,16	Position_KD	Derivative gain for position controller
4	R/W,16	Position_Filter_Gain	Derivative filter gain for position loop
5	R/W,16	Speed_KP	Proportional gain for speed controller
6	R/W,16	Speed_KI	Integral gain for speed controller
7	R/W,16	Speed_KD	Derivative gain for speed controller
8	R/W,16	Speed_Filter_Gain	Derivative filter gain for speed loop
9	R/W,16	Direct_Current_KP	d-axis current proportional gain
10	R/W,16	Direct_Current_KI	d-axis current integral gain
11	R/W,16	Quadrature_Current_KP	q-axis current proportional gain
12	R/W,16	Quadrature_Current_KI	q-axis current integral gain
13	R,16	Phase_A_ADC	Measured phase A current from ADC
14	R,16	Phase_B_ADC	Measured phase B current from ADC
15	R,20	Phase_A_Duty_Cycle	PWM duty cycle output for phase A
16	R,20	Phase_B_Duty_Cycle	PWM duty cycle output for phase B
17	R/W,32	id_ref	Direct-axis current reference
18	R/W,32	Position_Feedback	Encoder feedback; write triggers zeroing
19	R/W,32	Global_Period	PWM switching period
20	R,32	Quadrature_Current	q-axis current feedback
21	R,16	Position_PID_Out	Position controller output
22	R,16	Speed_PID_Out	Speed controller output
23	R,16	Quadrature_Voltage_Target	q-axis voltage command
24	R,32	Phase_A_Current_feedback	Filtered phase A current
25	R,32	Phase_B_Current_feedback	Filtered phase B current

Table 7: FPGA Resources of DE10-Nano (Cyclone V SE)

Resource	Value
FPGA Family	Intel Cyclone V SE SoC
Logic Elements (LEs)	110,000
DSP Blocks	112
Block RAM	5.6 Mb
UltraRAM	N/A
PLLs	6
User I/O Pins	145

be a fully functioning SoC FPGA system capable of booting an OS as well as interacting with the onboard memory and peripherals. A top view of the physical board is shown for reference in Figure 32. The GPIO banks on the top and bottom of the image shown in Figure 32 allow easy access to pins connected directly to the onboard FPGA fabric, which is ideal for interfacing the FPGA outputs and inputs directly with the physical hardware of the plant.

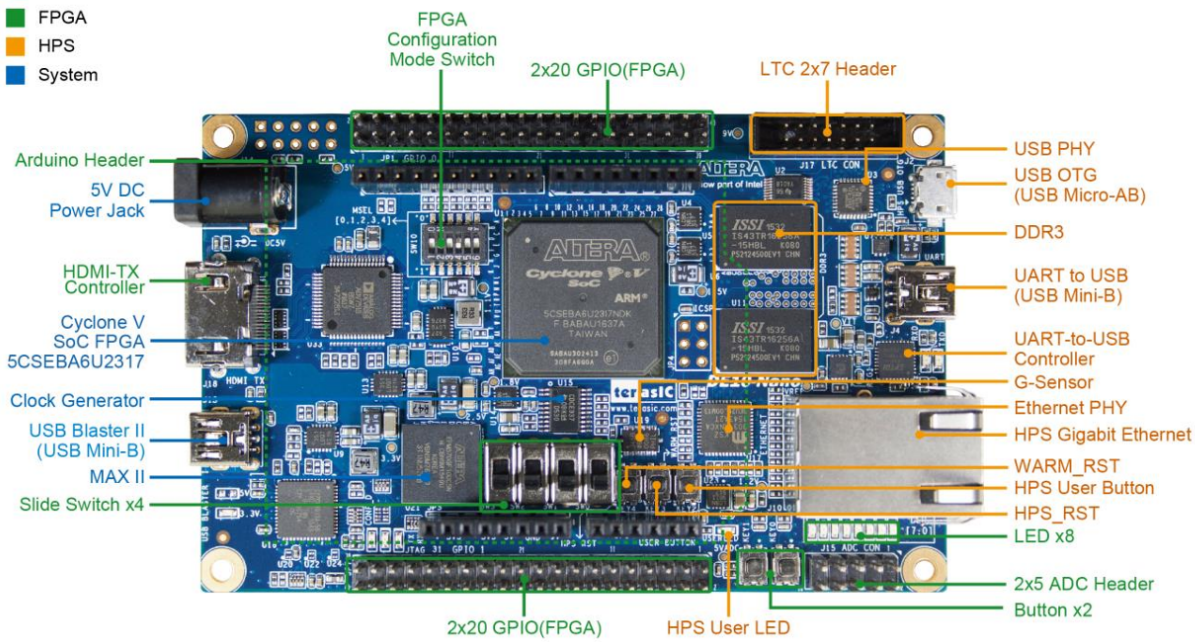


Figure 32: Image of the DE10-Nano development board with connections and key components labeled. [10]

Quartus Prime software and top level file

Quartus prime is the software used to generate the bit-stream that is loaded on to the FPGA and provides the information required to correctly configure the FPGA.

In Quartus, a top level file must be created that connects the output signals of the internal components to the pin signals on the specific board. This is where for the FOC stepper motor controller design the signals are connected to the GPIO pins in Figure 32.

Platform Designer Once the wrapper file is created the file can be uploaded to the Quartus prime software and placed in a tool called Platform Designer. Platform designer automatically detects the AXI bus signals in the component as long as they are named in accordance with the standard naming. Platform Designer then allows the user to connect the various AXI bus related signals via a GUI as well as instantiate PLLs (Phase-Locked loop) to generate different clock frequencies. Figure 33 shows the connections routed for the controller design.

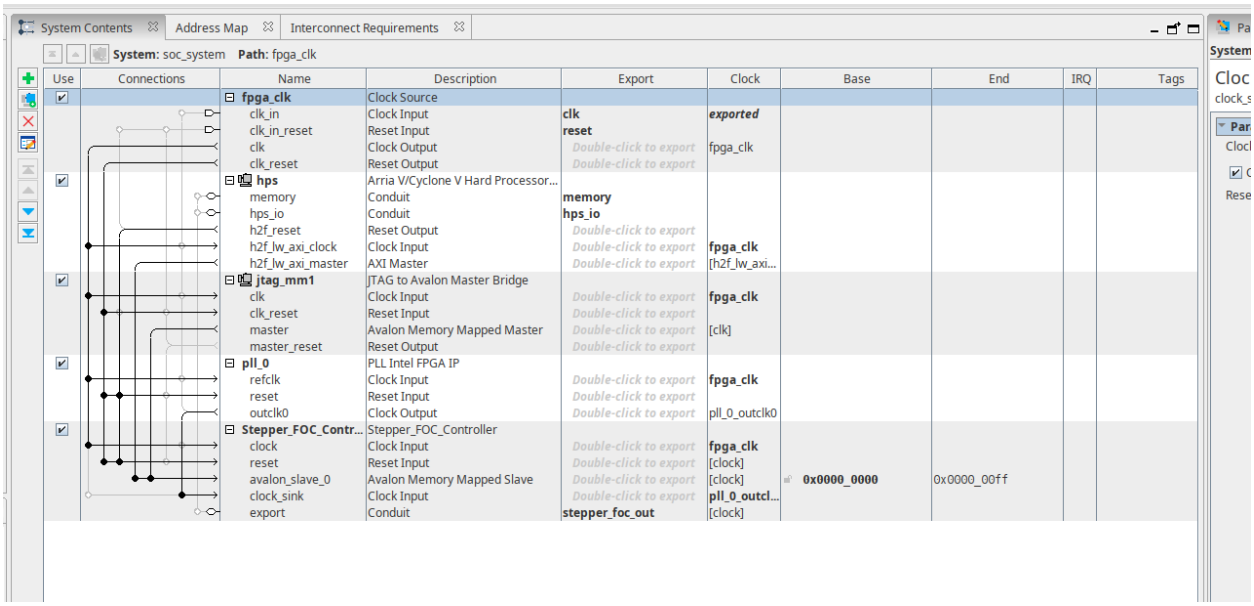


Figure 33: Quartus Platform designer system connections.

The registers were connected to the AXi bus and connected with a JTAG memory

mapped master to enable debugging directly from a command console integrated in to the Quartus Prime software. Additionally, a PLL was utilized to generate a lower frequency clock than the base clock frequency of the DE10-Nano which is 50 MHz. The PLL included in the Platform Designer layout reduced the frequency to 2 MHz and then utilizing a counter internal to the wrapper file it was reduced to the desired 500 KHz clock frequency. This is the same frequency that the Simulink model in Figure 29 was simulated at and generated with in the HDL generation tool.

Once the system is connected correctly in Platform Designer, a output file can be generated to be incorporated in to the main component structure of Quartus. This works similarly to an include file because it contains information for linking all the necessary components of the design.

Implementation Conclusion

The custom FPGA fabric components were successfully integrated with the rest of the system generated in Simulink. Additionally, the custom components such as the `adc_interface_16bit` and the `encoder_counter_zeroable` were successfully able to interface with the hardware of the control plant. Logic was implemented to handle calibrating the electrical angle of the stepper motor so that the FOC controller could have a correct initial electrical angle.

Blocks were created in Simulink to correctly format the input and output signals from the hardware with the Simulink model. The initial cascaded controller was created and tuned in the Simulink and then transformed in to HDL utilizing the HDL generation tool providing a foundational controller creation process.

The wrapper file successfully linked all of the custom components to the generated Simulink design as well as applied logic to handle synchronization and data type matching. Additionally, the wrapper file allowed the CPU system access to the controller tunable values

and signals useful for monitoring the state of the system.

The cascaded PID design provides a initial baseline for FPGA resource utilization and for how the data sizes affects the resource utilization. This system was the first attempt at placing the stepper motor FOC system on an FPGA and led to several discoveries and provided a basis for future work.

Issues and Future Improvements

The electrical angle is determined utilizing the relative position feedback by the rotary encoder which can cause issues. The potential problem arises if the electrical angle calibration is not correct when the stepper motor controller is enabled then the electrical angle error will persist continuously until a new calibration occurs. When this happens, it is particularly hard to detect as slight errors zeroing the angle will still result in a motor that performs similar to one that was calibrated with zero error. The difference is slight but the calibration error would most likely cause a difference in the quadrature current used for the same motion between a correctly calibrated system and an incorrectly calibrated system. This presents an issue in a machining application because the hope is to minimize quadrature current usage and then estimate cutting force from the quadrature current usage.

The fabric based encoder counter component also assumes an initial state of input and if the initial input does not match the assumed initial input the component will interpret that as a small rotation resulting in a slight error. This could be fixed by implementing more complex logic to account for different startup inputs which was attempted but still had issues.

The wrapper file had to perform several variable type changes to correctly link the system together due to re-use of some of the hardware components from previous designs. Changing these components to better match the data types of the rest of the components would simplify the wrapper file and reduce the potential for errors caused by incorrect

conversions.

During testing the ADC ICs were mounted on a bread board which led to some pretty inconsistent readings from the ADC. In the future the ADC communication should be improved by placing them on an IC with some shielding.

The system did use a significant amount of DSP blocks which does limit the potential with this system for more complex designs. Additionally the cascaded PID system does not efficiently utilize the parallelization of the FPGA because the cascaded system is inherently serial. This did cause some issues with timing closure, which resulted in the inclusion of delay blocks in the Simulink model in order to create a pipeline instead of executing the whole design on a single clock edge. An FPGA with more resources would improve the future proofing of the system and allow for more complex designs.

CONTROLLER ARCHITECTURE AND FPGA RESOURCE UTILIZATION

One of the key advantages of utilizing an FPGA-based controller is the ability to incorporate a wide range of feedback control architectures with minimal impact on computation time. The controller architectures explored in this section include the previously discussed cascaded proportional–integral–derivative (PID) design as well as active disturbance rejection control (ADRC)-based approaches.

Each controller architecture imposes different mathematical and computational requirements, which directly influence the amount of FPGA resources required for implementation within the FPGA fabric. These requirements include variations in arithmetic operations, internal state variables, and data paths, all of which contribute to overall resource utilization.

In conventional CPU-based systems, an increase in controller complexity typically results in an increase in computation time, even when parallel processing techniques are employed. In contrast, FPGA-based systems can exploit hardware-level parallelism to maintain deterministic execution timing. As a result, increases in controller complexity do not significantly affect computation latency, but instead lead to increased utilization of FPGA resources such as multipliers, registers, and logic elements.

The controller architectures were simulated in Simulink in the same manner as the previously discussed cascaded PID control. The Simulink models will be explored as well as the predicted FPGA resource utilization for each feedback control model.

Active Disturbance Rejection Control (ADRC)

Active disturbance rejection control (ADRC) can be viewed as an extension of the previously discussed state-space control methods. ADRC may appear significantly different from the state-space controllers discussed in the background material, but fundamentally ADRC adds an additional state that estimates disturbances. In classical state-space, the

system is represented using known system dynamics; however, this does not account for unknown disturbances or modeling errors. In ADRC, the system is structured such that the highest-order derivative of the output is expressed as a function of the input and a total disturbance term, as shown in Equation 32 [8].

$$y^{(n)}(t) = b_0 u(t) + f(t) \tag{32}$$

ADRC utilizes an extended state observer (ESO), which is structurally equivalent to a Luenberger observer with an additional state representing the total disturbance [8]. Equation 33 describes the observer dynamics, where the additional state represents an estimate of disturbances.

$$\dot{\hat{x}}(t) = A\hat{x}(t) + b\,u(t) + l\left(y(t) - c^T\hat{x}(t)\right) \tag{33}$$

Disturbances can arise from unmodeled dynamics in the plant or from external sources. ADRC enables real-time compensation for disturbances, improving system robustness by reducing reliance on accurate system modeling and allowing the system to compensate for unpredictable operating conditions. The estimated states and disturbances are then used in the control law shown in Equation 34 [8].

$$u(t) = \frac{1}{b_0}\left(k_1 r(t) - k^T\hat{x}(t)\right) \tag{34}$$

Simulink Models

The Simulink models utilized for the resource comparison were made such that they were as similar as possible to each other, so that the resource differences were solely due to feedback controller complexity changes. Additionally, the data types utilized for each design were kept consistent between models with the exception of a few extremely high gains required for the ADRC that exceeded the maximum value that can be represented with standard data size.

The first model still utilizes a PID controller connected with ADRC based blocks. The second model utilizes a higher order ADRC block to replace the cascaded position and speed control with a single block. The combination allows for more parallel computation in comparison to the first order ADRC model, which utilizes the capabilities of the FPGA better.

The controllers were intended to function as drop in replacements for the already made PID controller architecture and thus have the same input and output signals. This also means that the ADRC based models will integrate with the hardware components discussed in the FPGA Development section.

First Order ADRC Model The Simulink model that utilizes the lower order ADRC will be referred to as the first order ADRC model. The controller has the same outputs as the cascaded PID discussed previously and is integrated with the same stepper motor model from that section as well. The higher level architecture of the first order ADRC model is shown in Figure 34.

The structure of the controller shown in Figure 34 is very similar to that of the cascaded PID model discussed in the previous two sections but in this case the speed and current control blocks have been replaced with ADRC based feedback controllers. This means that in the same cascaded way as previously discussed, the reference signal for the speed controller

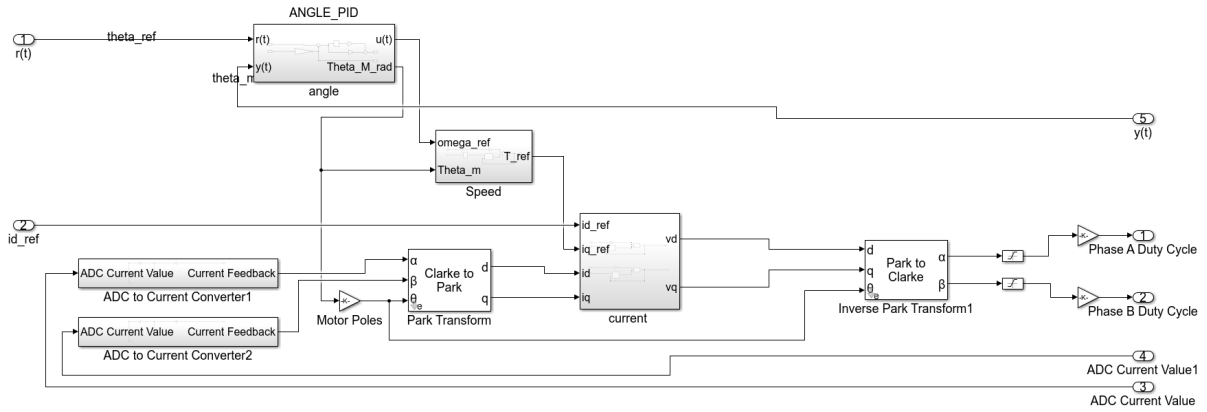


Figure 34: Cascaded ADRC and PID based Simulink stepper motor controller model.

is based on the output of the position controller and the reference for the quadrature current controller is based on the the output of the speed controller.

The internal controller architecture that implements ADRC on the motors speed is shown in Figure 35. This implementation assumes that disturbances affect the measured output signal, and the extended state observer estimates the disturbance term $f(t)$ based on this measurement.

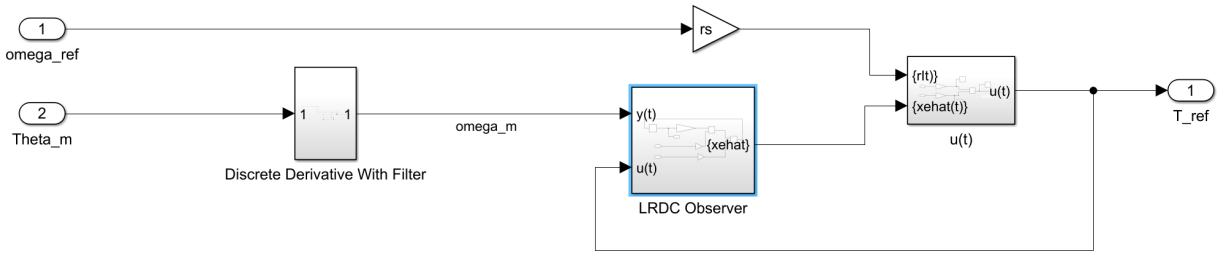


Figure 35: ADRC first order Simulink motor rotational speed controller model.

The LRDC Observer block in Figure 35 utilized the calculation in Equation 33 and estimates both the system states and disturbances. Then the block labeled `u(t)`uses the observer estimated state in combination with the reference state to compute the output signal in accordance with Equation 34.

The same controller architecture from Figure 35 is utilized in the current controller shown in Figure 35 for both the quadrature and direct current. This model was then compiled into an HDL compatible block utilizing the HDL toolbox in Simulink in the same manner as before so that its required computational resources could be compared to the other controllers.

Second Order ADRC Model The second order ADRC model, which is shown in Figure 36, utilizes a higher order observer. The higher order observer allows for the combination of the position and speed controller in to a single block. In the second-order ADRC implementation, the observer estimates position, velocity, and disturbance, whereas the first-order implementation estimates only velocity and disturbance. Increasing the order of the observer increases the number of required arithmetic operations, which directly impacts FPGA resource utilization.

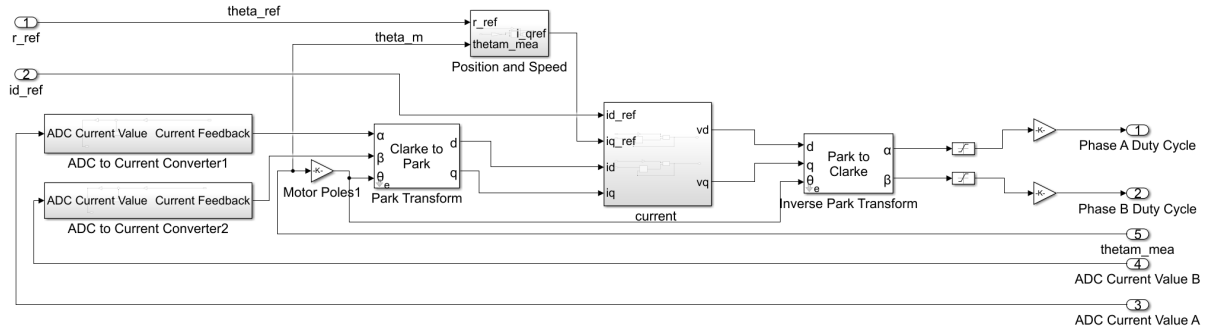


Figure 36: Second order ADRC Simulink model.

Resource Utilization

The three controller architectures being compared are the Cascaded PID Controller, first order ADRC and second order ADRC. The FPGA resource utilization of each controller was collected from the Simulink compilation report so that resource utilization can be

compared across designs. The designs were kept as similar as possible to ensure that the resource utilization differences were primarily due to architectural changes. Comparing the differences provides insight in to how resource utilization correlates with controller architectural complexity.

The resources to be compared include multipliers, adders/subtractors, registers and multiplexers. Multipliers are the most resource intensive components on the FPGA and are commonly the primary concern when trying to minimize resource utilization. These multipliers are of concern in this application primarily as they are utilized heavily in feedback control. Registers are responsible for storing data between calculations or for holding constant values that must be referenced. Adders and subtractors are self explanatory but are important to compare as they are also very common in feedback control architectures.

Table 8 summarizes the FPGA resource utilization reports generated by Simulink after each design was compiled. These provide a relatively accurate estimate for what each design would use when implemented on a physical board but are most useful as a relative comparison to each other. The results provide insight into how increasing controller complexity affects resource utilization.

Table 8: Simulink HDL Resource Comparison of Controller Architectures

Resource	**Second Order ADRC**	**First Order ADRC**	**Cascaded PID**
Multipliers	64	65	36
Adders/Subtractors	73	69	55
Registers	17	18	22
Total 1-bit Registers	320	240	360
Multiplexers	90	79	87

From Table 8 several trends can be observed. The ADRC based controllers require significantly more multipliers and adders/subtractors in comparison to the cascaded PID models. Additionally, the higher order ADRC model does not result in additional multipliers but does require some additional adders/subtractors as well as significantly more registers.

The multiplexers do increase significantly in the higher order ADRC in comparison to the lower order one but is relatively comparable to the cascaded PID controller.

The increase in resources required to implement the ADRC designs is directly related to the increase in required computations needed to implement the observers. The observer calculations along with the state-space controller gains utilized are directly responsible for the increased resource requirements. While the multiplier count does not increase with observer order, other resources such as adders and multiplexers do increase, indicating that higher-order ADRC primarily affects control logic complexity rather than multiplication operations.

Table 9: Estimated Controller Resource Utilization Relative to KR260 FPGA Resources

Resource	**Total**	**Second Order ADRC**	**First Order ADRC**	**Cascaded PID**
Multipliers (DSP Proxy)	1248	64 (5.13%)	65 (5.21%)	36 (2.88%)
Estimated Logic (LUT Eq.)	256 000	2100 (∼0.82%)	1950 (∼0.76%)	1200 (∼0.47%)
Estimated Memory (BRAM)	144	2 (1.39%)	2 (1.39%)	1 (0.69%)

Table 9 provides an estimate of the percentage of the total available DSP slices, logic resources, and block memory each design would require if implemented on the Kria KR260 SoM. The available FPGA resources for the Kria KR260 were collected from the manufacturer's datasheet [1]. The DSP utilization percentages are calculated by assuming that each multiplier reported in Table 8 maps directly to a DSP slice in the final implementation.

The logic utilization is estimated based on the number of arithmetic operations and registers reported by the Simulink HDL Coder. These values are used as a proxy for lookup table (LUT) utilization, as LUTs are the primary resource used to implement combinational logic within the FPGA fabric. Similarly, the reported memory elements are used as an

estimate of block RAM (BRAM) usage. The percentage values for both logic and memory are computed relative to the total available resources on the device.

It should be noted that these values are high-level estimates derived from the HDL generation process and do not directly correspond to post-synthesis resource utilization. A full synthesis and implementation flow would be required to obtain exact FPGA resource usage. The low percentages across the three different designs indicate that there is sufficient headroom to implement more complex designs or instantiate multiple instances of the same controller on a low-cost FPGA such as the KR260.

Implementation Conclusion

The PID controller architecture utilizes fewer resources in comparison to the ADRC controllers however, the robustness of the ADRC controller has advantages in performance. The difference in resource utilization between the higher order ADRC system and the first order ADRC system is minimal proving that resource requirements are primarily dependent on controller architecture. The relatively small difference in multiplier count between the first- and second-order ADRC implementations suggests that multiplier usage is primarily driven by the presence of the observer rather than its order. These results reinforce the concept that, in FPGA-based systems, increased algorithmic complexity is realized as increased hardware utilization rather than increased computation time. The results show that FPGA based feedback control is a viable digital control solution that enables robust and complex feedback control solutions without compromising on timing performance, provided sufficient hardware resources are available.

Future Improvements

All of the control architectures discussed only control a single plant, which in the context of a CNC machine would correspond to a single axis of motion. CNC machines generally

utilize multiple motors as they customarily have multiple axis of motion so either the FPGA must have multiple controllers placed in its fabric or multiple FPGAS must be used to control a single CNC machine. Implementing multiple axis controllers in the fabric of a low-cost FPGA such as the Kria KR260 is viable as a single controller utilized at most 6% of the available resources.

CONCLUSION

Small-scale CNC machines lack precision and repeatability due to reduced structural rigidity; however, advanced feedback control architectures and digital control solutions have the potential to make small-scale machines a viable platform for high-precision machining. Applying FOC with advanced control methods provides the added benefit of potentially estimating force at the cutting edge when integrated with a CNC machine.

Two digital control architectures were developed to implement FOC on a hybrid stepper motor. The initial CPU-based system demonstrated the feasibility of the approach but revealed limitations associated with non-deterministic latency. This motivated the development of an FPGA-based implementation, which provided deterministic timing and improved suitability for real-time control. Both hardware platforms, along with the associated PCB development and Simulink HDL workflow, were explored to support the integration of advanced feedback control strategies.

The Raspberry Pi based implementation was the first prototype developed and relied on a Raspberry Pi computer integrated with a custom PCB to control a hybrid stepper motor with a ball screw drive stage. The system had several electrical hardware related issues but served as an effective jumping off point for future development of PCB hardware. The CPU based control had non-deterministic latency which influenced the decision to swap from a CPU based system with non-deterministic latency to an FPGA based system with deterministic latency.

The FPGA motor control implementation utilized a DE10-Nano SoC development board as the digital control target for FOC stepper motor control system. Several Fabric based components were developed specifically for the implementation including an encoder counter, PWM generators, and various other components. A cascaded PID controller was developed using Simulink and then compiled in to HDL. Then it was integrated utilizing a

system wrapper file and the Quartus Prime development software. The FPGA development process and custom components provide a starting point for developing advanced feedback controls to be implemented in FPGA fabric for the purpose of CNC motor control applications. The FPGA provides a system with deterministic latency and efficient parallel computation which both directly improve the performance of the control system.

FPGA resource utilization was compared across different controller architectures to provide insight in to how controller complexity affects resource utilization. The comparison of FPGA resource utilization across controller architectures demonstrated that increased controller complexity results in higher hardware resource usage rather than increased computational latency. ADRC-based controllers required more multipliers, registers, and logic elements due to the inclusion of observer-based state estimation. However, the increase in resource utilization between first-order and higher-order ADRC implementations was relatively modest, indicating that the primary cost is associated with the inclusion of the observer itself rather than incremental increases in system order.

The work done demonstrates that an FPGA based control system offers a powerful platform for future development of motor control targeted at small-scale CNC machining. The FPGA offers deterministic timing and high capacity for parallel computation which is ideal for advanced control architectures such as ADRC. The parallel architecture and high computational resource density of FPGAs allow additional sensors and more sophisticated control algorithms to be implemented with minimal impact on computation time, with increased complexity instead manifesting itself as higher hardware resource utilization.

Future Work

Fully integrating an FPGA based controller with the motor utilizing a new version of the custom PCB described in the Raspberry Pi section of this thesis would allow for full systems testing and data collection. This was the original intention of the FPGA controller developed

in the FPGA section of this thesis but due to manufacturing delays and development time this was simply not possible in the given time.

Creating a reliable physical test bench with a standardized hardware setup would allow for characterization of performance in practice. This would allow for the comparison of different controller architectures implemented on an FPGA on physical motors and allow for direct comparison between control system performance and resource utilization. Trying various other control architectures to determine what architecture fits the needs of a CNC machine the best would also be interesting and be very beneficial to the overall mission of improving small-scale machining precision. The creation of a reliable test bench would also provide a system for studying the relationship between the quadrature current and the force being applied in opposition to the motors motion.

The system could be made to be more user friendly by fully integrating the CPU OS of the SoC or SoM FPGA with the controller. Essentially, adding the controller to Linux device tree would allow for the system to be controller from user space in Linux. This would simplify system data tracking and analysis by making the critical states of the system such as quadrature current easily accessible and making storing data during testing much simpler.

Applying the control systems developed primarily for the stepper motor to a three-phase motor would allow for performance comparisons to see if the same feedback control architecture that performs well on the stepper would perform equally as well on the BLDC motor.

Migrating all development to the Kria SoM line would make the system more adaptable for future improvements. The Kria SoM line has much more computational resources than the DE10-Nano board which provides overhead for future development of complex controller architectures or for the inclusion of additional sensors. The overhead in computational resources would also allow for multiple instantiations of the same motor controller on the same device which would enable a single SoM to control multiple axis of a CNC machine.

This would enable a single SoM based system to act as a controller for an entire CNC mill or lathe.

REFERENCES CITED

[1] Advanced Micro Devices (AMD). Kria kr260 robotics starter kit product page, 2023. URL https://www.amd.com/en/products/system-on-modules/kria/k26/kr260-robotics-starter-kit.html. Accessed: 2026-04-16.

[2] *MAX11162 16-Bit, 500ksps, Low Power, Single-Channel ADC Datasheet*. Analog Devices (Maxim Integrated), 2019. URL https://www.analog.com/...

[3] Wikimedia Commons contributors. Statefeedback2.png. https://commons.wikimedia.org/wiki/File:Statefeedback2.png. Licensed under CC BY-SA 4.0.

[4] Chung-Wuu Ding and Pi-Cheng Tung. A new approach to field-oriented control that substantially improves the efficiency of an induction motor with speed control. *Applied Sciences*, 15(9):4845, 2025. doi: 10.3390/app15094845.

[5] Bernard Friedland. *Control System Design: An Introduction to State-Space Methods*. Dover Publications, Mineola, New York, 2 edition, 2012.

[6] Mikell P. Groover. *Fundamentals of Modern Manufacturing: Materials, Processes, and Systems*. Wiley, 6 edition, 2016.

[7] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6 edition, 2019.

[8] Gernot Herbst and Rafal Madonski. Active disturbance rejection control: Theoretical perspectives. *arXiv preprint arXiv:1908.04610*, 2019.

[9] Austin Hughes and Bill Drury. *Electric Motors and Drives: Fundamentals, Types and Applications*. Newnes, Oxford, UK, 5 edition, 2019.

[10] Intel Corporation. De10-nano development kit, 2023. URL https://www.intel.com/content/www/us/en/developer/topic-technology/edge-5g/hardware/fpga-de10-nano.html. Accessed: 2026-04-01.

[11] Paul C. Krause, Oleg Wasynczuk, and Scott D. Sudhoff. *Analysis of Electric Machinery and Drive Systems*. Wiley, 3 edition, 2013.

[12] R. Krishnan. *Permanent Magnet Synchronous and Brushless DC Motor Drives*. CRC Press, 2010.

[13] Chiu-Keng Lai, Bo-Wei Lin, Hsiang-Yueh Lai, and Guan-You Chen. Fpga-based hybrid stepper motor drive system design by variable structure control. *Actuators*, 10(6), 2021. ISSN 2076-0825. doi: 10.3390/act10060113. URL https://www.mdpi.com/2076-0825/10/6/113.

[14] Lambtron. Quadrature diagram.png. https://commons.wikimedia.org/wiki/File:Quadrature_Diagram.png, 2019. Public domain (CC0 1.0).

[15] Katsuhiko Ogata. *Modern Control Engineering*. Prentice Hall, 5 edition, 2010.

[16] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Wiley, 10 edition, 2018.

[17] Ross K. Snider. *Advanced Digital System Design using SoC FPGAs: An Integrated Hardware/Software Approach*. Springer, 2023.

[18] Wikipedia contributors. Lathe, 2026. URL `https://en.wikipedia.org/wiki/Lathe`. Accessed: 2026-04-16.

[19] Wikipedia contributors. Milling (machining), 2026. URL `https://en.wikipedia.org/wiki/Milling_(machining)`. Accessed: 2026-04-16.

[20] Wikipedia contributors. Proportional–integral–derivative controller, 2026. URL `https://en.wikipedia.org/wiki/Proportional%E2%80%93integral%E2%80%93derivative_controller`. Accessed: 2026-04-16.

[21] Karl J. Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008.

APPENDIX

FPGA CONTROLLER SOURCE CODE

This appendix contains the key VHDL source files used in the FPGA-based motor control system. The code is organized hierarchically to reflect the system architecture described in Chapter 3. Both the primary modules and the required subcomponents are included where necessary to represent the implemented design.

Top-Level Wrapper

The top-level wrapper integrates the encoder interface, ADC interface, PWM generation, calibration logic, and Avalon-MM communication, while also instantiating the HDL-compatible controller generated from the Simulink design.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;
entity Stepper_FOC_Control_hps is
  port (
    clk   : in std_logic;
    reset : in std_logic;

    controller_clk : in std_logic;
    -- avalon memory-mapped slave interface
    avs_read      : in std_logic;
    avs_write     : in std_logic;
    avs_address   : in std_logic_vector(5 downto 0);
    avs_readdata  : out std_logic_vector(31 downto 0);
    avs_writedata : in std_logic_vector(31 downto 0);

    -- external I/O; export to top-level
    motor_encoder_input : in std_logic_vector(1 downto 0);

    -- pwm outputs
    Phase_A_PWM       : out std_logic := '0';
    Phase_A_Direction : out std_logic := '0';
    Phase_B_PWM       : out std_logic := '0';
    Phase_B_Direction : out std_logic := '0';

    -- Signal for Stepper Calibration
    user_button             : in std_logic                    := '0';
    calibration_enable      : in std_logic                    := '0';
    calibration_zero        : out std_logic                   := '0';
    calibration_phase_count : out std_logic_vector(1 downto 0) := "00";

    calibration_enable_out : out std_logic;

    -- ADC Signals
    Phase_A_SDO    : in std_logic  := '0';
    Phase_A_CONVST : out std_logic := '0';
    Phase_A_SCK    : out std_logic := '0';
    Phase_B_SDO    : in std_logic  := '0';
    Phase_B_CONVST : out std_logic := '0';
```

```
40      Phase_B_SCK     : out std_logic := '0'
41
42    );
43  end entity;
44
45  architecture Stepper_FOC_Control_hps_arch of Stepper_FOC_Control_hps is
46
47    -- Internal Signals
          ------------------------------------------------------------------
48    signal Phase_A_data         : std_logic_vector(15 downto 0) := "
          0000000000000000";
49    signal Phase_B_data         : std_logic_vector(15 downto 0) := "
          0000000000000000";
50    signal Phase_A_data_ready   : std_logic                     := '0';
51    signal Phase_B_data_ready   : std_logic                     := '0';
52    signal Phase_A_ADC_internal : std_logic_vector(15 downto 0) := "
          0000000000000000";
53    signal Phase_B_ADC_internal : std_logic_vector(15 downto 0) := "
          0000000000000000";
54
55    -- Controller Gains
56    signal Position_Target_internal       : std_logic_vector(15 downto 0) :=
          "0000000000000000";
57    signal Position_KP_internal           : std_logic_vector(15 downto 0) :=
          "0000001000000000";
58    signal Position_KI_internal           : std_logic_vector(15 downto 0) :=
          "0000000000000000";
59    signal Position_KD_internal           : std_logic_vector(15 downto 0) :=
          "0000000000000000";
60    signal Position_Filter_Gain_internal  : std_logic_vector(15 downto 0) :=
          "0000000000000000";
61    signal Speed_KP_internal              : std_logic_vector(15 downto 0) :=
          "0000001000000000";
62    signal Speed_KI_internal              : std_logic_vector(15 downto 0) :=
          "0000000000000000";
63    signal Speed_KD_internal              : std_logic_vector(15 downto 0) :=
          "0000000000000000";
64    signal Speed_Filter_Gain_internal     : std_logic_vector(15 downto 0) :=
          "0000000000000000";
65    signal Quadrature_Current_KP_internal : std_logic_vector(15 downto 0) :=
          "0000001000000000";
66    signal Quadrature_Current_KI_internal : std_logic_vector(15 downto 0) :=
          "0000000000000000";
67    signal Direct_Current_KP_internal     : std_logic_vector(15 downto 0) :=
          "0000000000000000";
68    signal Direct_Current_KI_internal     : std_logic_vector(15 downto 0) :=
          "0000000000000000";
69
70    -- Hardware Inital values
71    signal id_ref_internal                                : std_logic_vector(31
          downto 0) := "00000000000000000000000000000000";
```

```
72  signal Global_Period                                    : unsigned(31 downto
        0)          := "00000001000000000000000000000000";
73  signal Position_Target_internal_int                     : integer
                            := 0;
74  signal Position_Feedback_internal_int                   : integer
                            := 0;
75  signal Phase_A_Duty_cycle_internal_int                  : integer
                            := 0;
76  signal Phase_B_Duty_cycle_internal_int                  : integer
                            := 0;
77  signal Phase_A_Duty_cycle_calibration_internal_int      : integer
                            := 0;
78  signal Phase_B_Duty_cycle_calibration_internal_int      : integer
                            := 0;
79  signal Phase_A_Duty_cycle_signed                        : signed(19 downto 0)
                  := "00000000000000000000";
80  signal Phase_B_Duty_cycle_signed                        : signed(19 downto 0)
                  := "00000000000000000000";
81  signal encoder_zero_internal                            : std_logic
                          := '0';
82  signal encoder_zeroed_internal                          : std_logic
                          := '0';
83  signal calibration_enable_n                             : std_logic
                          := '0';
84
85  -- Registers with read access
86  signal Phase_A_Duty_Cycle_internal         : std_logic_vector(31 downto
        0) := "00000000000000000000000000000000";
87  signal Phase_B_Duty_Cycle_internal         : std_logic_vector(31 downto
        0) := "00000000000000000000000000000000";
88  signal Position_Feedback_internal          : std_logic_vector(31 downto
        0) := "00000000000000000000000000000000";
89  signal Quadratrue_Current_internal         : std_logic_vector(31 downto
        0) := "00000000000000000000000000000000";
90  signal Position_PID_Out_internal           : std_logic_vector(31 downto
        0) := "00000000000000000000000000000000";
91  signal Speed_PID_Out_internal              : std_logic_vector(31 downto
        0) := "00000000000000000000000000000000";
92  signal Quadrature_Voltage_Target_internal  : std_logic_vector(31 downto
        0) := "00000000000000000000000000000000";
93  signal Phase_A_Current_feedback_internal   : std_logic_vector(31 downto
        0) := "00000000000000000000000000000000";
94  signal Phase_B_Current_feedback_internal   : std_logic_vector(31 downto
        0) := "00000000000000000000000000000000";
95
96  -- Signals for clock gen process
97  signal edge_count    : integer range 0 to 2 := 0;
98  signal clock_divided : std_logic            := '0';
99  component HDL_Compatible_Stepper_Motor_FOC_Controller is
100   port (
101     clk                       : in std_logic;
```

```
102         reset                    : in std_logic;
103         clk_enable               : in std_logic;
104         Position_Target          : in std_logic_vector(15 downto 0); --
                sfix16_En8
105         Position_KP              : in std_logic_vector(15 downto 0); --
                ufix16_En8
106         Position_KI              : in std_logic_vector(15 downto 0); --
                ufix16_En8
107         Position_KD              : in std_logic_vector(15 downto 0); --
                ufix16_En8
108         Position_Filter_Gain     : in std_logic_vector(15 downto 0); --
                ufix16_En8
109         Speed_KP                 : in std_logic_vector(15 downto 0); --
                ufix16_En8
110         Speed_KI                 : in std_logic_vector(15 downto 0); --
                ufix16_En8
111         Speed_KD                 : in std_logic_vector(15 downto 0); --
                ufix16_En8
112         Speed_Filter_Gain        : in std_logic_vector(15 downto 0); --
                ufix16_En8
113         id_ref                   : in std_logic_vector(15 downto 0); --
                sfix16_En8
114         Direct_Current_KP        : in std_logic_vector(15 downto 0); --
                ufix16_En8
115         Direct_Current_KI        : in std_logic_vector(15 downto 0); --
                ufix16_En8
116         Quadrature_Current_KP    : in std_logic_vector(15 downto 0); --
                ufix16_En8
117         Quadrature_Current_KI    : in std_logic_vector(15 downto 0); --
                ufix16_En8
118         Phase_A_ADC_Reading      : in std_logic_vector(15 downto 0); --
                uint16
119         Phase_B_ADC_Reading      : in std_logic_vector(15 downto 0); --
                uint16
120         Position_Feedback        : in std_logic_vector(31 downto 0); --
                int32
121         ce_out                   : out std_logic;
122         Phase_A_DutyCycle        : out std_logic_vector(19 downto 0); --
                sfix20_En18
123         Phase_B_DutyCycle        : out std_logic_vector(19 downto 0); --
                sfix20_En18
124         Quadrature_Current       : out std_logic_vector(31 downto 0); --
                sfix32_En21
125         Position_PID_Out         : out std_logic_vector(15 downto 0); --
                sfix16_En8
126         Speed_PID_Out            : out std_logic_vector(15 downto 0); --
                sfix16_En8
127         Quadrature_Voltage_Target : out std_logic_vector(15 downto 0); --
                sfix16_En8
128         Phase_B_Current_feedback  : out std_logic_vector(31 downto 0); --
                sfix32_En21
```

```
129       Phase_A_Current_feedback  : out std_logic_vector(31 downto 0) -- sfix32_En21
130     );
131   end component;
132
133   component Stepper_Motor_Electrical_Angle_Calibration is
134     port (
135       clk          : in std_ulogic;
136       rst          : in std_ulogic;
137       user_button  : in std_ulogic;
138       enable       : in std_ulogic;
139
140       Phase_A_Duty_Cycle : out integer                      := 0;
141       Phase_B_Duty_Cycle : out integer                      := 0;
142       Phase_Count        : out std_logic_vector(1 downto 0) := "00";
143       zero               : out std_logic                    := '0'
144     );
145   end component;
146
147   component encoder_counter_zeroable is
148     port (
149       clk            : in std_ulogic;
150       rst            : in std_ulogic;
151       A              : in std_ulogic;
152       B              : in std_ulogic;
153       zero           : in std_ulogic;
154       zero_confirmed : out std_ulogic;
155       counts         : out integer
156     );
157   end component;
158
159   component Full_bridge_single_output_driver is
160
161     generic (
162       W_PERIOD : integer := 30
163     );
164     port (
165       clk : in std_logic;
166       rst : in std_logic;
167       -- PWM repetition period in milliseconds;
168       -- datatype (W.F) is individually assigned
169       period : in unsigned(W_PERIOD - 1 downto 0);
170       -- PWM duty cycle between [0 1]; out-of-range values are hard limited
171       -- datatype (W.F) is individually assigned;
172       duty_cycle : in integer;
173       enable     : out std_logic;
174       direction  : out std_logic
175     );
176   end component;
177
```

```
178   component adc_interface_16bit is
179     generic (
180       sys_clock : time := 20 ns
181     );
182     port (
183       clk        : in std_logic;
184       rst        : in std_logic;
185       enable     : in std_logic := '0';
186       SDO        : in std_logic;
187       CONVST     : out std_logic := '0';
188       SCK        : out std_logic;
189       data       : out std_logic_vector(15 downto 0) := "0000000000000000"
          ;
190       data_ready : out std_logic
191     );
192   end component;
193 begin
194
195   -- Invert enable switch
196   calibration_enable_n <= not calibration_enable;
197
198   counter_process : process (controller_clk, edge_count, reset)
199   begin
200     if reset = '1' then
201       clock_divided <= '0';
202       edge_count    <= 0;
203     elsif rising_edge(controller_clk) then
204       if edge_count = 1 then
205         clock_divided <= not clock_divided;
206         edge_count    <= 0;
207       else
208         edge_count <= edge_count + 1;
209       end if;
210     end if;
211   end process;
212   Motor_Controller : HDL_Compatible_Stepper_Motor_FOC_Controller
213   port map
214   (
215     clk                       => clock_divided,
216     reset                     => not calibration_enable_n, -- This is so
          the electrical angle calibration and the controller don't run at
          the sme time
217     clk_enable                => '1',
218     Position_Target           => Position_Target_internal(15 downto 0),
219     Position_KP               => Position_KP_internal(15 downto 0),
220     Position_KI               => Position_KI_internal(15 downto 0),
221     Position_KD               => Position_KD_internal(15 downto 0),
222     Position_Filter_Gain      => Position_Filter_Gain_internal(15 downto
          0),
223     Speed_KP                  => Speed_KP_internal(15 downto 0),
224     Speed_KI                  => Speed_KI_internal(15 downto 0),
```

```
225     Speed_KD                    => Speed_KD_internal(15 downto 0),
226     Speed_Filter_Gain           => Speed_Filter_Gain_internal(15 downto 0),
227     id_ref                      => id_ref_internal(15 downto 0),
228     Direct_Current_KP           => Direct_Current_KP_internal(15 downto 0),
229     Direct_Current_KI           => Direct_Current_KI_internal(15 downto 0),
230     Quadrature_Current_KP       => Quadrature_Current_KP_internal(15 downto
            0),
231     Quadrature_Current_KI       => Quadrature_Current_KI_internal(15 downto
            0),
232     Phase_A_ADC_Reading         => "1000000000000000", -- Hard coded to zero
             current flowing in the motor
233     Phase_B_ADC_Reading         => "1000000000000000",
234     Position_Feedback           => "00000000000000000000000000000000", --
            Hard coded to zero so i can turn ball screw and generate error
235     ce_out                      => open,
236     Phase_A_DutyCycle           => Phase_A_Duty_Cycle_internal(19 downto 0),
237     Phase_B_DutyCycle           => Phase_B_Duty_Cycle_internal(19 downto 0),
238     Quadrature_Current          => Quadratrue_Current_internal,
239     Position_PID_Out            => Position_PID_Out_internal(15 downto 0),
240     Speed_PID_Out               => Speed_PID_Out_internal(15 downto 0),
241     Quadrature_Voltage_Target   => Quadrature_Voltage_Target_internal(15
            downto 0),
242     Phase_A_Current_feedback    => Phase_A_Current_feedback_internal,
243     Phase_B_Current_feedback    => Phase_B_Current_feedback_internal
244
245   );
246
247   Phase_A_ADC : adc_interface_16bit
248   port map
249   (
250     clk         => clk,
251     rst         => reset,
252     enable      => '1',
253     SDO         => Phase_A_SDO,
254     CONVST      => Phase_A_CONVST,
255     SCK         => Phase_A_SCK,
256     data        => Phase_A_data,
257     data_ready  => Phase_A_data_ready
258   );
259
260   Phase_B_ADC : adc_interface_16bit
261   port map
262   (
263     clk         => clk,
264     rst         => reset,
265     enable      => '1',
266     SDO         => Phase_B_SDO,
267     CONVST      => Phase_B_CONVST,
268     SCK         => Phase_B_SCK,
269     data        => Phase_B_data,
270     data_ready  => Phase_B_data_ready
```

```
271   );
272
273   StepperCalibration : Stepper_Motor_Electrical_Angle_Calibration
274   port map
275   (
276     clk                => clk,
277     rst                => reset,
278     user_button        => user_button,
279     enable             => calibration_enable,
280     Phase_A_Duty_Cycle => Phase_A_Duty_cycle_calibration_internal_int,
281     Phase_B_Duty_Cycle => Phase_B_Duty_cycle_calibration_internal_int,
282     Phase_Count        => calibration_phase_count,
283     zero               => calibration_zero
284   );
285
286   motor_counter : encoder_counter_zeroable
287   port map
288   (
289     clk            => clk,
290     rst            => reset,
291     A              => motor_encoder_input(0),
292     B              => motor_encoder_input(1),
293     zero           => encoder_zero_internal,
294     zero_confirmed => encoder_zeroed_internal,
295     counts         => Position_Feedback_internal_int
296   );
297
298   Phase_A_PWM_Gen : Full_bridge_single_output_driver
299   port map
300   (
301     clk        => clk,
302     rst        => reset,
303     period     => Global_Period(29 downto 0),
304     duty_cycle => Phase_A_Duty_Cycle_internal_int,
305     enable     => Phase_A_PWM,
306     direction  => Phase_A_Direction
307   );
308   Phase_B_PWM_Gen : Full_bridge_single_output_driver
309   port map
310   (
311     clk        => clk,
312     rst        => reset,
313     period     => Global_Period(29 downto 0),
314     duty_cycle => Phase_B_Duty_cycle_internal_int,
315     enable     => Phase_B_PWM,
316     direction  => Phase_B_Direction
317   );
318   Phase_A_Duty_cycle_signed <= signed(Phase_A_Duty_Cycle_internal(19
        downto 0));
319   Phase_B_Duty_cycle_signed <= signed(Phase_B_Duty_Cycle_internal(19
        downto 0));
```

```
320
321   data_type_converter : process (clk, reset, Position_Target_internal_int,
          Position_Feedback_internal_int, Phase_A_Duty_cycle_signed,
         Phase_B_Duty_cycle_signed)
322   begin
323     if reset = '1' then
324       Position_Feedback_internal <= "00000000000000000000000000000000";
325
326     elsif (rising_edge(clk)) then
327       Position_Feedback_internal <= std_logic_vector(to_signed(
             Position_Feedback_internal_int, 32));
328       if calibration_enable = '1' then
329         Phase_A_Duty_cycle_internal_int <=
               Phase_A_Duty_cycle_calibration_internal_int;
330         Phase_B_Duty_cycle_internal_int <=
               Phase_B_Duty_cycle_calibration_internal_int;
331       else
332         Phase_A_Duty_cycle_internal_int <= to_integer(
               Phase_A_Duty_cycle_signed);
333         Phase_B_Duty_cycle_internal_int <= to_integer(
               Phase_B_Duty_cycle_signed);
334       end if;
335
336     else
337       null;
338     end if;
339   end process;
340
341   data_handler : process (clk, reset, Phase_A_data_ready,
         Phase_B_data_ready)
342   begin
343     if (reset = '1') then
344       Phase_A_ADC_internal <= "0000000000000000";
345       Phase_B_ADC_internal <= "0000000000000000";
346     elsif (rising_edge(clk)) then
347       if Phase_A_data_ready = '1' then
348         Phase_A_ADC_internal <= Phase_A_data;
349       else
350         Phase_A_ADC_internal <= Phase_A_ADC_internal;
351       end if;
352
353       if (Phase_B_data_ready = '1') then
354         Phase_B_ADC_internal <= Phase_B_data;
355       else
356         Phase_B_ADC_internal <= Phase_B_ADC_internal;
357       end if;
358
359     end if;
360   end process;
361
362   avalon_register_read : process (clk, avs_address, avs_read)
```

```
363   begin
364     if rising_edge(clk) and avs_read = '1' then
365       case avs_address is
366         when std_logic_vector(to_unsigned(0, 6)) =>
367           avs_readdata <= "0000000000000000" & Position_Target_internal;
368         when std_logic_vector(to_unsigned(1, 6)) =>
369           avs_readdata <= "0000000000000000" & Position_KP_internal;
370         when std_logic_vector(to_unsigned(2, 6)) =>
371           avs_readdata <= "0000000000000000" & Position_KI_internal;
372         when std_logic_vector(to_unsigned(3, 6)) =>
373           avs_readdata <= "0000000000000000" & Position_KD_internal;
374         when std_logic_vector(to_unsigned(4, 6)) =>
375           avs_readdata <= "0000000000000000" &
                Position_Filter_Gain_internal;
376         when std_logic_vector(to_unsigned(5, 6)) =>
377           avs_readdata <= "0000000000000000" & Speed_KP_internal;
378         when std_logic_vector(to_unsigned(6, 6)) =>
379           avs_readdata <= "0000000000000000" & Speed_KI_internal;
380         when std_logic_vector(to_unsigned(7, 6)) =>
381           avs_readdata <= "0000000000000000" & Speed_KD_internal;
382         when std_logic_vector(to_unsigned(8, 6)) =>
383           avs_readdata <= "0000000000000000" & Speed_Filter_Gain_internal;
384         when std_logic_vector(to_unsigned(9, 6)) =>
385           avs_readdata <= "0000000000000000" & Direct_Current_KP_internal;
386         when std_logic_vector(to_unsigned(10, 6)) =>
387           avs_readdata <= "0000000000000000" & Direct_Current_KI_internal;
388         when std_logic_vector(to_unsigned(11, 6)) =>
389           avs_readdata <= "0000000000000000" &
                Quadrature_Current_KP_internal;
390         when std_logic_vector(to_unsigned(12, 6)) =>
391           avs_readdata <= "0000000000000000" &
                Quadrature_Current_KI_internal;
392         when std_logic_vector(to_unsigned(13, 6)) =>
393           avs_readdata(15 downto 0) <= Phase_A_ADC_internal(15 downto 0);
394         when std_logic_vector(to_unsigned(14, 6)) =>
395           avs_readdata(15 downto 0) <= Phase_B_ADC_internal(15 downto 0);
396         when std_logic_vector(to_unsigned(15, 6)) =>
397           avs_readdata <= Phase_A_Duty_cycle_internal;
398         when std_logic_vector(to_unsigned(16, 6)) =>
399           avs_readdata <= Phase_B_Duty_Cycle_internal;
400         when std_logic_vector(to_unsigned(17, 6)) =>
401           avs_readdata <= id_ref_internal;
402         when std_logic_vector(to_unsigned(18, 6)) =>
403           avs_readdata <= Position_Feedback_internal;
404         when std_logic_vector(to_unsigned(19, 6)) =>
405           avs_readdata <= std_logic_vector(Global_Period);
406         when std_logic_vector(to_unsigned(20, 6)) =>
407           avs_readdata <= Quadratrue_Current_internal;
408         when std_logic_vector(to_unsigned(21, 6)) =>
409           avs_readdata <= Position_PID_Out_internal;
410         when std_logic_vector(to_unsigned(22, 6)) =>
```

```
411           avs_readdata <= Speed_PID_Out_internal;
412         when std_logic_vector(to_unsigned(23, 6)) =>
413           avs_readdata <= Quadrature_Voltage_Target_internal;
414           when std_logic_vector(to_unsigned(24, 6)) =>
415           avs_readdata <= Phase_A_Current_feedback_internal;
416           when std_logic_vector(to_unsigned(25, 6)) =>
417           avs_readdata <= Phase_B_Current_feedback_internal;
418         when others             =>
419           avs_readdata <= (others => '0');
420       end case;
421     end if;
422   end process avalon_register_read;
423
424   avalon_register_write : process (clk, reset, avs_address, avs_write)
425   begin
426     if reset = '1' then
427       Global_Period                 <= "00000000000010000000000000000000"
          ;
428       Position_Target_internal      <= "0000000000000000";
429       Position_KP_internal          <= "0000001000000000";
430       Position_KI_internal          <= "0000000000000000";
431       Position_KD_internal          <= "0000000000000000";
432       Position_Filter_Gain_internal <= "0000000000000000";
433       Speed_KP_internal             <= "0000001000000000";
434       Speed_KI_internal             <= "0000000000000000";
435       Speed_KD_internal             <= "0000000000000000";
436       Speed_Filter_Gain_internal    <= "0000000000000000";
437       Quadrature_Current_KP_internal <= "0000001000000000";
438       Quadrature_Current_KI_internal <= "0000000000000000";
439       Direct_Current_KP_internal    <= "0000000000000000";
440       Direct_Current_KI_internal    <= "0000000000000000";
441
442     elsif rising_edge(clk) and avs_write = '1' then
443       case avs_address is
444         when std_logic_vector(to_unsigned(0, 6)) =>
445           Position_Target_internal <= avs_writedata(15 downto 0);
446         when std_logic_vector(to_unsigned(1, 6)) =>
447           Position_KP_internal <= avs_writedata(15 downto 0);
448         when std_logic_vector(to_unsigned(2, 6)) =>
449           Position_KI_internal <= avs_writedata(15 downto 0);
450         when std_logic_vector(to_unsigned(3, 6)) =>
451           Position_KD_internal <= avs_writedata(15 downto 0);
452         when std_logic_vector(to_unsigned(4, 6)) =>
453           Position_Filter_Gain_internal <= avs_writedata(15 downto 0);
454         when std_logic_vector(to_unsigned(5, 6)) =>
455           Speed_KP_internal <= avs_writedata(15 downto 0);
456         when std_logic_vector(to_unsigned(6, 6)) =>
457           Speed_KI_internal <= avs_writedata(15 downto 0);
458         when std_logic_vector(to_unsigned(7, 6)) =>
459           Speed_KD_internal <= avs_writedata(15 downto 0);
460         when std_logic_vector(to_unsigned(8, 6)) =>
```

```
461           Speed_Filter_Gain_internal <= avs_writedata(15 downto 0);
462         when std_logic_vector(to_unsigned(9, 6)) =>
463           Direct_Current_KP_internal <= avs_writedata(15 downto 0);
464         when std_logic_vector(to_unsigned(10, 6)) =>
465           Direct_Current_KI_internal <= avs_writedata(15 downto 0);
466         when std_logic_vector(to_unsigned(11, 6)) =>
467           Quadrature_Current_KP_internal <= avs_writedata(15 downto 0);
468         when std_logic_vector(to_unsigned(12, 6)) =>
469           Quadrature_Current_KI_internal <= avs_writedata(15 downto 0);
470         when std_logic_vector(to_unsigned(17, 6)) =>
471           id_ref_internal <= avs_writedata;
472         when std_logic_vector(to_unsigned(18, 6)) =>
473           encoder_zero_internal <= '1';
474         when std_logic_vector(to_unsigned(19, 6)) =>
475           Global_Period <= unsigned(avs_writedata);
476         when others =>
477           null;
478       end case;
479       if (encoder_zeroed_internal = '1') then
480         encoder_zero_internal <= '0';
481       else
482         encoder_zero_internal <= encoder_zero_internal;
483       end if;
484     end if;
485   end process avalon_register_write;
486 end architecture;
```

Encoder Interface

The encoder interface provides quadrature position counting and zeroing functionality for mechanical position feedback.

Zeroable Encoder Counter

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4  entity encoder_counter_zeroable is
5    port (
6      clk            : in std_ulogic;
7      rst            : in std_ulogic;
8      A              : in std_ulogic;
9      B              : in std_ulogic;
10     zero           : in std_ulogic;
11     zero_confirmed : out std_ulogic;
12     counts         : out integer := 0
13   );
14 end entity;
15
```

```
16 architecture encoder_counter_zeroable_arch of encoder_counter_zeroable is
17
18   -- Signals -------------------------------
19   signal A_hold         : std_ulogic                     := '0';
20   signal B_hold         : std_ulogic                     := '0';
21   signal A_sync         : std_ulogic                     := '0';
22   signal B_sync         : STD_ULOGIC                     := '0';
23   signal zero_hold      : STD_ULOGIC                     := '0';
24   signal zero_sync      : STD_ULOGIC                     := '0';
25   signal current_input  : std_ulogic_vector(1 downto 0) := "00";
26   signal previous_input : std_ulogic_vector(1 downto 0) := "00";
27
28   signal counts_internal : integer := 0;
29
30   -- End of Signals -------------------------
31 begin
32   current_input <= A_sync & B_sync; -- Combine Synchonized Inputs
33   counts        <= counts_internal;
34
35   synchronizer : process (clk, rst, A, B, zero)
36   begin
37     if rst = '1' then
38       A_hold    <= '0';
39       B_hold    <= '0';
40       A_sync    <= '0';
41       B_sync    <= '0';
42       zero_hold <= '0';
43       zero_sync <= '0';
44
45     elsif (rising_edge(clk)) then
46       A_hold    <= A;
47       B_hold    <= B;
48       A_sync    <= A_hold;
49       B_sync    <= B_hold;
50       zero_hold <= zero;
51       zero_sync <= zero_hold;
52     end if;
53   end process;
54   counter : process (clk, rst, A_sync, B_sync)
55   begin
56     if rst = '1' then
57       previous_input  <= "XX";
58       counts_internal <= 0;
59
60     elsif (rising_edge(clk)) then
61       if zero_sync = '1' then
62         counts_internal <= 0;
63         zero_confirmed  <= '1';
64       else
65         zero_confirmed <= '0';
66       end if;
```

```
67
68        case previous_input is
69          when "00" =>
70            if current_input = "10" then
71              counts_internal <= counts_internal + 1;
72            elsif current_input = "01" then
73              counts_internal <= counts_internal - 1;
74            else
75              null;
76            end if;
77
78          when "01" =>
79            if current_input = "11" then
80              counts_internal <= counts_internal - 1;
81            elsif current_input = "00" then
82              counts_internal <= counts_internal + 1;
83            else
84              null;
85            end if;
86
87          when "10" =>
88            if current_input = "11" then
89              counts_internal <= counts_internal + 1;
90            elsif current_input = "00" then
91              counts_internal <= counts_internal - 1;
92            else
93              null;
94            end if;
95
96          when "11" =>
97            if current_input = "01" then
98              counts_internal <= counts_internal + 1;
99            elsif current_input = "10" then
100             counts_internal <= counts_internal - 1;
101           else
102             null;
103           end if;
104         when others =>
105           null;
106       end case;
107       previous_input <= current_input;
108     end if;
109   end process;
110 end architecture encoder_counter_zeroable_arch;
```

Encoder Counter

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4 entity encoder_counter is
```

```
5   port (
6     clk     : in std_ulogic;
7     rst     : in std_ulogic;
8     A       : in std_ulogic;
9     B       : in std_ulogic;
10    counts : out integer := 0
11  );
12 end entity;
13
14 architecture encoder_counter_arch of encoder_counter is
15
16   -- Signals -------------------------------
17   signal A_hold : std_ulogic := '0';
18   signal B_hold : std_ulogic := '0';
19   signal A_sync : std_ulogic := '0';
20   signal B_sync : STD_ULOGIC := '0';
21
22   signal current_input  : std_ulogic_vector(1 downto 0) := "00";
23   signal previous_input : std_ulogic_vector(1 downto 0) := "00";
24
25   signal counts_internal : integer := 0;
26
27   -- End of Signals ------------------------
28 begin
29   current_input <= A_sync & B_sync; -- Combine Synchonized Inputs
30   counts        <= counts_internal;
31
32   synchronizer : process (clk, rst, A, B)
33   begin
34     if rst = '1' then
35       A_hold <= '0';
36       B_hold <= '0';
37       A_sync <= '0';
38       B_sync <= '0';
39     elsif (rising_edge(clk)) then
40       A_hold <= A;
41       B_hold <= B;
42       A_sync <= A_hold;
43       B_sync <= B_hold;
44     end if;
45   end process;
46   counter : process (clk, rst, A_sync, B_sync)
47   begin
48     if rst = '1' then
49       previous_input  <= "XX";
50       counts_internal <= 0;
51
52     elsif (rising_edge(clk)) then
53       case previous_input is
54         when "00" =>
55           if current_input = "10" then
```

```
            counts_internal <= counts_internal + 1;
          elsif current_input = "01" then
            counts_internal <= counts_internal - 1;
          else
            null;
          end if;

        when "01" =>
          if current_input = "11" then
            counts_internal <= counts_internal - 1;
          elsif current_input = "00" then
            counts_internal <= counts_internal + 1;
          else
            null;
          end if;

        when "10" =>
          if current_input = "11" then
            counts_internal <= counts_internal + 1;
          elsif current_input = "00" then
            counts_internal <= counts_internal - 1;
          else
            null;
          end if;

        when "11" =>
          if current_input = "01" then
            counts_internal <= counts_internal + 1;
          elsif current_input = "10" then
            counts_internal <= counts_internal - 1;
          else
            null;
          end if;
        when others =>
          null;
      end case;
      previous_input <= current_input;
    end if;
  end process;
end architecture encoder_counter_arch;
```

ADC Interface

This module handles communication with the external 16-bit ADC used for motor phase current sensing.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity adc_interface_16bit is
```

```
5   generic (
6     sys_clock : time := 20 ns
7   );
8   port (
9     clk         : in std_logic;
10    rst         : in std_logic;
11    enable      : in std_logic := '0';
12    SDO         : in std_logic;
13    CONVST      : out std_logic := '0';
14    SCK         : out std_logic;
15    data        : out std_logic_vector(15 downto 0) := "0000000000000000";
16    data_ready : out std_logic
17  );
18 end entity;
19
20 architecture adc_interface_16bit_arch of adc_interface_16bit is
21
22   -- Signals ------------------------
23
24   signal counts           : integer                        := 0;
25   signal counts_unsigned  : unsigned (6 downto 0)          := "0000000";
26   signal sck_edge_counter : integer                        := 0;
27   signal SCK_internal     : std_logic                      := '0';
28   signal CONVST_internal  : std_logic                      := '0';
29   signal stored_SDO       : std_logic_vector(15 downto 0) := "
       0000000000000000";
30   -- End Signals --------------------
31
32 begin
33
34   counts_unsigned <= to_unsigned(counts, 7);
35   SCK             <= SCK_internal;
36   Counter_1 : process (clk, rst, counts, enable)
37   begin
38     if (rst = '1' or enable = '0') then
39       counts <= 0;
40     elsif (rising_edge(clk)) then
41       if (counts >= 108) then
42         counts <= 0;
43       else
44         counts <= counts + 1;
45       end if;
46     end if;
47   end process;
48
49   CONVST_Pulse_gen : process (clk, rst, counts, enable)
50   begin
51     if (rst = '1' or enable = '0') then
52       CONVST_internal <= '0';
53     elsif (rising_edge(clk)) then
54       if (counts < 75) then
```

```
55          CONVST <= '1';
56        else
57          CONVST <= '0';
58        end if;
59      end if;
60    end process;
61
62    SCK_Pulse_gen : process (clk, rst, enable)
63    begin
64      if (rst = '1' or enable = '0') then
65        SCK_internal <= '0';
66      elsif (rising_edge(clk)) then
67        if sck_edge_counter < 32 and counts > 75 then
68          sck_edge_counter <= sck_edge_counter + 1;
69          SCK_internal     <= not SCK_internal;
70        elsif (counts < 75) then
71          SCK_internal     <= '0';
72          sck_edge_counter <= 0;
73        end if;
74      end if;
75    end process;
76    Store_SDO_data : process (clk, rst, SCK_internal, sck_edge_counter,
        enable)
77    begin
78      if (rst = '1' or enable = '0') then
79        stored_SDO <= "0000000000000000";
80        data_ready <= '0';
81      elsif (rising_edge(clk)) then
82        case(sck_edge_counter) is
83          when 0 =>
84          data_ready     <= '0';
85          stored_SDO(15) <= SDO;
86          when 2 =>
87          stored_SDO(14) <= SDO;
88          when 4 =>
89          stored_SDO(13) <= SDO;
90          when 6 =>
91          stored_SDO(12) <= SDO;
92          when 8 =>
93          stored_SDO(11) <= SDO;
94          when 10 =>
95          stored_SDO(10) <= SDO;
96          when 12 =>
97          stored_SDO(9) <= SDO;
98          when 14 =>
99          stored_SDO(8) <= SDO;
100         when 16 =>
101         stored_SDO(7) <= SDO;
102         when 18 =>
103         stored_SDO(6) <= SDO;
104         when 20 =>
```

```
105         stored_SDO(5) <= SDO;
106         when 22 =>
107         stored_SDO(4) <= SDO;
108         when 24 =>
109         stored_SDO(3) <= SDO;
110         when 26 =>
111         stored_SDO(2) <= SDO;
112         when 28 =>
113         stored_SDO(1) <= SDO;
114         when 30 =>
115         stored_SDO(0) <= SDO;
116         data          <= stored_SDO;
117         data_ready    <= '1';
118         when 32 =>
119         data_ready <= '0';
120         when others =>
121         stored_SDO <= stored_SDO;
122         null;
123       end case;
124     end if;
125   end process;
126 end architecture;
```

PWM Generation

The PWM subsystem converts commanded duty-cycle values into switching signals for the motor driver. The full bridge driver uses an internal PWM controller subcomponent.

Full Bridge Single Output Driver

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4  entity Full_bridge_single_output_driver is
5    generic (
6      W_PERIOD : integer := 30
7    );
8    port (
9      clk : in std_logic;
10     rst : in std_logic;
11     -- PWM repetition period in milliseconds;
12     -- datatype (W.F) is individually assigned
13     period : in unsigned(W_PERIOD - 1 downto 0);
14     -- PWM duty cycle between [0 1]; out-of-range values are hard limited
15     -- datatype (W.F) is individually assigned;
16     duty_cycle : in integer;
17     enable     : out std_logic;
18     direction  : out std_logic
19   );
```

```
20 end entity Full_bridge_single_output_driver;
21
22 architecture Full_bridge_single_output_driver_arch of
      Full_bridge_single_output_driver is
23   signal internal_duty_cycle : std_logic_vector(19 downto 0) := "
       00000000000000000000";
24   signal pwm_output          : std_logic                      := '0';
25   component pwm_controller is
26     generic (
27       CLK_PERIOD    : time    := 20 ns;
28       W_PERIOD      : integer := 30;
29       W_DUTY_CYCLE : integer := 19
30     );
31     port (
32       clk         : in std_logic;
33       rst         : in std_logic;
34       period      : in unsigned(W_PERIOD - 1 downto 0);
35       duty_cycle : in std_logic_vector(W_DUTY_CYCLE - 1 downto 0);
36       output      : out std_logic
37     );
38   end component;
39
40 begin
41   pwm_driver : pwm_controller
42   generic map(
43     CLK_PERIOD    => 20 ns,
44     W_PERIOD      => 30,
45     W_DUTY_CYCLE => 19
46   )
47   port map(
48     clk         => clk,
49     rst         => rst,
50     period      => period,
51     duty_cycle => internal_duty_cycle(18 downto 0),
52     output      => enable
53   );
54
55   process (clk, rst, duty_cycle)
56   begin
57     if (duty_cycle < 0) then
58       internal_duty_cycle <= std_logic_vector(to_unsigned(duty_cycle *
           (-1), 20));
59       direction           <= '0';
60     else
61       internal_duty_cycle <= std_logic_vector(to_unsigned(duty_cycle, 20))
           ;
62       direction           <= '1';
63
64     end if;
65   end process;
66 end architecture;
```

PWM Controller

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity pwm_controller is
  generic (
    CLK_PERIOD    : time    := 20 ns;
    W_PERIOD      : integer := 30;
    W_DUTY_CYCLE : integer := 19
  );
  port (
    clk : in std_logic;
    rst : in std_logic;
    -- PWM repetition period in milliseconds;
    -- datatype (W.F) is individually assigned
    period : in unsigned(W_PERIOD - 1 downto 0);
    -- PWM duty cycle between [0 1]; out-of-range values are hard limited
    -- datatype (W.F) is individually assigned;
    duty_cycle : in std_logic_vector(W_DUTY_CYCLE - 1 downto 0);
    output     : out std_logic
  );
end entity pwm_controller;

architecture pwm_controller_arch of pwm_controller is
  constant counts_per_ms          : integer := 1 ms / clk_period; -- clock cycles / ms
  signal duty_cycle_time          : unsigned(48 downto 0); -- W.F format = 49.42
  signal PERIOD_LIMIT_TOTAL       : unsigned(45 downto 0);
  signal PERIOD_LIMIT_TOTAL_INT   : integer := 0;
  signal DUTY_CYCLE_LIMIT_TOT     : unsigned(64 downto 0);
  signal DUTY_CYCLE_LIMIT_TOT_INT : integer := 0;
  signal count                    : integer := 0;
begin

  process (clk, period, duty_cycle)
  begin
    PERIOD_LIMIT_TOTAL       <= period * to_unsigned(counts_per_ms, 16);
    PERIOD_LIMIT_TOTAL_INT   <= to_integer(PERIOD_LIMIT_TOTAL(45 downto 24));
    DUTY_CYCLE_LIMIT_TOT     <= period * unsigned(duty_cycle) * to_unsigned(counts_per_ms, 16);
    DUTY_CYCLE_LIMIT_TOT_INT <= to_integer(DUTY_CYCLE_LIMIT_TOT(64 downto 42));
  end process;

  process (clk, rst)
  begin
    if rst = '1' then
      count  <= 0;
      output <= '0';
```

```
    elsif (rising_edge(clk)) then
      count <= count + 1;
      if (duty_cycle(18) = '1') then
        output       <= '1';
      elsif (count <= DUTY_CYCLE_LIMIT_TOT_INT) then
        output       <= '1';
      else
        output <= '0';
      end if;
      if (count = PERIOD_LIMIT_TOTAL_INT) then
        count <= 0;
      else
      end if;
    end if;
  end process;
end architecture;
```

Electrical Angle Calibration

This module determines and applies the electrical angle offset required for correct field-oriented control operation.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Stepper_Motor_Electrical_Angle_Calibration is
  port (
    clk         : in std_logic;
    rst         : in std_logic;
    user_button : in std_logic;
    enable      : in std_logic;

    -- Outputs
    Phase_A_Duty_Cycle : out integer := 0;
    Phase_B_Duty_Cycle : out integer := 0;
    Phase_Count : out std_logic_vector(1 downto 0) := "00";
    zero        : out std_logic := '0'
  );
end entity;

architecture Stepper_Motor_Electrical_Angle_Calibration_arch of
   Stepper_Motor_Electrical_Angle_Calibration is
  -- Internal signals
  signal user_button_hold : std_logic := '0';
  signal user_button_sync : std_logic := '0';
  signal counter_enable   : std_logic := '0';
  signal counts           : integer   := 0;
  signal button_pressed   : std_logic := '0';

```

```
28    constant global_duty_cycle  : integer                       := 131072; --
          Should be approximatly 50% duty
29    signal phase_count_internal : std_logic_vector(1 downto 0) := "00";
30
31  begin
32
33    Phase_Count <= phase_count_internal;
34    synchronizer : process (clk, rst, user_button, enable)
35    begin
36      if (rst = '1' or enable = '0') then
37        user_button_hold <= '0';
38        user_button_sync <= '0';
39      elsif rising_edge(clk) then
40        user_button_hold <= user_button;
41        user_button_sync <= user_button_hold;
42
43      end if;
44    end process;
45
46    State_transition : process (clk, rst, counts, counter_enable,
          user_button_sync)
47    begin
48      if (rst = '1') then
49        counts <= 0;
50      elsif (rising_edge(clk)) then
51        if (counts >= 25000000) then
52          counts          <= 0;
53          counter_enable <= '0';
54        elsif (counter_enable = '1') then
55          counts <= counts + 1;
56        end if;
57
58        if (user_button_sync = '1' and counter_enable = '0') then
59          case(phase_count_internal) is
60            when "00" =>
61            phase_count_internal <= "01";
62            counter_enable       <= '1';
63            zero                 <= '0';
64            when "01" =>
65            phase_count_internal <= "10";
66            counter_enable       <= '1';
67            zero                 <= '0';
68            when "10" =>
69            phase_count_internal <= "11";
70            counter_enable       <= '1';
71            zero                 <= '0';
72            when "11" =>
73            phase_count_internal <= "00";
74            counter_enable       <= '1';
75            zero                 <= '1';
76            when others =>
```

```
77            counter_enable       <= counter_enable;
78            phase_count_internal <= phase_count_internal;
79            zero                 <= '0';
80          end case;
81        end if;
82      end if;
83    end process;
84
85    output_determination : process (phase_count_internal, enable)
86    begin
87      case phase_count_internal is
88        when "00" =>
89          if (enable = '0') then
90            Phase_A_Duty_Cycle <= 0;
91            Phase_B_Duty_Cycle <= 0;
92          else
93            Phase_B_Duty_Cycle <= 0;
94            Phase_A_Duty_Cycle <= global_duty_cycle;
95          end if;
96
97        when "01" =>
98          if (enable = '0') then
99            Phase_A_Duty_Cycle <= 0;
100           Phase_B_Duty_Cycle <= 0;
101         else
102           Phase_A_Duty_Cycle <= 0;
103           Phase_B_Duty_Cycle <= global_duty_cycle;
104         end if;
105       when "10" =>
106         if (enable = '0') then
107           Phase_A_Duty_Cycle <= 0;
108           Phase_B_Duty_Cycle <= 0;
109         else
110           Phase_B_Duty_Cycle <= 0;
111           Phase_A_Duty_Cycle <= global_duty_cycle * (-1);
112         end if;
113       when "11" =>
114         if (enable = '0') then
115           Phase_A_Duty_Cycle <= 0;
116           Phase_B_Duty_Cycle <= 0;
117         else
118           Phase_A_Duty_Cycle <= 0;
119           Phase_B_Duty_Cycle <= global_duty_cycle * (-1);
120         end if;
121       when others =>
122                 null;
123
124     end case;
125   end process;
126 end architecture;
```